

Table des matières

Introduction Generale	1
Chapitre 1 : Contexte et étude de l'existant	3
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.2 Contexte et problématique	4
1.2.1 Variabilité des équipements informatiques	4
1.2.2 Contraintes organisationnelles et techniques	5
1.3 Analyse des solutions existantes	5
1.3.1 Présentation des solutions existantes	5
1.3.2 Comparaison et limites	7
1.4 Solution proposée : Parc Informatique FMM	8
1.4.1 Architecture et fonctionnement	9
1.4.2 Apports de la solution	9
Conclusion	9
Chapitre 2 : Méthodologie et spécifications des besoins	10
Introduction	11
2.1 Analyse et spécification des besoins	11
2.1.1 Identification des acteurs	11
2.1.2 Spécification des besoins	11
2.2 Choix de la méthodologie	13
2.2.1 Étude comparative des méthodologies	13
2.2.2 Méthodologie retenue : Scrum	14
2.3 Présentation de Scrum	15
2.3.1 Artefacts Scrum et leur application	16
2.3.2 Événements Scrum et leur implémentation	16

2.4	Les rôles Scrum	17
2.5	Outils de support à Scrum	17
	Conclusion	18
Chapitre 3 : Étude architecturale du projet		19
	Introduction	20
3.1	Présentation de l'architecture générale	20
3.2	Modèle de conception logicielle adoptée	21
3.2.1	Hiérarchie organisationnelle et modèle de localisation	21
3.3	Interactions et fonctionnement du système	22
3.3.1	Diagramme des cas d'utilisation	22
3.3.2	Diagramme de classes	22
3.3.3	Diagrammes de séquence	23
3.3.4	Diagramme d'activités	27
3.4	Pourquoi ce choix d'architecture	28
	Conclusion	29
Chapitre 4 : Planification et préparation du projet Parc Informatique FMM		30
	Introduction	31
4.1	Pilotage du projet avec Scrum	31
4.1.1	Acteurs Scrum et Missions	31
4.1.2	Organisation du Backlog	31
4.1.3	Planification des sprints	32
4.2	Environnement de travail	34
4.2.1	Environnement matériel	34
4.2.2	Environnement de développement	35
4.2.3	Environnement logiciel	36
	Conclusion	38
Chapitre 5 : Mise en place de l'architecture et gestion des équipements		39
	Introduction	40

5.1	Objectifs et livrables	41
5.2	Architecture et modélisation UML du Sprint 1	41
5.2.1	Clarification terminologique : Assignment et Affectation	43
5.2.2	Diagramme de classes Sprint 1	43
5.2.3	Diagrammes de séquence Sprint 1	43
5.3	Tâches principales	47
5.3.1	Configuration du serveur backend	47
5.3.2	Création des schémas de base de données	48
5.3.3	Développement de l'API REST et Architecture des Endpoints	49
5.3.4	Mécanismes Transverses : Sécurité, Validation et Cycle de Vie	50
5.4	Critères d'acceptation	51
5.5	Implémentation et résultats	53
5.6	Tests et validation	54
5.7	Défis et solutions	54
5.8	Planification pour le sprint suivant	55
	Conclusion	55
Chapitre 6 : Gestion des tickets et maintenance		56
	Introduction	57
6.1	Objectifs du Sprint 2	57
6.2	Architecture et modélisation UML du Sprint 2	58
6.2.1	Diagramme de classes Sprint 2	58
6.2.2	Diagrammes de séquence Sprint 2	59
6.3	Livrables attendus	65
6.4	Tâches principales	65
6.4.1	Qualité et tests	65
6.4.2	Notifications et communication	65
6.4.3	Automatisation des workflows	68
6.4.4	Optimisations et performance	68
6.4.5	Déploiement et résilience	68

6.5	Critères d'acceptation	69
6.6	Implémentation et résultats	69
6.7	Tests et validation	69
6.8	Défis et solutions	70
6.9	Planification pour le sprint suivant	70
	Conclusion	70
Chapitre 7 : Notifications, tableaux de bord et reporting		71
	Introduction	72
7.1	Objectifs du Sprint 3	72
7.2	Architecture et modélisation UML du Sprint 3	72
7.2.1	Diagramme de classes Sprint 3	73
7.2.2	Diagrammes de séquence Sprint 3	74
7.3	Tableaux de bord et reporting Web	79
7.4	Optimisation des performances	79
	Conclusion	80
Conclusion et perspectives		81
Bibliographie		82

Remerciements

« La reconnaissance est non seulement la plus grande des vertus, mais aussi la mère de toutes les autres. » — Cicéron

C'est avec un grand honneur et un grand plaisir que je dédie ces quelques lignes afin de remercier toute personne ayant contribué de près ou de loin à l'accomplissement de ce projet.

À Monsieur Ridha A.F.S.I pour son encadrement, pour son aide, sa patience, ses consignes et ses recommandations qui ont été d'un grand intérêt et qui m'ont permis de mener à terme ce projet.

À Monsieur Charfeddine A.M.R.I, le Doyen de la Faculté de Médecine, pour l'aide, le support, les précieux conseils, et de m'avoir accueilli chaleureusement.

À mon encadrante professionnel Racha F.S.B.I, pour son assistance, son soutien, ses conseils pertinents et son encadrement direct et amical durant ce stage.

Je suis très reconnaissante à toutes les personnes qui ont contribué à l'élaboration de ce travail à savoir Monsieur Ridha A.F.S.I, monsieur Adnène R.O.U.A.T.B.I, monsieur Hatem S.A.N.D.I.D, Monsieur Wissem L.T.A.I.E.F, monsieur Hichem M.S.E.K, pour leur soutien moral, leur encouragement et leur conseil et pour tout le temps qu'ils ont sacrifié.

Dédicaces

Je dédie ce travail :

À maman, pour son amour, son sacrifice, son soutien et son dévouement. Il n'y a pas de mots qui puissent exprimer toute ma joie, ma gratitude et ma reconnaissance pour tout ce qu'elle fait pour moi. Que Dieu lui accorde santé, bonheur et longue vie.

À tous les membres de ma famille, mon Mari et Mes enfants, pour toute leur tendresse, leur considération et leur soutien permanent. Je leur dois énormément de choses merveilleuses.

À mes chères amies, pour tous les merveilleux moments et les souvenirs partagés.

À toute l'équipe de la Faculté de Médecine de Monastir, qui m'a encouragée et soutenue durant la période de ce stage. Je suis reconnaissante envers tous ceux que j'aime et à tous ceux qui m'aiment.

*Hela Boussaid
Année universitaire 2025-2026*

Table des figures

1.1	Logo de la FMM	4
1.2	Logo du SmartLab	4
1.3	Logo de GLPI	6
1.4	Logo de iTop	6
1.5	Logo de Asset Panda	7
2.6	Processus Scrum.	15
2.7	Rôles Scrum.	17
2.8	Logo de ClickUp.	18
2.9	Logo de Slack.	18
2.10	Logo de Git.	18
3.11	Architecture globale du Parc Informatique FMM	20
3.12	Modèle MVC du Parc Informatique FMM	21
3.13	Diagramme des cas d'utilisation global	22
3.14	Diagramme de classes du Parc Informatique FMM	23
3.15	Diagramme de séquence d'authentification	24
3.16	Diagramme de séquence de création et assignation d'un ticket	25
3.17	Diagramme de séquence de traitement et mise à jour d'un ticket	26
3.18	Diagramme de séquence de clôture et consultation de l'historique d'un ticket	27
3.19	Diagramme d'activité du flux des processus	28
5.20	Architecture technique multicouche du Sprint 1 — Flux et composants du système	40
5.21	Diagramme de classes du Sprint 1 — Gestion des équipements et des utilisateurs	43
5.22	Diagramme de séquence d'authentification utilisateur	44
5.23	Diagramme de séquence de création d'un équipement	45
5.24	Diagramme de séquence d'affectation d'un équipement	46
5.25	Diagramme de séquence de consultation de l'inventaire	46
5.26	Diagramme de séquence de consultation des détails d'un équipement	47

5.27	Diagramme entité–relation de la base MySQL	48
5.28	Interface Swagger UI des API du Sprint 1	54
6.29	Architecture améliorée du Sprint 2 — tests, notifications et optimisations	57
6.30	Diagramme de classes du Sprint 2	59
6.31	Diagramme de séquence de création d’un ticket	60
6.32	Diagramme de séquence d’affectation automatique d’un ticket	61
6.33	Diagramme de séquence de consultation des tickets assignés	62
6.34	Diagramme de séquence de traitement d’un ticket	62
6.35	Diagramme de séquence de création d’une intervention	63
6.36	Diagramme de séquence de clôture d’un ticket	64
6.37	Diagramme de séquence de consultation de l’historique d’un ticket	64
6.38	Architecture des notifications Firebase FCM et événements métier	66
6.39	Diagramme de séquence de gestion d’un ticket d’incident	66
7.40	Diagramme de classes du Sprint 3	74
7.41	Diagramme de séquence d’envoi d’une notification lors de la création d’un ticket	75
7.42	Diagramme de séquence de notification lors d’un changement de statut	76
7.43	Diagramme de séquence de consultation du tableau de bord	77
7.44	Diagramme de séquence de génération d’un rapport	77
7.45	Diagramme de séquence d’export des statistiques	78
7.46	Diagramme de séquence de consultation des indicateurs du parc	79
7.47	Page d’accueil	79
7.48	Connexion	79
7.49	Accueil	79
7.50	Liste des équipements	79
7.51	Détails de l’équipement	79
7.52	Notifications d’intervention	79
7.53	Créer une intervention	80
7.54	Détails du ticket	80
7.55	Historique de maintenance	80

TABLE DES FIGURES

7.56 Profil utilisateur	80
7.57 Historique des notifications	80
7.58 Calendrier de maintenance préventive	80
7.59 Ajouter un rapport de maintenance	80
7.60 Activer les notifications	80
7.61 Liste détaillée des équipements	80

Liste des tableaux

1.1	Exemples d'équipements informatiques	4
1.2	Comparaison des solutions de gestion du parc informatique	7
1.3	Fonctionnalités principales du système	9
2.4	Acteurs du système et leurs rôles	11
2.5	Tableau comparatif des méthodologies de développement logiciel.	13
4.6	Acteurs du projet Scrum	31
4.7	Backlog du Projet avec Priorités MoSCoW	32
4.8	Planification des Sprints du Projet Parc Informatique FMM	33
5.9	Modèles de données du système Parc Informatique FMM	42
5.10	Spécifications techniques et politiques de sécurité des routes de l'API	50
5.11	Résultats de validation des critères d'acceptation du Sprint 1	54
6.12	Résultats de validation des critères d'acceptation du Sprint 2	69
7.13	Sprint Backlog pour le sprint 3	72

CHAPITRE 0

Introduction Générale

Table des matières

Apports du projet

Le système proposé répond à plusieurs besoins fonctionnels structurants liés à la gestion des parcs informatiques :

- **Utilisateurs finaux** : amélioration de la visibilité sur les équipements informatiques et optimisation de leur cycle de vie.
- **Personnel technique** : réduction des tâches manuelles grâce à la centralisation et à l'automatisation du suivi des incidents.
- **Établissements** : optimisation des ressources, réduction des coûts opérationnels et amélioration de la traçabilité des actifs.

Ainsi, ce projet s'inscrit dans une logique d'optimisation des processus et d'amélioration continue des performances des systèmes d'information.

Structure du rapport

Ce rapport est structuré de manière progressive afin de présenter l'ensemble des étapes du projet.

Le **Chapitre 1** introduit le cadre général du projet et présente une étude de l'existant ainsi que les limites des solutions actuelles.

Le **Chapitre 2** décrit la méthodologie adoptée et les choix technologiques retenus pour la réalisation du système.

Le **Chapitre 3** présente l'architecture globale de la solution et ses principaux composants.

Le **Chapitre 4** détaille l'implémentation des fonctionnalités principales.

Les **Chapitres 5, 6 et 7** présentent les résultats des différents sprints de développement.

Enfin, le **Chapitre 8** conclut le travail et propose des perspectives d'évolution.

À la suite de cette introduction générale, le chapitre suivant est consacré à l'analyse du contexte et à l'étude des solutions existantes, afin d'identifier les limites des systèmes actuels et de justifier les choix de conception adoptés dans ce projet.

CHAPITRE 1

Chapitre 1 : Contexte et étude de l'existant

Table des matières

Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.2 Contexte et problématique	4
1.2.1 Variabilité des équipements informatiques	4
1.2.2 Contraintes organisationnelles et techniques	5
1.3 Analyse des solutions existantes	5
1.3.1 Présentation des solutions existantes	5
1.3.2 Comparaison et limites	7
1.4 Solution proposée : Parc Informatique FMM	8
1.4.1 Architecture et fonctionnement	9
1.4.2 Apports de la solution	9
Conclusion	9

Introduction

Le présent chapitre a pour objectif de présenter le cadre général du projet *Parc Informatique FMM*, réalisé au sein du SmartLab de la Faculté de Médecine de Monastir (FMM)[1]. Il introduit le

contexte organisationnel du projet, met en évidence les problématiques liées à la gestion des parcs informatiques dans les établissements tunisiens, et analyse les solutions existantes sur le marché.

L'ensemble de ce chapitre est structuré comme suit : une présentation de l'organisme d'accueil, une analyse du contexte et des problématiques, une étude des solutions existantes, ainsi qu'une description de la solution proposée.

1.1. Présentation de l'organisme d'accueil

Le projet a été réalisé au sein du SmartLab, une Unité de Service Commun et de Recherche (USCR) rattachée à la Faculté de Médecine de Monastir (FMM), institution universitaire tunisienne fondée en 1980 et reconnue pour la qualité de sa formation et de ses activités de recherche.

La FMM constitue un établissement de référence dans le domaine des sciences médicales en Tunisie. Elle s'inscrit dans une démarche de qualité, attestée notamment par sa certification ISO 21001[2], et vise à assurer une formation académique conforme aux standards internationaux tout en favorisant l'innovation scientifique.

Dans ce cadre, le SmartLab joue un rôle central en tant qu'environnement de recherche et de développement technologique. Il a pour mission de favoriser la collaboration entre chercheurs, enseignants et ingénieurs afin de concevoir des solutions innovantes adaptées aux problématiques réelles du secteur de la santé et des systèmes d'information.

Le SmartLab dispose d'infrastructures techniques modernes, comprenant des équipements informatiques, des serveurs, ainsi que des outils de développement et de test. Il constitue ainsi un environnement propice à la conception et à la validation de solutions numériques dans un contexte académique et professionnel.



FIGURE 1.1 – Logo de la FMM



FIGURE 1.2 – Logo du SmartLab

1.2. Contexte et problématique

Dans de nombreux établissements publics et privés en Tunisie, la gestion des parcs informatiques repose encore sur des approches traditionnelles, souvent semi-manuelles. Ces approches incluent l'utilisation de fichiers Excel, de registres papier ou de systèmes non centralisés pour le suivi des équipements et la gestion des incidents.

Ce mode de fonctionnement engendre plusieurs limitations, notamment en termes de traçabilité, de réactivité et de fiabilité de l'information. Il dépend fortement de l'intervention humaine, ce

qui augmente le risque d'erreurs et ralentit les processus de maintenance et de suivi.

1.2.1. Variabilité des équipements informatiques

Les parcs informatiques sont constitués d'un ensemble hétérogène d'équipements, incluant notamment les postes de travail, les serveurs, ainsi que les licences logicielles. Ces ressources diffèrent selon leur nature, leur durée de vie et leur mode d'acquisition.

À titre d'exemple, un ordinateur portable présente généralement une durée de vie comprise entre trois et cinq ans, tandis que les licences logicielles sont soumises à des contraintes de renouvellement périodique. Cette diversité rend la gestion des actifs particulièrement complexe en l'absence d'un système centralisé.

TABLE 1.1 – Exemples d'équipements informatiques

Catégorie	Exemple	Durée de vie	Mode d'acquisition
Ordinateur portable	Dell Latitude 7420	3 à 5 ans	Achat
Logiciel	Microsoft Office	Selon licence	Abonnement / licence
Serveur	Serveur dédié	5 à 7 ans	Achat / location

1.2.2. Contraintes organisationnelles et techniques

L'analyse du contexte tunisien met en évidence plusieurs contraintes majeures impactant la gestion des parcs informatiques :

- **Sous-effectif technique** : les équipes informatiques sont souvent réduites par rapport au volume d'équipements à gérer, ce qui limite la réactivité en cas d'incident.
- **Absence d'automatisation** : les opérations de suivi et de gestion sont majoritairement manuelles, ce qui augmente la charge de travail et le risque d'erreurs.
- **Hétérogénéité du parc informatique** : la diversité des équipements et des systèmes utilisés complique l'intégration et la standardisation des processus de gestion.

Ces contraintes mettent en évidence la nécessité de mettre en place une solution centralisée, automatisée et adaptée aux ressources disponibles, capable d'améliorer la gestion opérationnelle des équipements informatiques.

1.3. Analyse des solutions existantes

Cette section présente une étude des principales solutions existantes dans le domaine de la gestion des parcs informatiques. Elle vise à analyser leurs fonctionnalités ainsi que leurs limites, afin de positionner la solution proposée par rapport à l'existant.

1.3.1. Présentation des solutions existantes

Plusieurs solutions sont actuellement utilisées pour la gestion des actifs et des services informatiques dans les organisations. Ces solutions permettent principalement l'inventaire des équipe-

ments, la gestion des incidents, ainsi que le suivi des opérations de maintenance.

Parmi les solutions les plus répandues, on retrouve **GLPI**, **iTop** et **Asset Panda**, qui se distinguent par leurs architectures et leurs niveaux de complexité.

- **GLPI** est une solution open-source de gestion des services informatiques (ITSM - IT Service Management) largement adoptée dans les environnements académiques, industriels et administratifs. Elle permet la centralisation et la structuration de la gestion du parc informatique à travers plusieurs modules fonctionnels.

Parmi ses principales fonctionnalités, on distingue la gestion de l'inventaire matériel et logiciel, le suivi des tickets d'incidents, la gestion des demandes utilisateurs, ainsi que la planification des interventions de maintenance. GLPI permet également la gestion des affectations des équipements aux utilisateurs et la traçabilité des changements effectués sur le parc informatique.

L'un de ses principaux atouts réside dans son caractère open-source, qui garantit une grande flexibilité d'adaptation, ainsi que l'existence d'une communauté active assurant des mises à jour régulières et de nombreux plugins d'extension. Cependant, certaines fonctionnalités avancées, notamment la notification en temps réel et l'automatisation poussée des workflows, nécessitent des configurations supplémentaires ou l'ajout de modules externes.[3]



FIGURE 1.3 – Logo de GLPI

- **iTop** est une solution ITSM open-source orientée vers la gestion des services informatiques et la modélisation des infrastructures via une CMDB (Configuration Management Database). Elle se distingue par une approche structurée permettant de représenter les relations entre les différents éléments du système d'information, tels que les équipements, les applications et les services.

iTop intègre des fonctionnalités avancées de gestion des incidents, des problèmes et des changements, conformément aux bonnes pratiques ITIL. Elle permet également la génération de rapports détaillés et la mise en place de processus de validation adaptés aux environnements organisationnels complexes.

Toutefois, la richesse fonctionnelle d'iTop s'accompagne d'une complexité de configuration relativement élevée, nécessitant un temps d'adaptation important ainsi que des compétences techniques avancées pour son déploiement et sa personnalisation.[4]



FIGURE 1.4 – Logo de iTop

- **Asset Panda** est une solution SaaS (Software as a Service) dédiée à la gestion des actifs informatiques et physiques. Elle repose sur une architecture entièrement cloud, permettant un accès centralisé et sécurisé aux données relatives aux équipements.

Cette solution offre des fonctionnalités de suivi en temps réel des actifs, de gestion du cycle de vie des équipements, ainsi que des outils mobiles facilitant les opérations de scan, d'inventaire et de mise à jour sur le terrain. Elle permet également l'intégration avec d'autres systèmes d'information via des API, ce qui renforce son interopérabilité.

Asset Panda est particulièrement adaptée aux organisations recherchant une solution clé en main, sans infrastructure locale complexe. Néanmoins, sa dépendance à une connexion Internet stable ainsi que son modèle basé sur l'abonnement peuvent constituer des limites dans certains contextes, notamment dans les établissements disposant de budgets restreints ou d'infrastructures réseau limitées.[5]



FIGURE 1.5 – Logo de Asset Panda

1.3.2. Comparaison et limites

Afin d'évaluer de manière objective les solutions étudiées, une comparaison multicritère est réalisée dans le Tableau 1.2. Cette comparaison repose sur des critères techniques et organisationnels directement liés aux besoins de gestion d'un parc informatique, notamment l'automatisation des processus, la compatibilité avec différents environnements, la gestion des notifications en temps réel, la facilité de déploiement ainsi que l'efficacité globale du système.

TABLE 1.2 – Comparaison des solutions de gestion du parc informatique

Critère	GLPI	iTop	Asset Panda
Automatisation	±	✓	✓
Compatibilité	✓	±	±
Notifications temps réel	∅	±	✓
Déploiement	✓	±	∅
Efficacité globale	±	✓	✓

Légende : ✓ : optimal, ± : moyen, ∅ : non adapté

Interprétation du tableau : L'analyse du Tableau 1.2 permet de mettre en évidence des différences significatives entre les trois solutions étudiées selon les critères retenus.

En termes d'**automatisation**, les solutions iTop et Asset Panda présentent un niveau avancé grâce à l'intégration de workflows structurés et d'outils de gestion automatisée des processus. À l'inverse, GLPI offre des fonctionnalités d'automatisation plus limitées, nécessitant souvent l'ajout de plugins ou de configurations supplémentaires pour atteindre un niveau équivalent.

Concernant la **compatibilité avec les équipements et environnements hétérogènes**, GLPI se distingue par une meilleure adaptabilité, notamment grâce à sa large communauté et à ses nombreux modules d'extension. iTop et Asset Panda présentent un niveau intermédiaire, bien qu'ils restent compatibles avec la majorité des infrastructures standards.

Pour ce qui est des **notifications en temps réel**, Asset Panda se démarque clairement grâce à son architecture cloud native, permettant une synchronisation et des alertes instantanées. iTop propose un support partiel de ce type de fonctionnalités, tandis que GLPI reste limité dans ce domaine sans intégration externe.

En ce qui concerne la **facilité de déploiement**, GLPI apparaît comme la solution la plus accessible, notamment en raison de sa nature open-source et de sa large documentation. iTop nécessite une phase de configuration plus complexe, tandis qu'Asset Panda, bien que simple à utiliser, impose une dépendance à une infrastructure cloud et à un modèle d'abonnement.

Enfin, l'**efficacité globale** met en évidence un équilibre entre richesse fonctionnelle et complexité d'utilisation. iTop et Asset Panda offrent des performances globalement élevées, mais avec des contraintes différentes, tandis que GLPI présente une solution plus simple mais moins avancée sur certains aspects critiques.

Synthèse critique L'analyse globale des résultats montre que, malgré la maturité technologique de ces solutions, chacune présente des limites structurelles qui restreignent leur adéquation avec certains contextes organisationnels.

GLPI, bien qu'efficace et largement adopté, montre des limites en matière d'automatisation avancée et de gestion des notifications modernes. iTop, quant à lui, offre une grande richesse fonctionnelle mais au prix d'une complexité de mise en œuvre importante. Asset Panda propose une solution moderne et performante, mais son modèle SaaS ainsi que sa dépendance à une connexion Internet stable peuvent constituer des contraintes dans des environnements à ressources limitées.

Conclusion Ainsi, aucune des solutions étudiées ne répond de manière complète et optimale à l'ensemble des besoins identifiés, notamment dans le contexte des établissements tunisiens. Les contraintes liées au coût, à la complexité de déploiement, ainsi qu'à la nécessité d'une meilleure adaptabilité locale justifient le développement d'une solution plus flexible, légère et adaptée aux réalités du terrain.

1.4. Solution proposée : Parc Informatique FMM

Le projet *Parc Informatique FMM* vise à proposer une solution centralisée de gestion des équipements informatiques, adaptée aux contraintes des établissements tunisiens. Cette solution repose sur une architecture web moderne et modulaire, permettant l'automatisation des processus de gestion des actifs et des incidents.

Le système s'appuie sur des technologies open-source telles que **Node.js (Express)**[6], **React**[7] et **MySQL**[8], garantissant ainsi flexibilité, évolutivité et maîtrise des coûts.

1.4.1. Architecture et fonctionnement

L'application est structurée selon une architecture client-serveur. Le backend expose une API REST assurant la gestion des utilisateurs, des équipements et des tickets, ainsi que l'intégration avec un système de notifications en temps réel via Firebase[9].

Le frontend, développé avec React et Material UI[10], permet une interaction fluide avec le système à travers des interfaces dédiées à la gestion des équipements, des incidents et des statistiques.

TABLE 1.3 – Fonctionnalités principales du système

Module	Description
Gestion des actifs	Inventaire, affectation, suivi et génération de QR codes.
Gestion des incidents	Création, affectation et suivi des tickets.
Reporting	Génération de statistiques et export des données.

1.4.2. Apports de la solution

La solution proposée permet d'améliorer significativement la traçabilité des équipements, de réduire les délais de traitement des incidents et d'optimiser la gestion des ressources informatiques. Elle se distingue par sa simplicité de déploiement, son coût réduit et son adaptation au contexte local.

Conclusion

Ce chapitre a permis de présenter le cadre général du projet, d'analyser les contraintes du domaine, ainsi que d'étudier les solutions existantes. Cette analyse met en évidence les limites des systèmes actuels et justifie le développement d'une solution adaptée, qui sera détaillée dans les chapitres suivants.

Le chapitre suivant se concentrera sur la méthodologie adoptée pour la réalisation du projet, en détaillant les étapes de développement, les choix technologiques ainsi que les processus de gestion de projet mis en place pour assurer le succès de l'initiative. Nous aborderons également les défis rencontrés et les solutions mises en œuvre pour garantir une livraison efficace et conforme aux besoins identifiés.

CHAPITRE 2

Chapitre 2 : Méthodologie et spécifications des besoins

Table des matières

Introduction	11
2.1 Analyse et spécification des besoins	11
2.1.1 Identification des acteurs	11
2.1.2 Spécification des besoins	11
2.2 Choix de la méthodologie	13
2.2.1 Étude comparative des méthodologies	13
2.2.2 Méthodologie retenue : Scrum	14
2.3 Présentation de Scrum	15
2.3.1 Artefacts Scrum et leur application	16
2.3.2 Événements Scrum et leur implémentation	16
2.4 Les rôles Scrum	17
2.5 Outils de support à Scrum	17
Conclusion	18

Introduction

Dans ce chapitre, nous présentons la méthodologie adoptée pour le développement du projet **Parc Informatique FMM**, ainsi que les spécifications des besoins fonctionnels et non fonctionnels.

2.1. Analyse et spécification des besoins

L'analyse des besoins est une étape cruciale pour définir les fonctionnalités et les contraintes du système. Elle permet de s'assurer que le projet répond aux attentes des utilisateurs finaux et des responsables informatiques.

2.1.1. Identification des acteurs

Le système interagit avec plusieurs acteurs essentiels, chacun jouant un rôle distinct dans son fonctionnement. Ces acteurs, résumés dans un tableau dédié, correspondent aux besoins de gestion d'un parc informatique au sein du SmartLab de la Faculté de Médecine de Monastir.

TABLE 2.4 – Acteurs du système et leurs rôles

Acteur	Description
Administrateur du parc	Gère les équipements informatiques, les utilisateurs, les affectations et les paramètres du système.
Technicien informatique	Traite les tickets d'incidents, réalise les maintenances et met à jour l'état des équipements.
Responsable du parc informatique	Planifie les maintenances préventives, suit les budgets et contrôle les inventaires.
Utilisateur final	Signale les incidents, consulte le statut de son matériel et effectue des demandes de support.
Système d'inventaire	Base de données et interface qui stocke les équipements, les licences, les historiques et les rapports.

2.1.2. Spécification des besoins

La spécification des besoins est essentielle pour définir les fonctionnalités et les contraintes du système. Elle se divise en deux catégories principales : les besoins fonctionnels et les besoins non fonctionnels.

Besoins fonctionnels :

Le projet **Parc Informatique FMM** a pour principal objectif d'automatiser la gestion du parc informatique, d'optimiser la résolution des incidents et de fournir une interface de suivi efficace. Les besoins fonctionnels traduisent ces objectifs sous forme de fonctionnalités concrètes :

- **Gestion des équipements** : Enregistrement, modification et suivi des actifs informatiques (ordinateurs, serveurs, périphériques, licences).

- **Gestion des tickets** : Création, assignation, suivi et clôture des incidents signalés par les utilisateurs.
- **Affectation des ressources** : Attribution des équipements aux utilisateurs, suivi des emplacements et gestion des transferts.
- **Maintenance préventive et curative** : Planification des interventions, suivi des actions réalisées et génération de notifications pour les échéances.
- **Reporting et tableaux de bord** : Visualisation des indicateurs clés (taux d'incidents, état des équipements, maintenances programmées) et export de rapports.
- **Gestion des utilisateurs et des rôles** : Authentification, autorisation et profils adaptatifs selon les rôles (administrateur, technicien, utilisateur).
- **Notifications** : Envoi d'alertes aux techniciens et aux responsables lors des incidents, des maintenances à venir ou des équipements en anomalie.
- **Import et intégration** : Possibilité d'importer des listes d'équipements et d'intégrer des données existantes via API ou fichiers CSV.

Besoins non fonctionnels :

Les besoins non fonctionnels garantissent la qualité, la robustesse et la conformité du système dans un environnement institutionnel exigeant. Ils s'articulent autour des axes suivants :

- **Disponibilité** : Le système doit être accessible en permanence, avec un temps d'arrêt minimal afin de garantir la continuité du service.
- **Sécurité des données** : Les informations sur les équipements, les utilisateurs et les incidents doivent être protégées par des mécanismes d'authentification, d'autorisation et de chiffrement.
- **Performance** : L'application doit rester réactive même avec un grand nombre d'équipements et de tickets.
- **Interopérabilité** : La solution doit pouvoir s'intégrer avec des outils existants, des bases de données externes et des formats standards (CSV, API REST).

- **Évolutivité** : L’architecture doit permettre d’ajouter de nouvelles fonctionnalités sans re-fonte majeure.
- **Usabilité** : L’interface web doit être intuitive, facilitant la prise en main par les techniciens et les utilisateurs non spécialistes.

2.2. Choix de la méthodologie

Pour le développement du projet Parc Informatique FMM, il existe plusieurs méthodologies de développement logiciel, chacune ayant ses avantages et inconvénients. Parmi les méthodologies les plus courantes, on trouve Scrum, Waterfall, Lean et Kanban.

2.2.1. Étude comparative des méthodologies

Le choix d’une méthodologie de développement constitue une étape déterminante dans la réussite d’un projet logiciel. Il influence directement la qualité du produit final, la capacité d’adaptation aux changements, ainsi que la gestion des risques et des coûts tout au long du cycle de vie du système.

Dans le cadre de ce projet, une comparaison des principales méthodologies de développement logiciel a été réalisée afin d’identifier l’approche la plus adaptée au contexte du projet *Parc Informatique FMM*. Le Tableau 2.5 synthétise cette analyse en se basant sur plusieurs critères essentiels, notamment l’adaptabilité aux changements, la fréquence des livraisons, le niveau de collaboration, la gestion des risques ainsi que la qualité et la maintenabilité du produit.

TABLE 2.5 – Tableau comparatif des méthodologies de développement logiciel.

Caractéristiques	Scrum	Waterfall	Lean	Kanban
Adaptabilité aux changements	✓	X	X	✓
Livraisons fréquentes	✓	X	X	X
Collaboration renforcée	✓	X	X	✓
Réponse rapide aux changements	✓	X	X	X
Gestion des risques	✓	✓	✓	✓
Niveau d’implication du client	Élevé	Faible	Moyen	Moyen
Efficacité des coûts	Élevée	Moyenne	Élevée	Moyenne
Assurance qualité	Continue	Finale	Continue	Continue
Scalabilité	Moyenne	Élevée	Élevée	Moyenne
Maintenance et support	Facile	Difficile	Facile	Facile

L’analyse du Tableau 2.5 met en évidence des différences importantes entre les méthodologies étudiées. Ces différences permettent de comprendre les avantages et limites de chaque approche en fonction du type de projet et du niveau de flexibilité requis.

Les méthodes agiles, notamment **Scrum** et **Kanban**, se distinguent clairement par leur capacité d'adaptation aux changements. Elles permettent une grande flexibilité dans la gestion des besoins, ce qui est particulièrement important dans les projets où les exigences peuvent évoluer au cours du développement. Scrum, en particulier, se caractérise par des livraisons fréquentes et une forte implication du client, ce qui favorise un meilleur alignement entre le produit développé et les besoins réels.

En revanche, la méthodologie **Waterfall** (ou cycle en V) repose sur une approche linéaire et séquentielle. Elle offre une meilleure structuration et une gestion des risques bien définie, mais elle reste peu adaptée aux changements une fois la phase de conception terminée. Cela la rend moins flexible dans des environnements dynamiques.

Les approches **Lean** et **Kanban** se positionnent comme des méthodes intermédiaires, visant principalement à optimiser le flux de travail et à réduire les gaspillages. Elles assurent une amélioration continue du processus, mais leur niveau de structuration reste parfois moins rigide que celui des approches traditionnelles.

En termes de qualité, toutes les méthodologies permettent une gestion des risques, mais avec des approches différentes : Scrum et Kanban favorisent une assurance qualité continue grâce à des cycles courts et des ajustements réguliers, tandis que Waterfall privilégie une validation finale après chaque phase.

Enfin, du point de vue de la maintenance et de l'évolution du système, les approches agiles offrent une meilleure flexibilité et une facilité d'adaptation, contrairement à Waterfall qui rend les modifications plus coûteuses et complexes après le déploiement.

Ainsi, cette analyse comparative met en évidence la supériorité des approches agiles, notamment Scrum, pour les projets nécessitant flexibilité, interaction continue avec le client et adaptation progressive des besoins, ce qui correspond au contexte du projet *Parc Informatique FMM*.

2.2.2. Méthodologie retenue : Scrum

À l'issue de l'étude comparative des différentes approches de développement, la méthodologie **Scrum** a été retenue pour la réalisation du projet *Parc Informatique FMM*. Ce choix repose sur un ensemble de critères techniques et organisationnels directement alignés avec les besoins du projet et les contraintes du contexte de développement.

Tout d'abord, Scrum se distingue par son approche itérative et incrémentale, qui permet de livrer le système par petites versions successives. Cette caractéristique offre la possibilité d'intégrer progressivement les fonctionnalités du système, notamment la gestion des équipements, des incidents et des utilisateurs, tout en validant continuellement leur conformité aux besoins exprimés.

Par ailleurs, cette méthodologie favorise l'implication continue des parties prenantes, en particulier les utilisateurs finaux tels que les techniciens et les responsables du SmartLab de la Faculté de Médecine de Monastir. Les retours réguliers recueillis à la fin de chaque sprint permettent

d'ajuster les exigences fonctionnelles et non fonctionnelles, réduisant ainsi le risque de divergence entre le besoin exprimé et la solution développée.

En outre, Scrum offre une grande flexibilité face aux changements, ce qui constitue un avantage majeur dans le cadre de ce projet, où certaines contraintes techniques et fonctionnelles ne sont pas entièrement figées au début du développement. Cette adaptabilité permet d'optimiser les choix techniques au fur et à mesure de l'avancement du projet, notamment lors de l'intégration des différents modules du système.

Enfin, la méthodologie Scrum encourage une collaboration étroite entre les différents acteurs du projet, à savoir le Product Owner, le Scrum Master et l'équipe de développement. Cette organisation favorise une communication fluide et une meilleure coordination des tâches, contribuant ainsi à une amélioration continue de la qualité du produit final.

Ainsi, le choix de Scrum s'inscrit dans une démarche pragmatique et adaptée au contexte du projet, en garantissant à la fois flexibilité, qualité et amélioration progressive du système développé.

2.3. Présentation de Scrum

Scrum est un framework agile qui repose sur des cycles de développement appelés sprints, comme présenté dans la figure 2.6. Chaque sprint a une durée définie (généralement entre 2 et 4 semaines) et aboutit à un produit fonctionnel et potentiellement livrable. Le processus Scrum comprend plusieurs éléments clés tels que le *Product Backlog*, le *Sprint Backlog*, l'*Incrément*, ainsi que des événements tels que la planification de sprint, les réunions quotidiennes (*Daily Scrum*), la revue de sprint et la rétrospective de sprint [11].

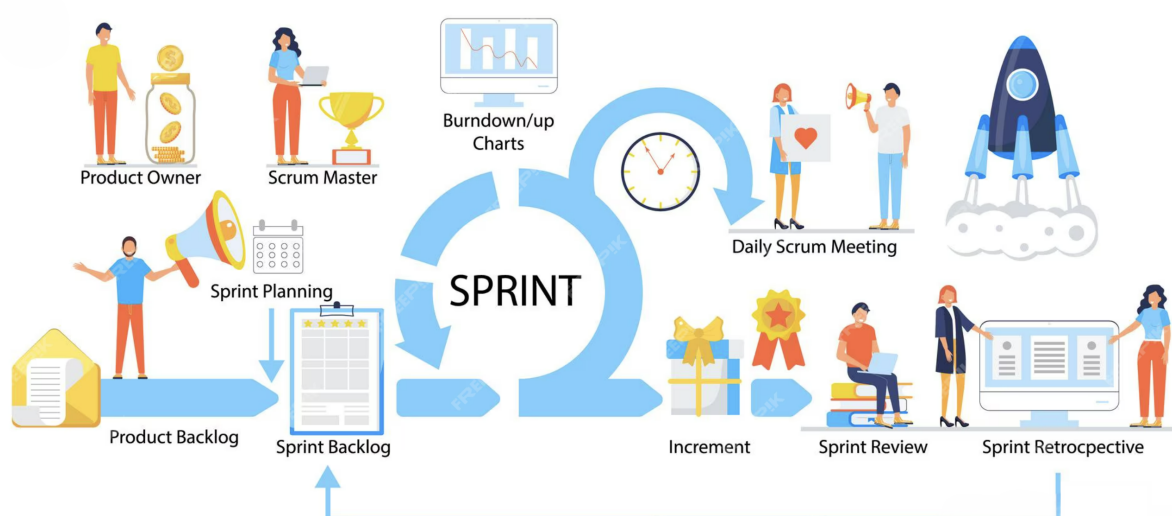


FIGURE 2.6 – Processus Scrum.

2.3.1. Artefacts Scrum et leur application

Scrum repose sur trois artefacts principaux qui structurent le développement du projet : le *Product Backlog*, le *Sprint Backlog* et l'*Incrément*. Ces artefacts sont appliqués comme suit dans le cadre du projet [12] :

- **Product Backlog** : Le *Product Backlog* regroupe l'ensemble des fonctionnalités à développer, telles que définies dans le tableau 4.7 (voir section 4.1.2). Il est géré par le Product Owner, qui priorise les tâches en utilisant la méthode MoSCoW pour classer les fonctionnalités en *Must*, *Should*, *Could* et *Won't*. Ce backlog est régulièrement affiné pour intégrer les retours des parties prenantes et garantir que les fonctionnalités prioritaires répondent aux besoins critiques du parc informatique.
- **Sprint Backlog** : À chaque sprint, un sous-ensemble de tâches est sélectionné à partir du *Product Backlog* pour constituer le *Sprint Backlog*. Par exemple, pour le Sprint 1, les tâches se concentrent sur la mise en place du modèle de données des équipements et des tickets, ainsi que sur l'authentification des utilisateurs.
- **Incrément** : Chaque sprint produit un incrément fonctionnel et potentiellement livrable. Par exemple, à l'issue du Sprint 1, un module de gestion des équipements et un premier prototype de ticketing peuvent être livrés et testés par les techniciens.

2.3.2. Événements Scrum et leur implémentation

Scrum repose sur des événements clés qui structurent le processus de développement. Ces événements sont implémentés comme suit dans le cadre du projet [13] :

- **Planification de sprint** : Lors de la réunion de planification, l'équipe sélectionne les tâches du *Product Backlog* à inclure dans le *Sprint Backlog*, en estimant l'effort requis. Par exemple, pour le Sprint 1, la construction du schéma de données des équipements et la création du module de ticketing peuvent être estimés en jours.
- **Daily Scrum** : Des réunions quotidiennes de 15 minutes sont organisées pour suivre l'avancement des tâches et identifier les obstacles. Par exemple, si un problème de connexion à la base de données est détecté, il est abordé immédiatement pour éviter un retard sur le sprint.
- **Revue de sprint** : À la fin de chaque sprint, l'incrément est présenté aux parties prenantes, comme les techniciens ou les responsables du parc informatique. Leurs retours permettent de valider la conformité des fonctionnalités et d'ajuster les priorités pour le sprint suivant.
- **Rétrospective de sprint** : Après chaque sprint, l'équipe analyse les succès et les défis rencontrés. Par exemple, si l'ergonomie du tableau de bord des équipements nécessite des améliorations, la rétrospective permet d'ajouter ces éléments au backlog.

2.4. Les rôles Scrum

Dans Scrum, trois rôles principaux sont définis pour assurer le bon fonctionnement du processus de développement [14] :

- **Product Owner** : Représentant des parties prenantes, il définit et priorise les fonctionnalités à développer en fonction des besoins des utilisateurs et des objectifs du projet.
- **Scrum Master** : Responsable de la bonne application du framework Scrum. Il facilite la communication, aide à surmonter les obstacles et veille à l'amélioration continue des processus.
- **Équipe de développement** : Composée de développeurs, designers et autres spécialistes, elle est responsable de la mise en œuvre des fonctionnalités et de la livraison des incréments du produit.

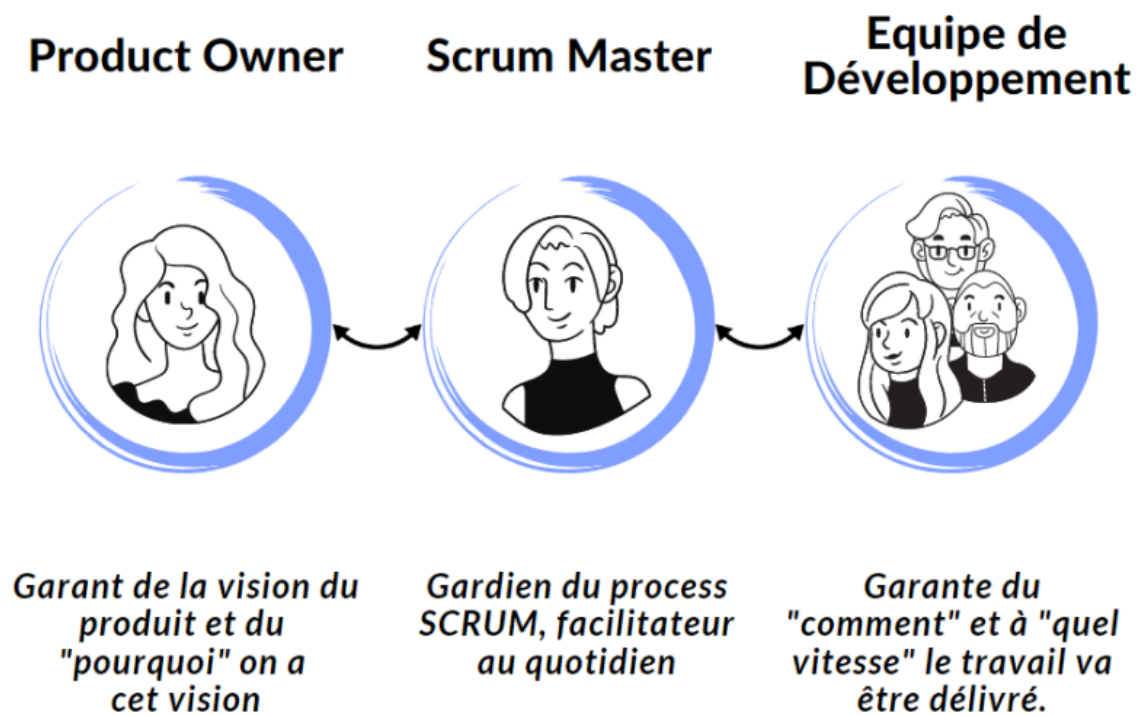


FIGURE 2.7 – Rôles Scrum.

2.5. Outils de support à Scrum

Pour faciliter la mise en œuvre de Scrum, plusieurs outils numériques sont utilisés pour gérer les processus, la communication et le suivi des tâches :

- **ClickUp** : Utilisé pour gérer le *Product Backlog* et le *Sprint Backlog*, ClickUp permet de suivre les tâches, d'assigner des responsabilités et de visualiser l'avancement des sprints à l'aide de tableaux Kanban ou Scrum [15]. Les tâches du projet, comme la mise en place de l'inventaire des équipements ou la création du module de tickets, sont suivies via des tickets

ClickUp avec des estimations de temps et des statuts mis à jour quotidiennement.



FIGURE 2.8 – Logo de ClickUp.

- **Slack** : Cette plateforme de communication permet des échanges en temps réel entre l'équipe de développement, le Product Owner et le Scrum Master [16]. Les *Daily Scrum* peuvent être partiellement gérés via des canaux dédiés, où les membres signalent leurs progrès et obstacles.



FIGURE 2.9 – Logo de Slack.

- **Git** : Utilisé pour le contrôle de version, Git permet de gérer le code source du projet et de faciliter la collaboration entre les développeurs [17]. Les commits sont alignés avec les tâches du *Sprint Backlog*, garantissant une traçabilité des contributions.



FIGURE 2.10 – Logo de Git.

Conclusion

Ce chapitre a présenté la méthodologie Scrum adoptée pour le projet **Parc Informatique FMM**, en détaillant les spécifications des besoins fonctionnels et non fonctionnels, ainsi que les rôles, artefacts et événements Scrum.

Le chapitre suivant se concentrera sur l'architecture du système, en décrivant les choix technologiques, les composants principaux et les interactions entre les différentes parties du système. Nous aborderons également les défis techniques rencontrés et les solutions mises en place pour garantir la robustesse et la scalabilité de l'application.

CHAPITRE 3

Chapitre 3 : Étude architecturale du projet

Table des matières

Introduction	20
3.1 Présentation de l'architecture générale	20
3.2 Modèle de conception logicielle adoptée	21
3.2.1 Hiérarchie organisationnelle et modèle de localisation	21
3.3 Interactions et fonctionnement du système	22
3.3.1 Diagramme des cas d'utilisation	22
3.3.2 Diagramme de classes	22
3.3.3 Diagrammes de séquence	23
3.3.4 Diagramme d'activités	27
3.4 Pourquoi ce choix d'architecture	28
Conclusion	29

Introduction

Ce chapitre décrit l'architecture du projet **Parc Informatique FMM**. L'objectif est d'expliquer les choix technologiques, les composants du système et leurs interactions afin de répondre aux besoins de gestion des équipements, des incidents et des maintenances au sein du SmartLab de la Faculté de Médecine de Monastir.

3.1. Présentation de l'architecture générale

Le Parc Informatique FMM utilise une architecture en trois couches : gestion des ressources, traitement des services et interfaces utilisateur. Chaque couche a un rôle clair, comme montré dans la Figure 3.11. Cette organisation permet de structurer la gestion des actifs informatiques, des tickets et des rapports de manière évolutive.

- **Gestion des ressources** : Cette couche gère les équipements informatiques, les licences, les utilisateurs et les emplacements via une base de données centralisée. Elle permet de consigner l'état des actifs et de suivre leur historique.
- **Traitement des services** : Cette couche prend en charge la logique métier du ticketing, de la gestion des maintenances et des notifications. Le backend traite les demandes, assigne les incidents et génère les alertes selon les règles définies.
- **Interfaces utilisateur** : Cette couche propose une interface web pour que les administrateurs, techniciens et utilisateurs puissent consulter les actifs, créer des tickets et suivre les interventions.

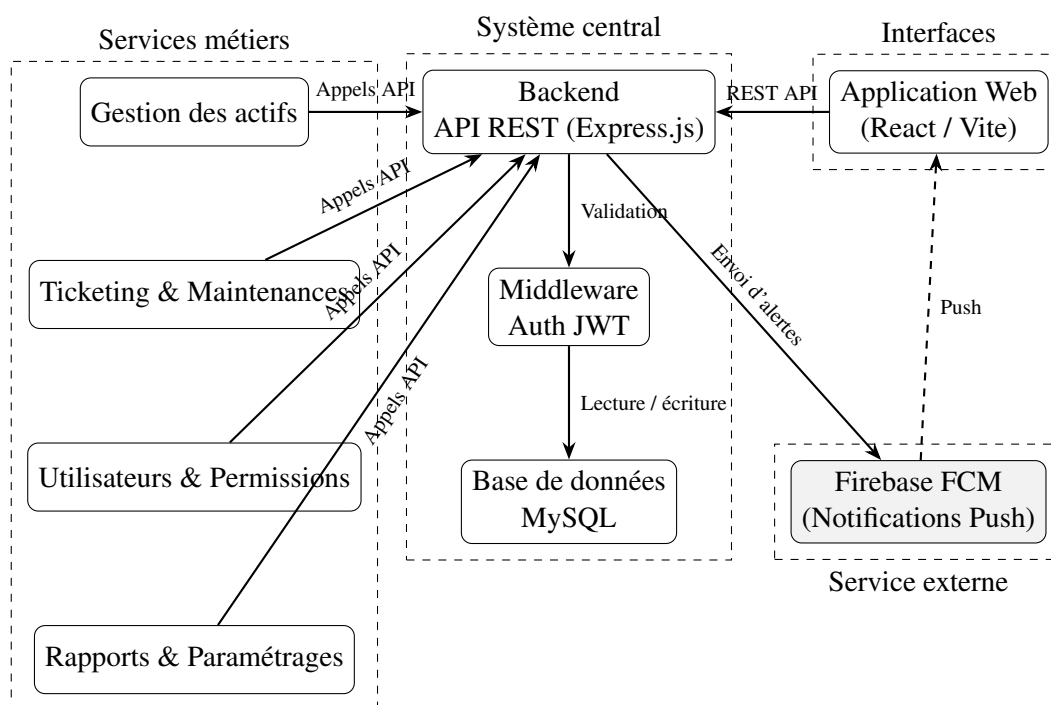


FIGURE 3.11 – Architecture globale du Parc Informatique FMM

3.2. Modèle de conception logicielle adoptée

Pour organiser le système Parc Informatique FMM, nous avons choisi le modèle MVC (Modèle-Vue-Contrôleur). Ce modèle est adapté pour séparer les données, l’affichage et la logique métier, ce qui facilite la maintenance, les tests et l’évolution du système.

- **Modèle** : Cette couche gère les données. La base MySQL centralise les informations relatives aux équipements, aux licences, aux utilisateurs, aux tickets et aux maintenances.
- **Vue** : Cette couche propose des interfaces visuelles. L’application web affiche les tableaux de bord, les états des équipements et les tickets.
- **Contrôleur** : Cette couche gère la logique métier. Le backend traite les requêtes, exécute les règles de gestion, assigne les tickets et déclenche les notifications.

3.2.1. Hiérarchie organisationnelle et modèle de localisation

Le système Parc Informatique FMM modélise l’organisation physique du SmartLab selon une hiérarchie à trois niveaux de localisation. Contrairement à une approche monolithique, cette structure décentralisée offre une flexibilité maximale pour représenter les emplacements complexes :

- **Bloc** : Entité de plus haut niveau représentant un bâtiment, aile ou bloc de locaux
- **Département** : Division administrativo-fonctionnelle au sein d’un Bloc, permettant de regrouper les ressources par domaine métier
- **Salle** : Espace physique concret (laboratoire, salle de cours, bureau, serveur) hébergeant équipements et utilisateurs

Les relations sont structurées comme suit :

- Bloc (1 : *) Département
- Bloc (1 : *) Salle (emplacements directs)
- Département (1 : *) Salle (emplacements subdivisés)

Cette hiérarchie n’est pas rigide : une Salle peut être liée directement à un Bloc (sans passer par Département), ou via un Département intermédiaire. Les équipements et utilisateurs héritent leur localisation via les clés étrangères `DepartmentId`, `SalleId` et `BlocId`.

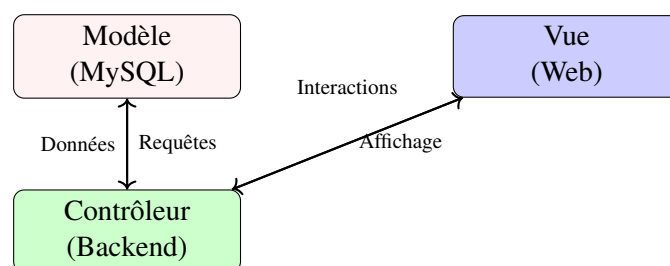


FIGURE 3.12 – Modèle MVC du Parc Informatique FMM

Cette structure permet de séparer clairement les responsabilités, rendant le système plus modulaire et adaptable.

3.3. Interactions et fonctionnement du système

Cette section présente les diagrammes UML qui montrent comment les parties du système travaillent ensemble.

3.3.1. Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation (Figure 3.13) montre comment les acteurs (administrateur, technicien, utilisateur) interagissent avec le système. Il illustre les actions comme la consultation des actifs, la création de tickets, la validation des interventions et la génération de rapports.

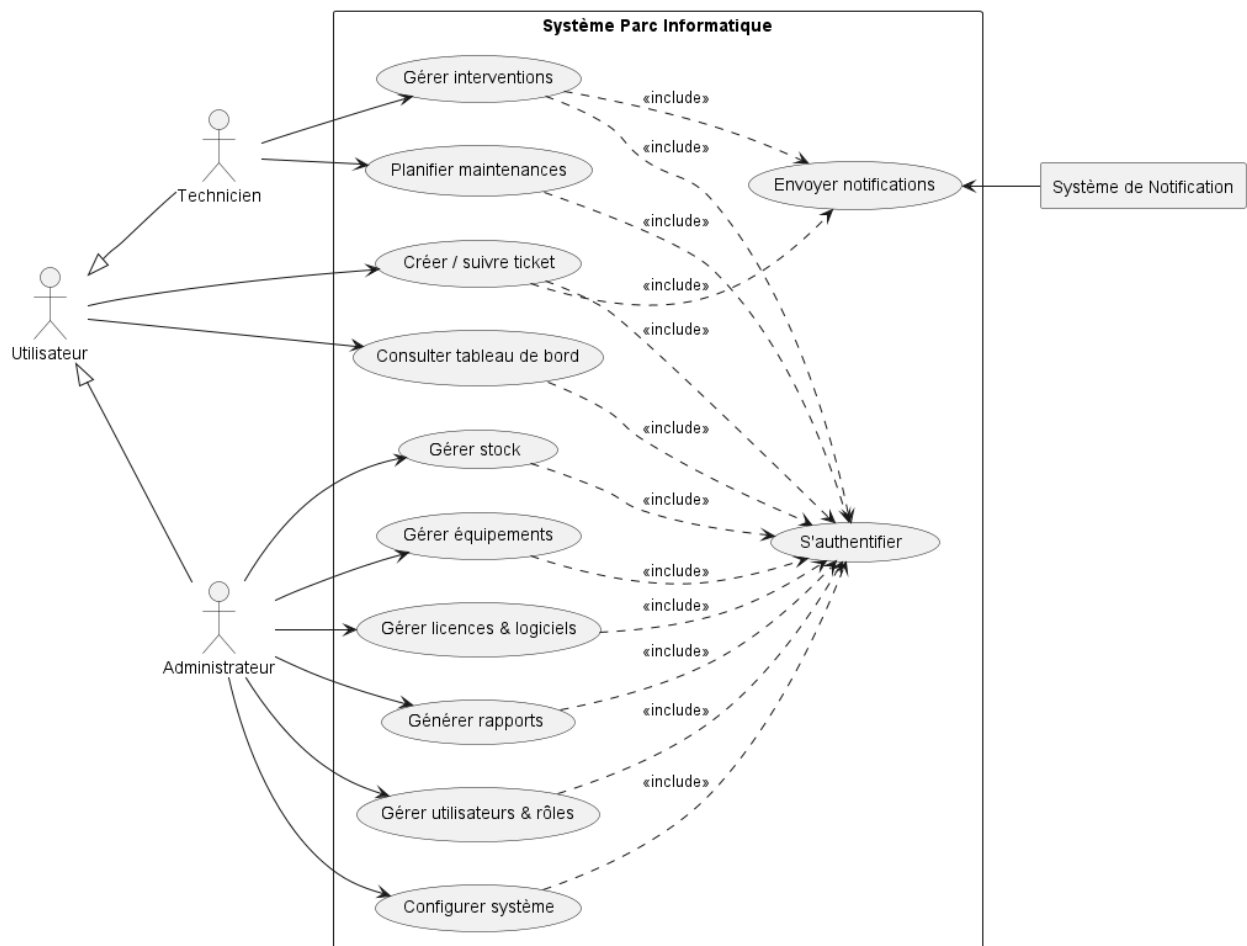


FIGURE 3.13 – Diagramme des cas d'utilisation global

3.3.2. Diagramme de classes

Le diagramme de classes (Figure 3.14) montre la structure des entités du système, avec des éléments comme *Equipement*, *Ticket*, *Utilisateur*, *Maintenance* et *Licence*. Il explique comment les données sont organisées et reliées.

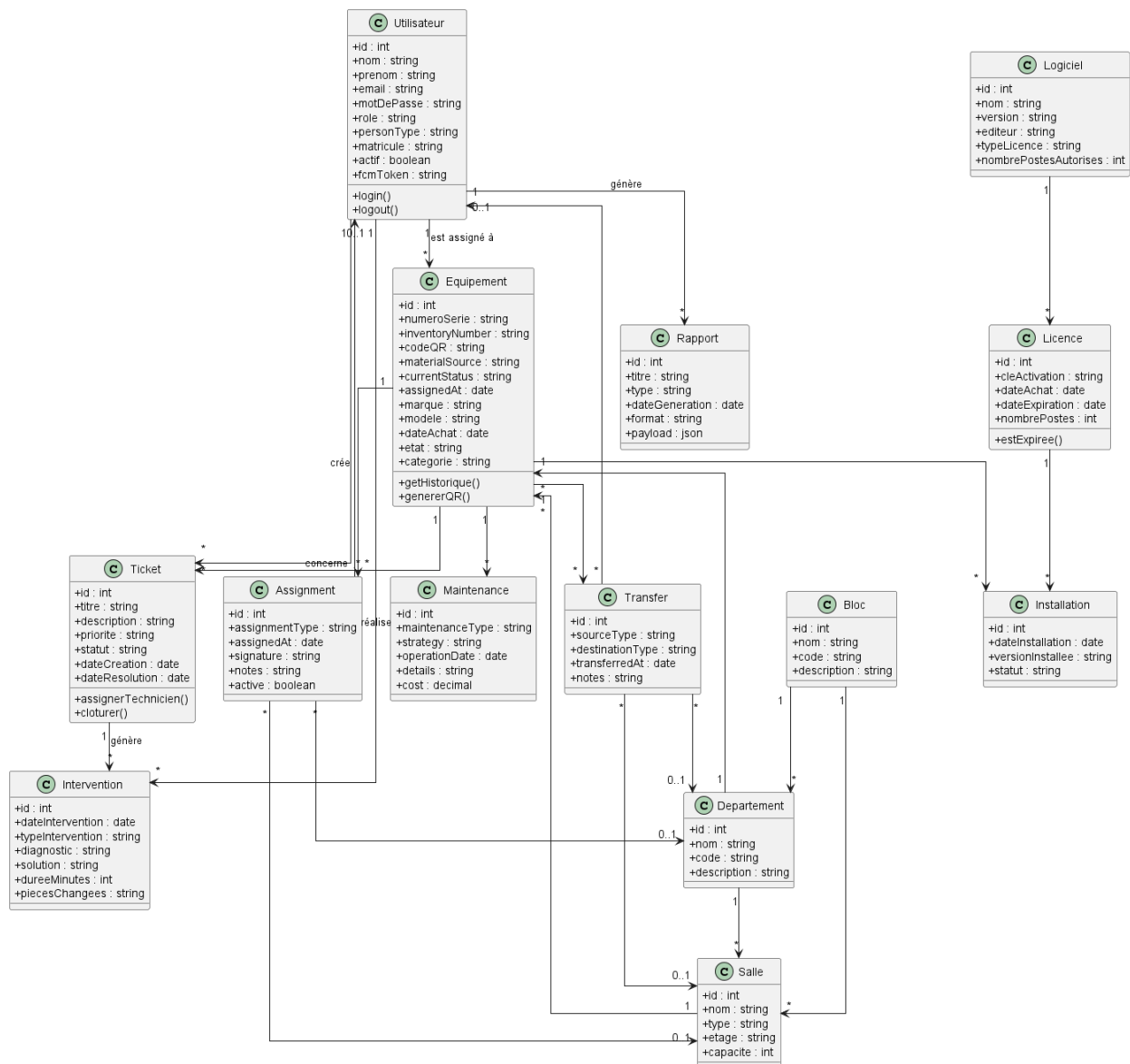


FIGURE 3.14 – Diagramme de classes du Parc Informatique FMM

3.3.3. Diagrammes de séquence

Les diagrammes de séquence constituent un élément essentiel de la modélisation dynamique du système *Parc Informatique FMM*. Ils permettent de représenter de manière chronologique les échanges entre les différents composants du système, notamment l'utilisateur, le frontend, le backend, la base de données ainsi que les services externes tels que Firebase Cloud Messaging (FCM).

Dans ce projet, plusieurs scénarios fonctionnels représentatifs ont été modélisés afin d'illustrer les principaux flux métiers et de mettre en évidence les interactions entre les différentes couches de l'architecture.

Authentification de l'utilisateur

Le diagramme de séquence d'authentification 3.15 illustre le processus permettant à un utilisateur d'accéder au système de manière sécurisée.

Dans ce scénario, l'utilisateur saisit ses identifiants via l'interface web. Ces informations sont ensuite transmises par le frontend au backend, qui procède à leur vérification auprès de la base de données. Si les identifiants sont valides, le système génère un token JWT qui est renvoyé au client. Le frontend stocke alors ce token afin d'autoriser l'accès aux ressources protégées, notamment le tableau de bord.

Ce mécanisme garantit un accès sécurisé au système basé sur une authentification par token.

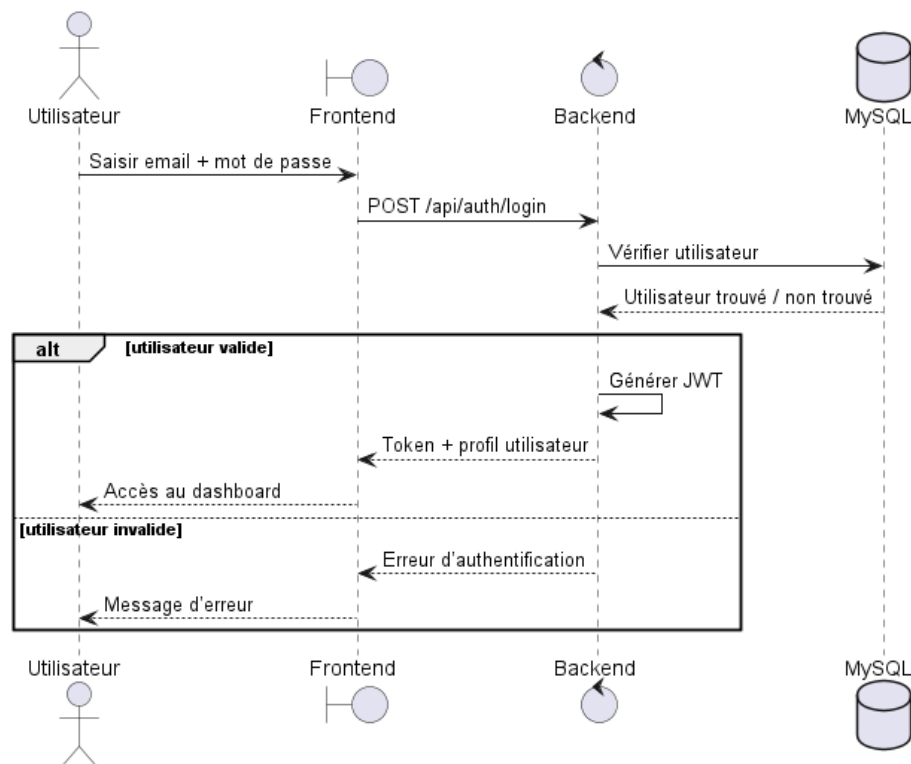


FIGURE 3.15 – Diagramme de séquence d'authentification

Création et assignation d'un ticket d'incident

Ce diagramme 6.31 décrit le flux principal de gestion des incidents, depuis la création d'un ticket jusqu'à son assignation automatique à un technicien disponible.

Dans ce scénario, l'utilisateur crée un ticket en spécifiant les détails de l'incident tels que le titre, la priorité et l'équipement concerné. Le frontend envoie ensuite ces données au backend, qui enregistre le ticket dans la base de données avec un statut initial *ouvert*.

Par la suite, le système déclenche un processus d'affectation automatique afin de rechercher un technicien disponible. Une fois le technicien sélectionné, le ticket lui est assigné et une notification est envoyée via Firebase Cloud Messaging afin de l'informer en temps réel.

Ce processus permet d'assurer une prise en charge rapide et automatisée des incidents.

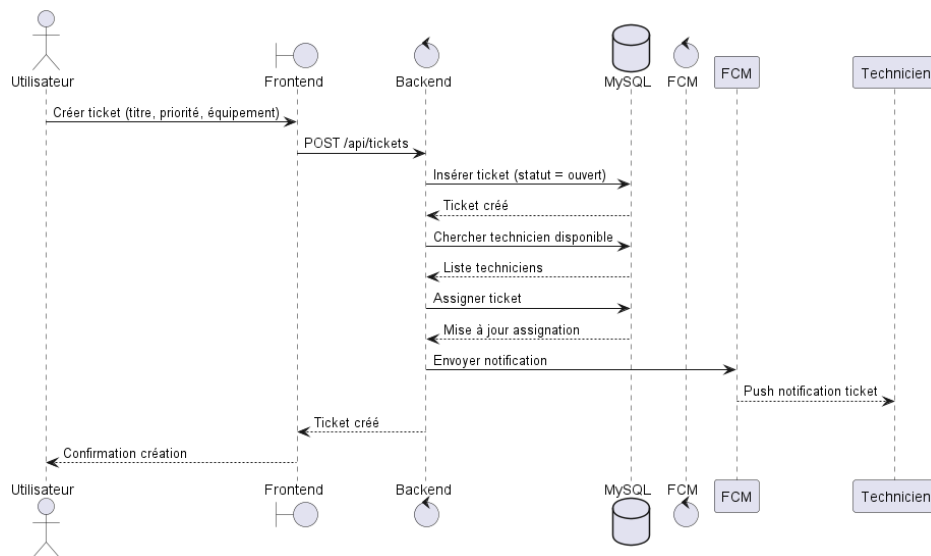


FIGURE 3.16 – Diagramme de séquence de création et assignation d'un ticket

Traitement et mise à jour d'un ticket

Ce diagramme 3.17 illustre les interactions entre le technicien et le système lors du traitement d'un ticket.

Le technicien commence par consulter la liste des tickets qui lui sont attribués. Le frontend interroge alors le backend afin de récupérer les données correspondantes depuis la base de données. Ensuite, le technicien met à jour le statut du ticket en fonction de l'avancement de son intervention (en cours, résolu, etc.).

Ces modifications sont enregistrées par le backend et persistées dans la base de données, ce qui permet de maintenir un historique complet et cohérent des actions effectuées.

Ce flux garantit ainsi la traçabilité des interventions techniques.

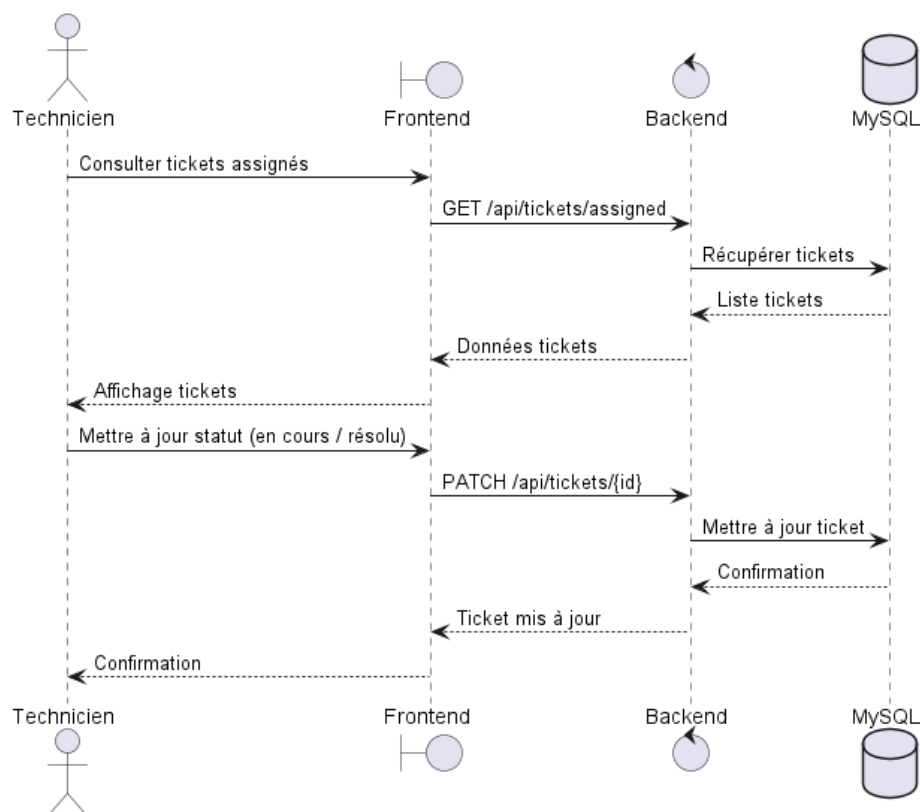


FIGURE 3.17 – Diagramme de séquence de traitement et mise à jour d'un ticket

Clôture et consultation de l'historique

Ce dernier diagramme 6.36 présente le processus de clôture d'un ticket ainsi que la consultation de son historique.

Une fois l'incident résolu, le ticket est clôturé par l'utilisateur ou automatiquement par le système. Le backend met alors à jour son statut à *fermé* et enregistre la date de clôture dans la base de données.

L'utilisateur peut par la suite consulter l'historique complet du ticket afin de suivre l'ensemble des actions réalisées depuis sa création jusqu'à sa fermeture.

Ce mécanisme renforce la traçabilité et améliore le suivi global des incidents.

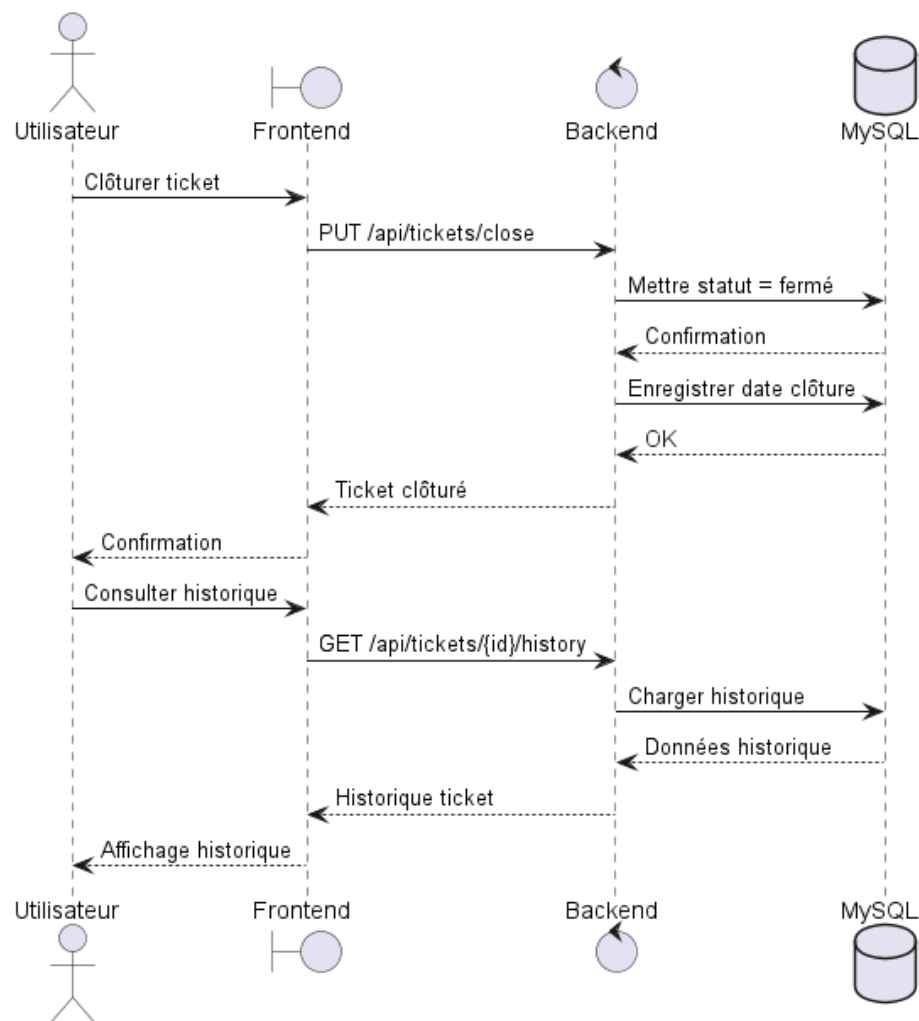


FIGURE 3.18 – Diagramme de séquence de clôture et consultation de l'historique d'un ticket

Synthèse L'ensemble de ces diagrammes de séquence met en évidence la cohérence globale de l'architecture du système ainsi que la fluidité des interactions entre ses différents composants. Ils soulignent également le rôle central du backend dans la coordination des traitements métiers, ainsi que l'importance des services externes tels que Firebase Cloud Messaging pour assurer la communication en temps réel entre les acteurs du système.

3.3.4. Diagramme d'activités

Le diagramme d'activités (Figure 3.19) illustre le processus complet de gestion des incidents dans le système Parc Informatique FMM. Il débute par la soumission d'un ticket par l'utilisateur, incluant titre, priorité, équipement et photo. Une validation des données est effectuée ; si elles sont valides, le ticket est enregistré avec le statut "ouvert" et une notification push est envoyée au technicien via Firebase Cloud Messaging (FCM). Le technicien consulte les tickets, prend en charge celui-ci, changeant le statut à "assigné". Il crée ensuite une intervention, mettant le ticket à "en cours", puis effectue la réparation sur le terrain. Si l'intervention réussit, il saisit la solution, les pièces changées et le rapport, terminant l'intervention. Le système génère automatiquement

une maintenance corrective liée à l'intervention, marque le ticket comme "résolu". L'utilisateur confirme alors la résolution, clôturant le ticket avec la date de clôture et l'archivant. En cas d'échec de l'intervention, le statut est mis à "échec", et le ticket est réouvert ou escaladé. Si les données initiales ne sont pas valides, la demande est rejetée avec un message d'erreur.

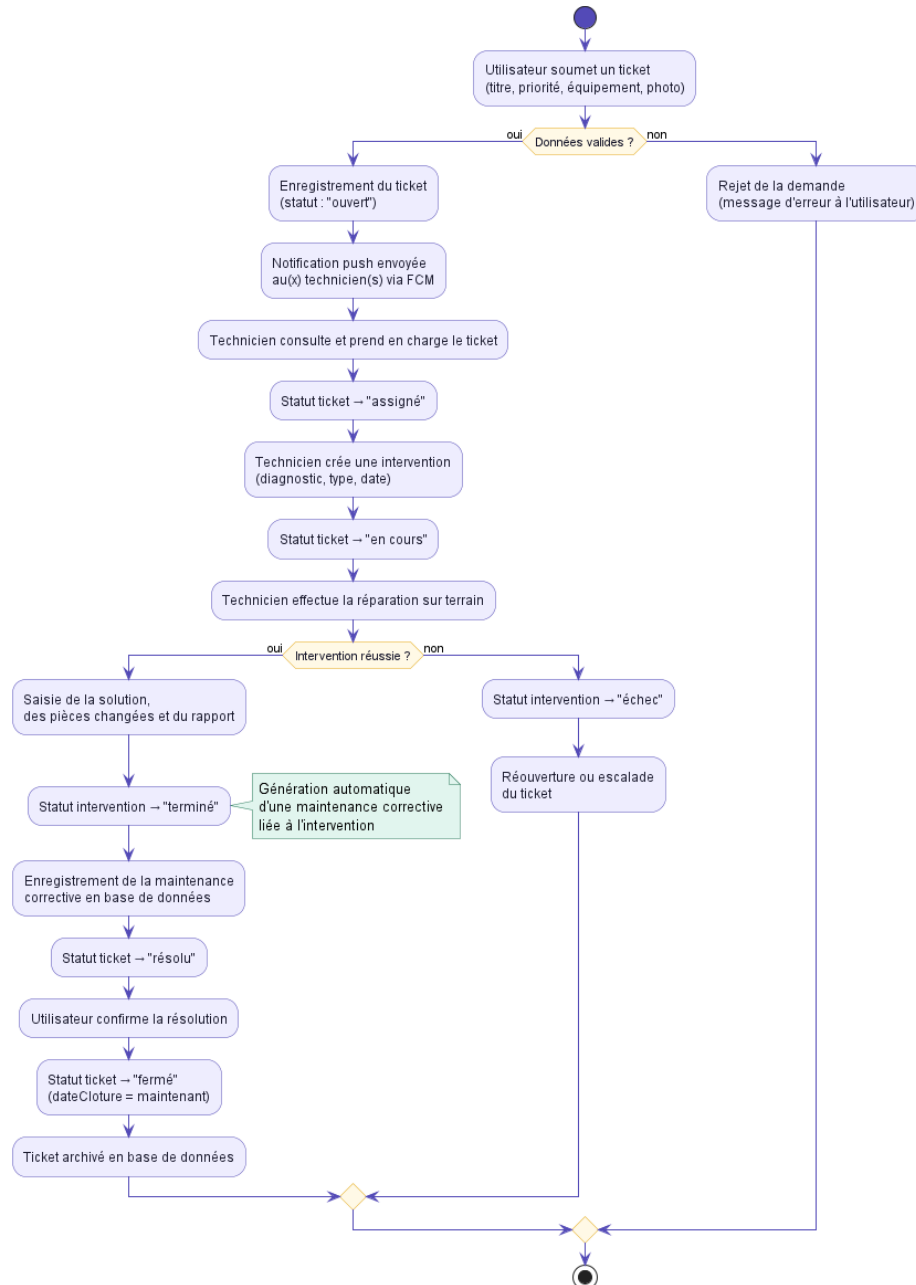


FIGURE 3.19 – Diagramme d'activité du flux des processus

3.4. Pourquoi ce choix d'architecture

Le choix de cette architecture repose sur plusieurs considérations techniques et fonctionnelles visant à garantir un système performant, évolutif et facile à maintenir.

- **Simplicité et séparation des responsabilités** : L'architecture en couches permet de distin-

guer clairement les différentes parties du système, notamment la gestion des données, la logique métier et l'interface utilisateur. Cette séparation facilite la compréhension du système ainsi que son évolution.

- **Maintenabilité** : Grâce à cette organisation modulaire, toute modification ou amélioration peut être effectuée sur une couche spécifique sans impacter l'ensemble du système, ce qui réduit les risques de régression.
- **Réactivité du système** : Le traitement des tickets et des notifications est conçu de manière à assurer une prise en charge rapide des incidents, améliorant ainsi la réactivité globale du système.
- **Adaptabilité et extensibilité** : L'architecture retenue permet de supporter différents types d'équipements et de s'adapter facilement à divers environnements d'exécution, qu'il s'agisse d'infrastructures locales ou de solutions cloud.
- **Sécurité** : Le système intègre des mécanismes d'authentification, d'autorisation et de protection des données afin de garantir la confidentialité et l'intégrité des informations.

Ainsi, ces choix architecturaux répondent aux exigences des environnements professionnels tels que le SmartLab, en offrant une solution modulaire, robuste et évolutive, adaptée à la gestion d'un parc informatique moderne.

Conclusion

Ce chapitre a présenté l'architecture globale du projet *Parc Informatique FMM*, en mettant en évidence l'organisation en couches, l'approche MVC ainsi que les principaux diagrammes UML utilisés pour la modélisation du système.

L'architecture retenue garantit une meilleure structuration du système, tout en assurant sa flexibilité, sa maintenabilité et sa capacité d'évolution afin de répondre efficacement aux besoins de gestion d'un parc informatique moderne. Les diagrammes de séquence, d'activités et de classes ont permis d'illustrer les interactions entre les différents composants du système, ainsi que les flux de données et les processus métiers clés.

Dans le chapitre suivant, nous détaillerons les choix technologiques effectués pour la mise en œuvre du projet, en expliquant les raisons de ces choix et leur impact sur le développement et la performance du système.

CHAPITRE 4

Chapitre 4 : Planification et préparation du projet Parc Informatique FMM

Table des matières

Introduction	31
4.1 Pilotage du projet avec Scrum	31
4.1.1 Acteurs Scrum et Missions	31
4.1.2 Organisation du Backlog	31
4.1.3 Planification des sprints	32
4.2 Environnement de travail	34
4.2.1 Environnement matériel	34
4.2.2 Environnement de développement	35
4.2.3 Environnement logiciel	36
Conclusion	38

Introduction

Le Sprint 0 constitue la phase initiale du projet Parc Informatique FMM, qui vise à développer un système de gestion des équipements informatiques, des incidents et des maintenances au sein du SmartLab de la Faculté de Médecine de Monastir. Cette étape fondamentale permet d'identifier les acteurs clés, de recueillir les besoins fonctionnels et non fonctionnels, et de définir la méthodologie Scrum qui guidera les phases ultérieures du développement. L'objectif est d'assurer un alignement avec les ambitions du projet, à savoir améliorer la gestion des actifs informatiques grâce à une collecte automatisée des données, une gestion des incidents en temps réel et des rapports détaillés, comme décrit dans les chapitres d'introduction générale et d'étude architecturale.

4.1. Pilotage du projet avec Scrum

Le projet adopte la méthodologie Scrum pour assurer un développement itératif et une collaboration étroite avec les parties prenantes, comme justifié dans le chapitre consacré à la méthodologie. Cette section détaille les rôles Scrum, l'organisation du backlog et la planification des sprints.

4.1.1. Acteurs Scrum et Missions

Les rôles Scrum et leurs responsabilités sont définis dans un tableau spécifique, garantissant une approche collaborative alignée sur les principes décrits précédemment. Le Product Owner est chargé de définir et de prioriser les fonctionnalités du produit, tout en validant les livrables pour s'assurer qu'ils répondent aux besoins du projet. L'équipe de développement conçoit, développe et teste les fonctionnalités, en veillant à la qualité du code produit. Le Scrum Master facilite les événements Scrum, élimine les obstacles et veille à ce que l'équipe adhère aux pratiques de la méthodologie.

TABLE 4.6 – Acteurs du projet Scrum

Rôle	Mission	Acteur
Product Owner	Définir et prioriser les fonctionnalités du produit (Product Backlog), valider les livrables.	Pr Ammeri Charfeddine
Équipe de Développement	Concevoir, développer et tester les fonctionnalités, assurer la qualité du code.	Hela Boussaid
Scrum Master	Faciliter les événements Scrum, supprimer les obstacles, assurer l'adhésion à Scrum.	Racha Zaibi

4.1.2. Organisation du Backlog

Le Product Backlog est géré à l'aide d'un outil de gestion de projet, organisé selon la méthode de priorisation MoSCoW pour refléter les objectifs stratégiques du projet. Chaque fonctionnalité est formulée sous forme de User Story pour garantir un développement centré sur l'utilisateur. Les

tâches prioritaires incluent la gestion des équipements informatiques avec ajout, modification et suppression, la gestion des incidents avec création de tickets et assignation aux techniciens, ainsi que la gestion des maintenances préventives et correctives. Les interfaces web permettent la visualisation des équipements et la gestion des tickets, tandis que l'authentification et la gestion des utilisateurs assurent un accès sécurisé basé sur les rôles (administrateur, technicien, utilisateur). Le stockage des données et des historiques dans une base de données relationnelle est également prioritaire, tout comme la génération de rapports sur les équipements et les incidents. D'autres tâches, jugées importantes mais non critiques pour la première version, incluent les notifications push pour les nouveaux tickets, la gestion des licences logicielles, et l'optimisation des performances pour un grand nombre d'équipements. Des fonctionnalités optionnelles, comme l'intégration avec des outils externes ou le support multilingue, pourraient être ajoutées si les ressources le permettent. Enfin, certaines tâches, telles que l'analyse prédictive des pannes avec intelligence artificielle, sont hors du périmètre actuel.

TABLE 4.7 – Backlog du Projet avec Priorités MoSCoW

ID	Tâche	Priorité
M-01	Gestion des équipements : ajout, modification, suppression et visualisation.	M
M-02	Gestion des incidents : création de tickets, assignation et résolution.	M
M-03	Gestion des maintenances : préventives et correctives.	M
M-04	Interface web pour administrateurs et techniciens.	M
M-05	Authentification et gestion des utilisateurs (rôles : admin, technicien, utilisateur).	M
M-06	Stockage et historique des données dans une base de données MySQL.	M
M-07	Génération de rapports sur les équipements et incidents.	M
S-01	Notifications push pour nouveaux tickets via Firebase FCM.	S
S-02	Gestion des licences logicielles associées aux équipements.	S
S-03	Optimisation des performances pour gestion de nombreux équipements.	S
C-02	Support multilingue pour l'interface.	C
W-01	Analyse prédictive des pannes avec IA.	W

4.1.3. Planification des sprints

Le projet est structuré en trois sprints de trois semaines chacun. Le tableau ci-dessous présente une synthèse des objectifs, livrables, tâches, critères d'acceptation et risques associés à chaque sprint.

TABLE 4.8 – Planification des Sprints du Projet Parc Informatique FMM

Sprint	Objectifs et livrables	Tâches principales	Critères d'acceptation
Sprint 1 Mise en place de l'architecture et gestion des équipements	– Backend API REST avec Express.js. – Base de données MySQL configurée. – Module de gestion des équipements fonctionnel.	– Configuration du serveur backend. – Création des schémas de base de données. – Développement des endpoints pour CRUD des équipements. – Tests unitaires des fonctionnalités.	– Équipements ajoutés, modifiés et supprimés via API. – Données persistées correctement en base. – Interface web basique pour visualisation.
Sprint 2 Gestion des incidents et maintenances	– Système de ticketing opérationnel. – Module de maintenances implémenté. – Notifications push intégrées.	– Développement des endpoints pour tickets et maintenances. – Intégration de Firebase FCM pour notifications. – Assignment automatique des techniciens. – Tests d'intégration.	– Tickets créés et assignés aux techniciens. – Maintenances générées automatiquement. – Notifications envoyées en temps réel.
Sprint 3 Finalisation et rapports	– Interface web complète avec React. – Système de rapports fonctionnel. – Authentification et autorisation sécurisées.	– Développement de l'interface web. – Implémentation de l'authentification JWT. – Génération de rapports PDF/JSON. – Tests de sécurité et performance.	– Interface utilisateur intuitive et responsive. – Rapports générés sur demande. – Accès sécurisé selon les rôles.

4.2. Environnement de travail

Ce projet est développé sur une configuration locale Windows avec une architecture applicative fondée sur un backend Node.js/Express.js et un frontend React/Vite. Les sections suivantes décrivent l'environnement matériel, l'environnement de développement et l'environnement logiciel utilisés par l'équipe.

4.2.1. Environnement matériel

L'environnement matériel utilisé pour le développement du projet repose sur une machine de travail performante, permettant de garantir de bonnes performances lors de l'exécution des outils de développement, des tests et du déploiement local des applications.

Le poste de développement est un ordinateur portable équipé d'un processeur Intel Core i7 de 10^e génération, de 16 Go de mémoire RAM et d'un stockage SSD de 512 Go. Cette configuration assure une exécution fluide des environnements backend et frontend ainsi qu'une bonne réactivité lors de la compilation et des tests.

Le système d'exploitation utilisé est Windows, choisi pour sa stabilité et sa compatibilité avec les technologies employées dans le projet. La base de données MySQL est installée en local afin d'assurer une gestion efficace, rapide et sécurisée des données du système. Enfin, l'architecture réseau repose sur un environnement local (*localhost*), permettant de réaliser les tests et le développement sans dépendre d'une infrastructure externe.

Système d'exploitation



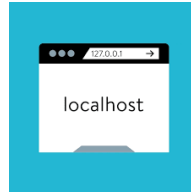
Windows est utilisé comme système principal de développement. Il offre une compatibilité avec les outils essentiels tels que Node.js, les environnements de développement intégrés (IDE) et les outils de gestion de bases de données.

Base de données



MySQL est utilisé comme système de gestion de base de données relationnelle. Il est installé en local et fonctionne sur le port 3306, assurant la persistance et l'intégrité des données du système.

Réseau



L'environnement réseau repose sur une configuration locale (*localhost*), utilisée pour le développement et les tests, garantissant un déploiement rapide et indépendant de toute infrastructure externe.

4.2.2. Environnement de développement

L'environnement de développement regroupe l'ensemble des outils utilisés pour la conception, l'implémentation et la documentation du projet. Il permet d'assurer un cycle de développement complet, allant de l'écriture du code jusqu'au test et à la génération de la documentation technique.

L'éditeur principal utilisé est Visual Studio Code, choisi pour sa légèreté, sa richesse en extensions et son support avancé des technologies web modernes.



Le backend repose sur Node.js accompagné de npm, qui constitue le gestionnaire de paquets permettant l'installation et la gestion des dépendances du projet.



Plusieurs outils complètent l'environnement de développement afin d'améliorer la productivité et la qualité du code :

- **Nodemon** : outil de rechargement automatique du serveur backend lors des modifications du code.

- **Vite** : outil moderne de développement frontend offrant un démarrage rapide et le Hot Module Replacement (HMR).
- **ESLint** : outil d'analyse statique permettant de détecter les erreurs et d'assurer la qualité du code JavaScript.
- **Git** : système de contrôle de version utilisé pour le suivi des modifications et la collaboration.
- **Swagger UI** : interface de documentation interactive permettant de tester et visualiser les endpoints de l'API REST.

4.2.3. Environnement logiciel

L'environnement logiciel du projet repose sur une architecture JavaScript complète (Full Stack), séparée en backend, frontend et base de données. Cette organisation permet une meilleure modularité, une maintenance simplifiée et une évolutivité du système.

Backend

Le backend est développé sous Node.js et repose sur Express.js pour la création de l'API REST. Il est responsable de la logique métier, de la gestion des utilisateurs, des équipements, des tickets ainsi que de la communication avec la base de données.

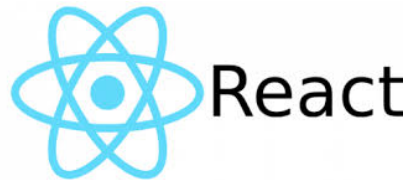


Les principales bibliothèques utilisées sont :

- **Sequelize** : ORM facilitant la communication avec MySQL et la gestion des relations.
- **MySQL2** : pilote de connexion à la base de données.
- **JWT (JSON Web Token)** : gestion de l'authentification et sécurisation des API.
- **bcryptjs** : hachage sécurisé des mots de passe.
- **Firebase Admin** : envoi de notifications push via FCM.
- **Multer** : gestion des uploads de fichiers (images de tickets).
- **PDFKit** : génération de rapports PDF.
- **ExcelJS** : export des données en format Excel.
- **Nodemailer** : envoi d'e-mails transactionnels.
- **Swagger (JSDoc/UI)** : documentation interactive de l'API REST.
- **Morgan** : journalisation des requêtes HTTP.
- **dotenv** : gestion des variables d'environnement.

Frontend

Le frontend est développé avec React et Vite, permettant la création d'une interface utilisateur dynamique et réactive adaptée aux besoins du système.



Les principales technologies utilisées sont :

- **React Router DOM** : gestion de la navigation dans l'application SPA.
- **Material UI (MUI)** : bibliothèque de composants UI modernes.
- **MUI DataGrid** : gestion des tableaux de données avancés.
- **Axios** : communication avec l'API backend.
- **TanStack Query** : gestion du cache et des requêtes serveur.
- **Firebase** : notifications push côté client.
- **Recharts** : visualisation des données sous forme de graphiques.
- **html5-qrcode / qrcode.react** : génération et lecture de QR codes.
- **i18next** : internationalisation (FR/EN).
- **react-hot-toast** : notifications utilisateur.
- **date-fns** : manipulation des dates.
- **lucide-react** : icônes modernes.

Base de données

La base de données utilisée est MySQL, qui assure la persistance et l'intégrité des données du système.



Elle est caractérisée par :

- Nom de la base : **gmao_parc**
- Port : **3306**
- ORM utilisé : **Sequelize** (migrations et relations)

Conclusion

Le Sprint 0 a permis de poser les bases solides du projet Parc Informatique FMM en définissant les rôles Scrum, en priorisant le backlog selon MoSCoW et en planifiant les sprints. Avec l'équipe composée de Hela Boussaid en développement et Rasha Zaibi en tant que Scrum Master, le projet est prêt à entrer dans les phases de développement itératif. Les prochains chapitres détailleront l'implémentation des sprints et les résultats obtenus.

CHAPITRE 5

Chapitre 5 : Mise en place de l'architecture et gestion des équipements

Table des matières

Introduction	40
5.1 Objectifs et livrables	41
5.2 Architecture et modélisation UML du Sprint 1	41
5.2.1 Clarification terminologique : Assignment et Affectation	43
5.2.2 Diagramme de classes Sprint 1	43
5.2.3 Diagrammes de séquence Sprint 1	43
5.3 Tâches principales	47
5.3.1 Configuration du serveur backend	47
5.3.2 Création des schémas de base de données	48
5.3.3 Développement de l'API REST et Architecture des Endpoints	49
5.3.4 Mécanismes Transverses : Sécurité, Validation et Cycle de Vie	50
5.4 Critères d'acceptation	51
5.5 Implémentation et résultats	53
5.6 Tests et validation	54

5.7 Défis et solutions	54
5.8 Planification pour le sprint suivant	55
Conclusion	55

Introduction

Ce chapitre est entièrement dédié au **Sprint 1**, qui constitue la première itération majeure dans le cycle de développement du système de gestion des équipements informatiques destiné au *SmartLab* de la Faculté de Médecine de Monastir. Faisant suite à l'étude architecturale globale et respectant la planification de notre feuille de route (détaillée précédemment dans le Tableau 4.8), ce premier jalon s'inscrit pleinement dans la méthodologie Agile Scrum adoptée pour ce projet.

L'objectif primordial de ce sprint est de jeter des fondations techniques à la fois robustes, évolutives et hautement sécurisées. L'effort de développement s'articule ainsi autour de trois axes directeurs :

- **La mise en place de l'infrastructure Backend** : Déploiement d'une architecture orientée services capable de centraliser la logique métier.
- **La persistance et la modélisation des données** : Configuration et structuration fine de la base de données relationnelle pour garantir l'intégrité des informations.
- **Le cœur fonctionnel initial** : Implémentation du module de gestion des équipements (opérations CRUD, filtres avancés et fonctionnalités d'importation).

Afin de soutenir ces objectifs et d'assurer une séparation claire des responsabilités (SOC - *Separation of Concerns*), nous avons opté pour une **architecture découplée en 3 couches (3-Tier)** : une couche de Présentation (React.js), une couche Application & Logique Métier (Express.js & Sequelize), et une couche de Données (MySQL).

La Figure 5.20 détaille la cartographie de cette architecture technique sous forme de niveaux (Tiers), mettant en exergue l'alignement des flux et l'intégration des différents modules développés lors de cette itération.

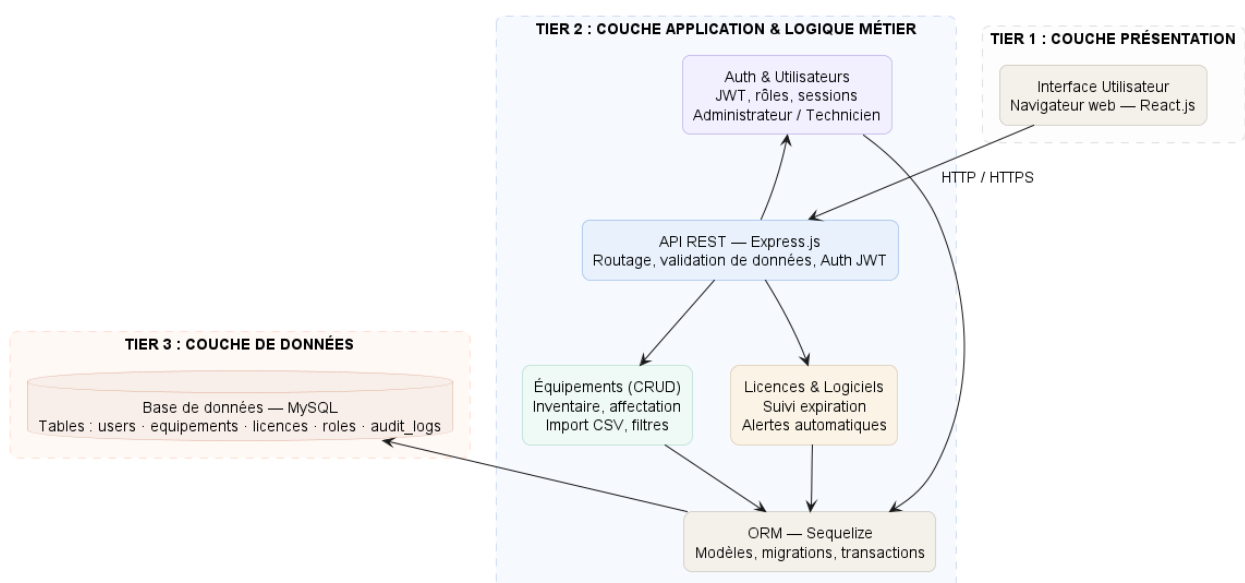


FIGURE 5.20 – Architecture technique multicouche du Sprint 1 — Flux et composants du système

5.1. Objectifs et livrables

L'objectif principal du Sprint 1 est d'établir une infrastructure backend solide et fonctionnelle permettant la gestion complète des équipements informatiques et des processus d'exploitation associés. Cette phase vise à mettre en place un serveur d'application performant basé sur Express.js[6] et Sequelize[18], et à configurer une base de données relationnelle capable de persister les données avec fiabilité et cohérence. Les cinq livrables clés de ce sprint sont :

- **Backend API REST avec Express.js[6] et Sequelize ORM[18]** : Une API REST complète et documentée, implémentée avec Express.js, exposant 16 suites d'endpoints sécurisés couvrant les équipements, les tickets, les interventions, les maintenances, les installations, les licences et plus. L'API intègre une authentification JWT Bearer, une validation des entrées via Sequelize, une gestion d'erreurs robuste, et des logs structurés via Morgan.
- **Base de données relationnelle MySQL[8] avec 18 modèles** : Une base de données MySQL avec un schéma complexe modélisant 18 entités métier incluant Équipement, Ticket, Intervention, Maintenance, Installation, Licence, Logiciel, Assignment, Transfer, Rapport, Bloc, Salle, et Département. Chaque modèle implémente des relations définies via Sequelize avec indices d'optimisation et contraintes d'intégrité.
- **Module de gestion des équipements avec CRUD avancé** : Un module complet implémentant 9 opérations pour les équipements (création, lecture, modification, suppression, historique, assignation, transfert, disposition). Chaque opération inclut la validation métier, le contrôle d'accès basé sur les rôles (RBAC), et la génération d'événements auditable.
- **Système de gestion des incidents et maintenances** : Un module pour la création et la gestion de tickets informatiques, des interventions associées, et des plans de maintenance préventive et corrective, avec assignation automatique des techniciens et suivi du statut.
- **Interface web intuitive et gestion des licences logicielles** : Une interface frontend React[7] avec Material-UI[10], construite avec Vite[19], offrant une authentification, gestion des utilisateurs, des licences logicielles, des installations et des transferts d'équipements, générant des rapports PDF/Excel et des statistiques.

Ces objectifs s'alignent avec les priorités M (Must have) du Product Backlog (voir Tableau 4.7), en particulier les tâches M-01 « Gestion des équipements », M-02 « Gestion des incidents », M-03 « Gestion des maintenances », et M-06 « Stockage et historique des données », comme détaillé dans le chapitre précédent. Le périmètre du Sprint 1 couvre également les fonctionnalités S (Should have) pour assurer une base production-ready complète.

5.2. Architecture et modélisation UML du Sprint 1

Le Sprint 1 pose les fondations du projet Parc Informatique FMM en privilégiant une architecture web moderne basée sur React pour le frontend, Express.js pour le backend, MySQL pour la persistance des données et Firebase pour les notifications. Cette phase établit les modèles fonda-

mentaux pour la gestion des équipements, utilisateurs et maintenances. Les acteurs impliqués sont l'Administrateur, le Technicien et l'Utilisateur.

Entités et modèles de données

L'ensemble du système repose sur 17 modèles persistants :

TABLE 5.9 – Modèles de données du système Parc Informatique FMM

Modèle	Responsabilité
Utilisateur	Profils utilisateur avec rôles RBAC, tokens FCM
Équipement	Inventaire d'actifs informatiques avec métadonnées
Bloc	Bâtiments ou zones organisationnelles de haut niveau
Département	Divisions administratives au sein des Blocs
Salle	Espaces physiques concrets
Assignment	Historique des assignations d'équipements
Transfer	Traces d'audit des mouvements d'équipements
Ticket	Incidents et demandes de support
Intervention	Enregistrement des interventions de maintenance
Maintenance	Suivi des maintenances correctives
MaintenancePreventive	Planification des maintenances préventives récurrentes
Logiciel	Inventaire des applications avec versions
Licence	Licences logicielles avec dates d'expiration
Installation	Historique d'installation de logiciels sur équipements
Rapport	Documents générés (PDF/Excel/CSV) pour rapports de workflow
Parametrage	Configuration système et paramètres i18n (français/anglais)
Permission	Permissions granulaires par module et par rôle (RBAC)

Champs étendus pour les équipements

Le modèle Équipement enrichit l'inventaire standard avec plusieurs attributs :

- **numeroSerie** : Identifiant fabricant (unique, obligatoire)
- **inventoryNumber** : Numéro d'inventaire interne (unique, obligatoire)
- **codeQR** : Code QR pour suivi rapide via scanner (stocké en chaîne de caractères)

- **marque** et **modele** : Identifiants commerciaux
- **categorie** : Classification de l'équipement (ex : PC, Imprimante, Serveur - chaîne de caractères)
- **etat** : État opérationnel (fonctionnel, en_panne, en_réparation, réformé)
- **materialSource** : Provenance (PURCHASED, DONATED, INHERITED)
- **currentStatus** et **currentHolderType** : Localisation courante
- **dateAchat** : Date d'acquisition
- **assignedAt** : Timestamp de la dernière assignation

Note importante : Les champs *codeQR* et *categorie* sont stockés comme chaînes de caractères pour simplifier la gestion. Une future évolution pourrait les promouvoir en entités séparées si un suivi granulaire devient nécessaire.

5.2.1. Clarification terminologique : Assignment et Affectation

La documentation du projet utilise le terme français **Affectation** pour décrire l'assignation d'équipements aux utilisateurs, départements ou salles. Le code source implémente cette notion via le modèle `Assignment.js`. Dans ce document, nous utilisons indifféremment les deux termes comme synonymes, en privilégiant **Affectation** dans les descriptions métier.

5.2.2. Diagramme de classes Sprint 1

Le diagramme de classes du Sprint 1 formalise les principales entités métiers du module de gestion des équipements. Il met en évidence les relations entre les entités, les contraintes de propriété et les objets supports nécessaires pour la traçabilité des affectations et des identifiants QR.

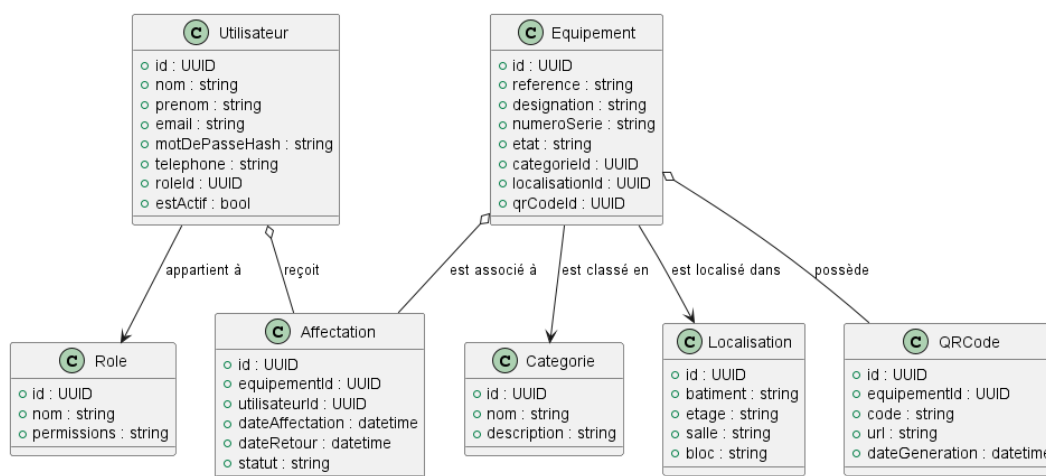


FIGURE 5.21 – Diagramme de classes du Sprint 1 — Gestion des équipements et des utilisateurs

5.2.3. Diagrammes de séquence Sprint 1

Cette série de diagrammes de séquence décrit les principaux scénarios dynamiques du Sprint 1. Chacun met en lumière la collaboration entre l'acteur humain, le frontend React, le backend Ex-

press et la base de données MySQL, tout en intégrant les règles de sécurité liées à l'authentification JWT et au contrôle d'accès RBAC.

Authentification utilisateur

Le diagramme de séquence d'authentification modélise l'accès sécurisé au système. L'utilisateur soumet son identifiant et son mot de passe sur le frontend, lequel transmet la requête au backend. Express vérifie les informations dans MySQL et génère un jeton JWT en cas de succès, qui permet l'accès aux fonctionnalités en fonction du rôle.

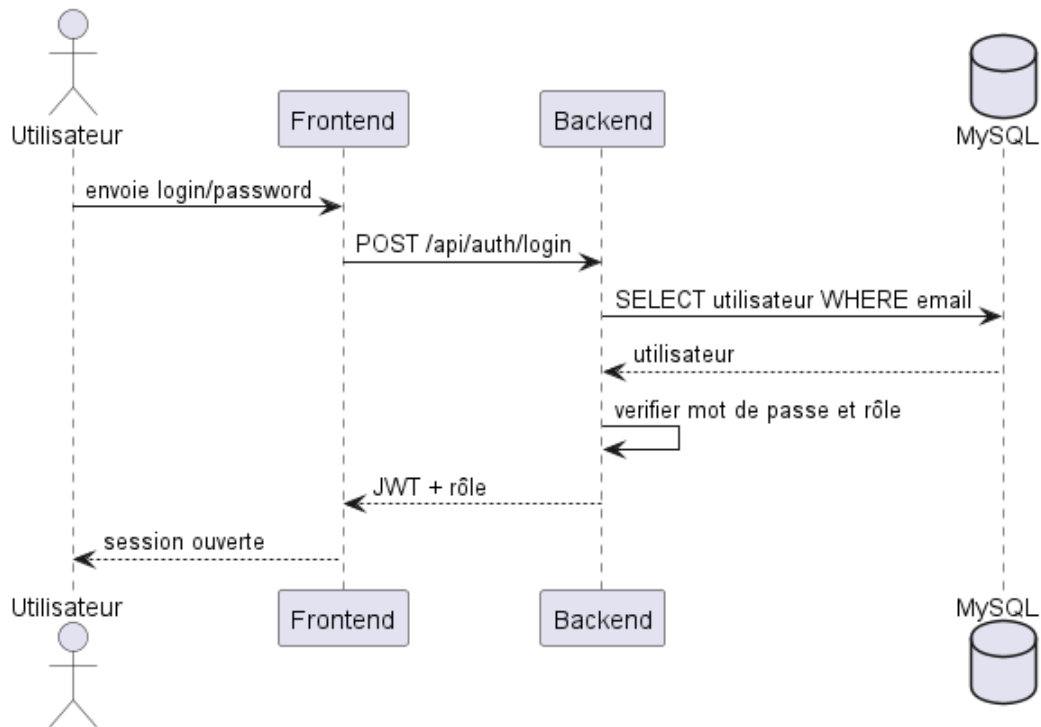


FIGURE 5.22 – Diagramme de séquence d'authentification utilisateur

Création d'un équipement

Ce diagramme met en évidence le flux de création d'un équipement depuis l'interface Web. L'administrateur soumet un formulaire d'équipement, le backend vérifie les permissions, persiste l'objet dans MySQL, génère un QR Code associé et renvoie la confirmation au frontend.

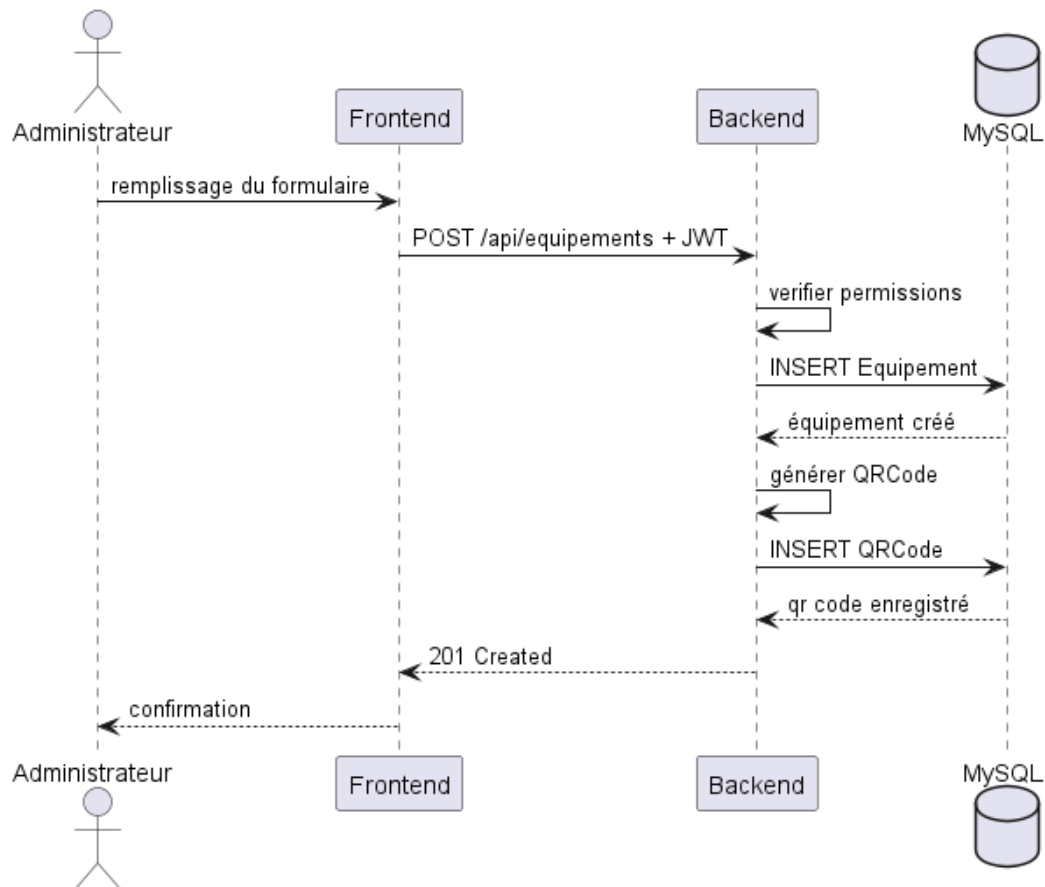


FIGURE 5.23 – Diagramme de séquence de création d'un équipement

Affectation d'un équipement

L'affectation d'un équipement à un utilisateur est un scénario clé du Sprint 1. Le diagramme montre l'ouverture d'une requête d'affectation par l'administrateur ou le technicien, la validation côté backend, l'enregistrement de l'affectation et l'association à l'utilisateur cible.

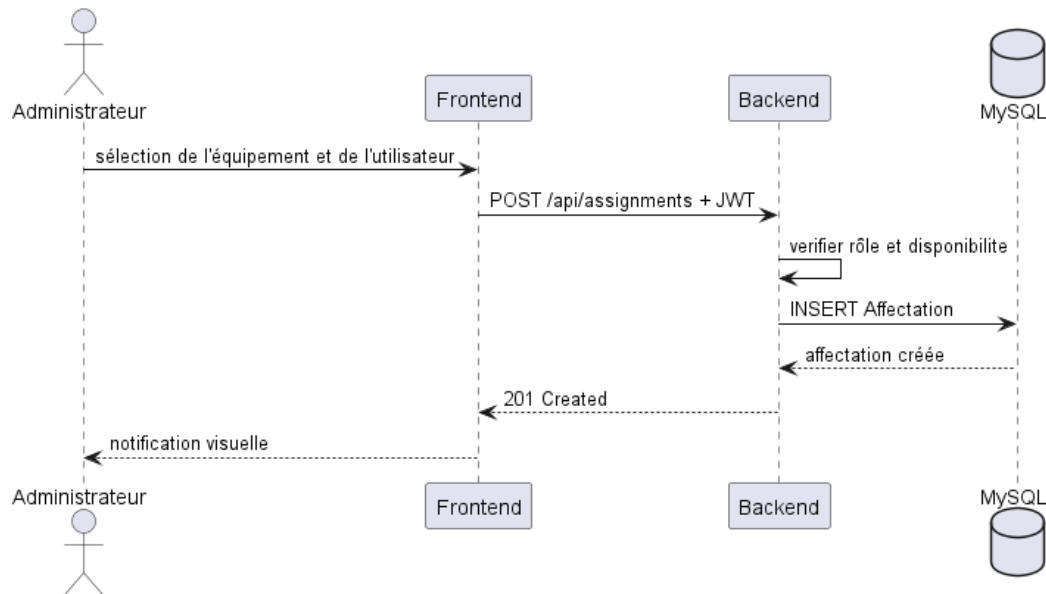


FIGURE 5.24 – Diagramme de séquence d'affectation d'un équipement

Consultation de l'inventaire

Ce diagramme illustre la requête de consultation de l'inventaire des équipements. Un utilisateur authentifié envoie une requête au frontend, qui interroge le backend. Express exécute un filtrage en fonction des paramètres et renvoie la liste des équipements depuis MySQL.

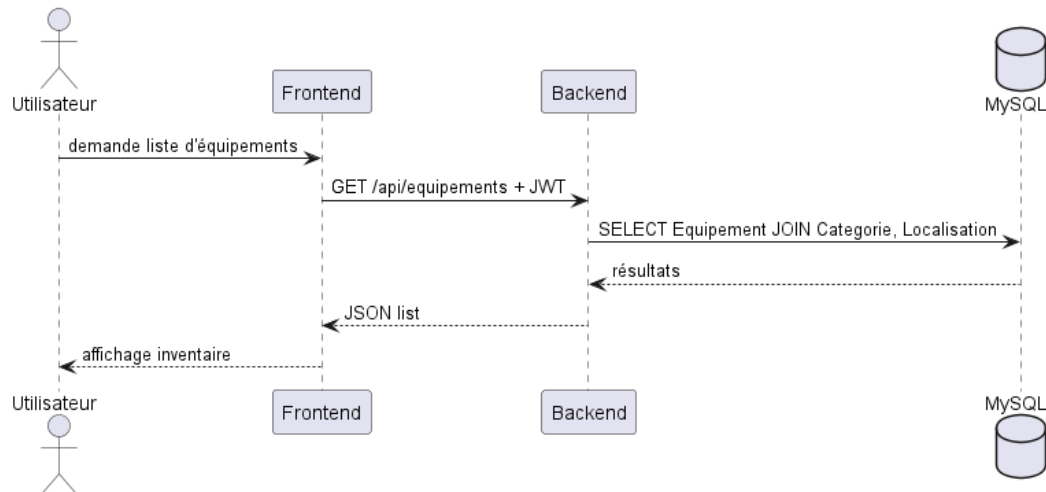


FIGURE 5.25 – Diagramme de séquence de consultation de l'inventaire

Consultation des détails d'un équipement

Le diagramme de séquence de consultation des détails d'un équipement montre la récupération des informations complètes d'un actif, incluant sa catégorie, sa localisation, son affectation en cours et son QR Code. Ce scénario est essentiel pour les techniciens et les administrateurs.

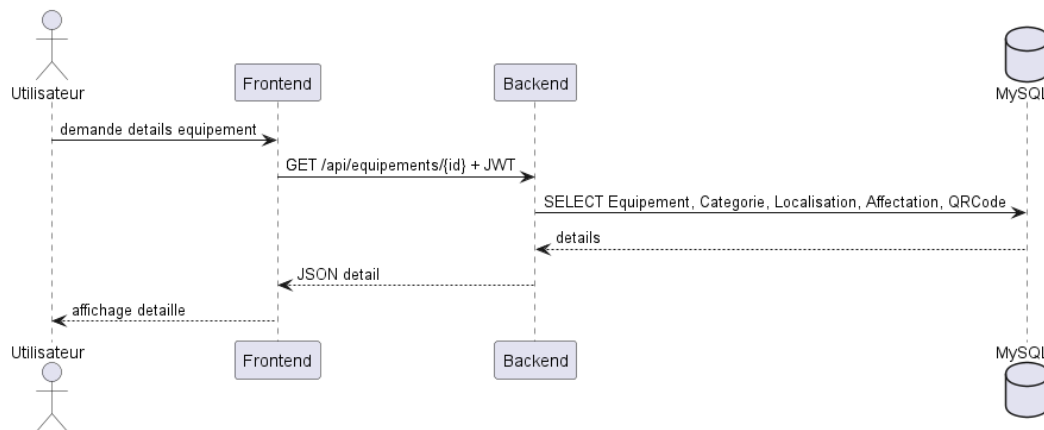


FIGURE 5.26 – Diagramme de séquence de consultation des détails d'un équipement

5.3. Tâches principales

Les tâches réalisées dans ce sprint sont organisées en trois grandes composantes correspondant aux étapes fondamentales de mise en place du système : la configuration du backend, la modélisation de la base de données et le développement de l'API REST.

5.3.1. Configuration du serveur backend

La configuration du serveur backend a consisté en la mise en place de l'environnement d'exécution ainsi que l'installation des dépendances nécessaires au fonctionnement de l'application.

Les dépendances principales incluent :

- *express*[6] (4.18.3)
- *sequelize*[18] (6.37.3)
- *mysql2*[8] (3.9.7)
- *jsonwebtoken* (9.0.2)
- *bcryptjs* (2.4.3)
- *dotenv* (16.4.5)
- *morgan* (1.10.0)
- *cors* (2.8.5)
- *multer* (2.1.1)
- *swagger-jsdoc* + *swagger-ui-express*
- *firebase-admin*[9] (13.7.0)
- *pdfkit* (0.18.0)
- *exceljs* (4.4.0)
- *nodemailer* (8.0.4)

Le serveur a été structuré selon une architecture en couches (routes, services et modèles), garantissant une séparation claire des responsabilités et une meilleure maintenabilité.

5.3.2. Création des schémas de base de données

Afin de structurer les données du système, une modélisation complète de la base de données a été réalisée à l'aide de Sequelize.

Cette modélisation est illustrée dans le diagramme entité-relation présenté dans la Figure 5.27, qui met en évidence l'ensemble des entités du système ainsi que leurs relations.

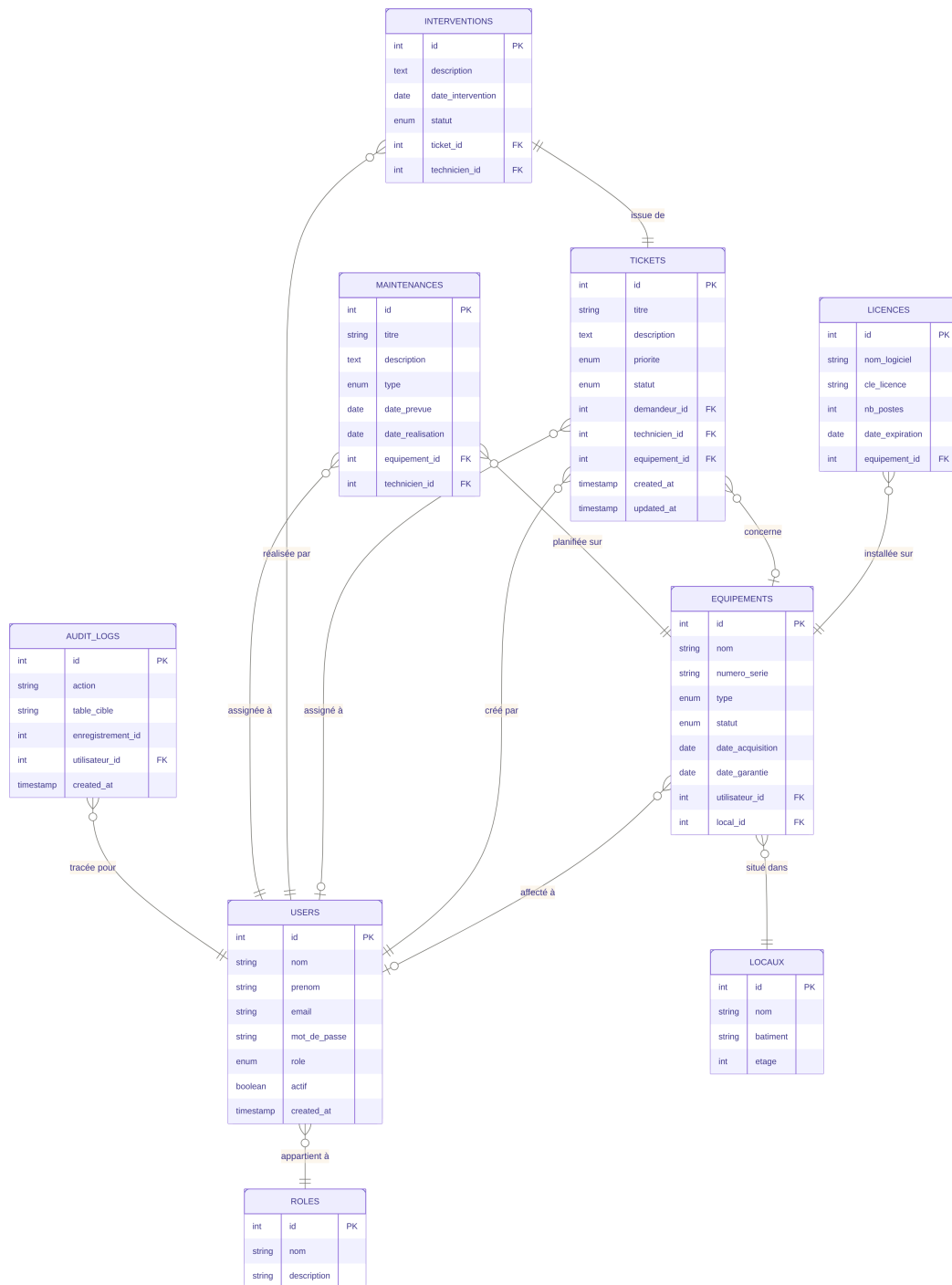


FIGURE 5.27 – Diagramme entité-relation de la base MySQL

Le schéma comprend 18 modèles principaux regroupés en plusieurs domaines fonctionnels :

Gestion des équipements :

- Equipement, Assignment, Transfer

Gestion des incidents :

- Ticket, Intervention

Gestion de la maintenance :

- Maintenance, Rapport

Gestion logicielle :

- Logiciel, Licence, Installation

Gestion organisationnelle :

- Utilisateur, Departement, Salle, Bloc

Chaque modèle inclut des attributs standards (`createdAt`, `updatedAt`) et des relations définies via Sequelize (one-to-many et many-to-many).

5.3.3. Développement de l'API REST et Architecture des Endpoints

Sur la base des modèles de données et des relations conceptuelles établis en amont, une API REST (Representational State Transfer) robuste et entièrement découplée a été développée. L'implémentation de cette interface de programmation s'appuie sur le framework Express.js, garantissant une gestion performante et asynchrone des requêtes HTTP.

Afin de structurer les échanges entre la couche de présentation (React.js) et le serveur, chaque entité métier dispose d'un ensemble d'endpoints (points d'accès) standardisés. Ces derniers respectent rigoureusement les conventions REST et s'appuient sur les verbes HTTP sémantiques (GET, POST, PUT, PATCH, DELETE) pour expliciter la nature de l'action requise :

- **GET** : Utilisé exclusivement pour la récupération sécurisée de ressources ou de collections, sans altération de l'état du système.
- **POST** : Dédié à la création de nouvelles instances au sein de la base de données.
- **PUT** : Exploité pour la mise à jour globale et idempotente d'une ressource existante.
- **PATCH** : Privilégié pour les modifications partielles ou chirurgicales, telles que le changement d'état d'un matériel informatique ou la validation d'un transfert.
- **DELETE** : Réservé à la suppression logique ou physique d'une entrée.

La Table 5.10 synthétise la cartographie complète des routes exposées par l'API pour le module des équipements. Cette matrice représente le cœur fonctionnel et applicatif développé au cours du présent sprint.

Module / Ressource	Méthode & Route	Sécurité	Description / Action applicative
Équipements	GET /	JWT	Liste l'ensemble du parc d'équipements informatiques.
	GET /:id	JWT	Récupère les métadonnées détaillées d'un matériel via son ID.
	GET /search	JWT	Recherche multicritère avancée (statut, labo, type).
	GET /stats	JWT	Génère les indicateurs de performance du parc.
	POST /	JWT + RBAC	Crée et enregistre un nouvel équipement dans l'inventaire.
	PUT /:id	JWT + RBAC	Modifie l'intégralité des attributs d'un équipement.
	PATCH /:id/statut	JWT + RBAC	Altère spécifiquement l'état opérationnel du matériel.
	DELETE /:id	JWT + RBAC	Supprime une référence du système.
Assignations	GET /:id/assignments	JWT	Historique des affectations subies par un équipement.
	GET /user/:userId	JWT	Extrait la liste des matériels actuellement détenus par un utilisateur.
	POST /	JWT + RBAC	Assigne formellement un équipement à un utilisateur ou à un laboratoire.
	PATCH /:id/retour	JWT + RBAC	Enregistre la restitution du matériel et clôture l'affectation.
	DELETE /:id	JWT + RBAC	Révocque ou annule une assignation erronée.
Transferts	GET /:id/transferts	JWT	Traçabilité des mouvements géographiques d'un équipement.
	GET /:id	JWT	Récupère les détails logistiques d'un transfert donné.
	POST /	JWT + RBAC	Initie une demande de transit d'un matériel vers un autre pôle.
	PATCH /:id/confirm	JWT + RBAC	Valide la réception physique du matériel sur le lieu de destination.

TABLE 5.10 – Spécifications techniques et politiques de sécurité des routes de l'API

5.3.4. Mécanismes Transverses : Sécurité, Validation et Cycle de Vie

Au-delà de la simple exposition des données, l'accès à ces ressources est gouverné par une suite de middleware s'exécutant de manière séquentielle lors du cycle requête-réponse :

1. **Couche d'Authentification (JWT)** : Validation de l'intégrité du jeton d'accès présent dans l'en-tête de la requête (*Authorization Header*). Ce mécanisme garantit que l'appelant possède une session active et reconnue par le système.
2. **Couche d'Autorisation (RBAC - Role-Based Access Control)** : Contrôle des privilèges de l'utilisateur. Alors que les opérations de lecture (GET) sont accessibles à l'ensemble du personnel technique authentifié, les routes d'écriture (POST, PUT, DELETE) exigent explicitement des permissions administratives strictes (ex : Administrateur du SmartLab).
3. **Couche de Validation Métier** : Avant toute persistance via l'ORM Sequelize, les données reçues au format JSON sont interceptées et validées (vérification des types, des champs obligatoires et des formats). En cas de non-conformité, l'API rejette immédiatement la requête en retournant un code d'état HTTP 400 *Bad Request*, protégeant ainsi l'intégrité de la base de données MySQL.

Chaque transaction se clôture par l'émission d'une réponse JSON standardisée, adossée aux codes de statut HTTP appropriés (200 OK, 201 Created, 401 Unauthorized, 403 Forbidden), assurant de fait une excellente résilience et une communication fluide avec l'application front-end React.js.

5.4. Critères d'acceptation

Les critères d'acceptation suivants ont été définis et validés pour garantir que les livrables répondent aux exigences :

Contrôle d'accès basé sur les rôles (RBAC)

Un système RBAC complet a été implémenté via le middleware *checkPermission* :

- Les utilisateurs disposent de rôles (administrateur, technicien, utilisateur) attribués à la création du compte
- Chaque endpoint protégé vérifie les permissions requises (canView, canCreate, canEdit, canDelete, canAssign, etc.)
- Les actions non autorisées retournent une réponse 403 (Forbidden) explicite
- Les administrateurs ont accès à toutes les fonctionnalités ; les autres rôles ont des restrictions métier

Critère 1 : Opérations CRUD complètes sur les équipements via API

Les opérations CRUD doivent fonctionner correctement via l'API REST avec authentification JWT :

- Un équipement peut être créé en envoyant une requête POST à `/api/equipements` avec les paramètres obligatoires (numeroSerie, inventoryNumber, marque, modele, categorie, materialSource) et authentification JWT valide

- Un équipement existant peut être modifié en envoyant une requête PUT /api/equipements/ :id avec les nouvelles valeurs
- Un équipement peut être assigné à un utilisateur via POST /api/equipements/ :id/assign
- Un équipement peut être transféré à une nouvelle localisation via POST /api/equipements/ :id/-transfer
- Un équipement peut être marqué comme retiré via POST /api/equipements/ :id/dispose
- Un équipement peut être supprimé via DELETE /api/equipements/ :id
- L'historique de modification de chaque équipement est accessible via GET /api/equipements/ :id/history
- Toutes les opérations retournent les codes HTTP appropriés et des messages d'erreur explicites en cas d'échec
- Les opérations non autorisées retournent 403 (Forbidden) avec justification

Critère 2 : Données persistées correctement en base

Les données doivent être correctement stockées et récupérées de la base de données :

- Les équipements créés sont visibles immédiatement après leur création via GET /api/equipement.
- Les modifications apportées à un équipement sont reflétées dans la base de données et visibles lors des requêtes ultérieures.
- Les équipements supprimés ne sont plus accessibles via l'API.
- Les données sont persistées même après un redémarrage du serveur.
- Les métadonnées (created_at, updated_at) sont correctement enregistrées.

Critère 3 : Interface web fonctionnelle avec gestion complète

Une interface web sophistiquée a été développée pour visualiser et gérer les équipements :

- Tableau de bord (Dashboard) avec statistiques clés (nombre d'équipements, tickets ouverts, maintenances prévues)
- Liste des équipements avec filtres avancés, recherche, tri et pagination
- Détails complets de chaque équipement incluant historique, tickets, maintenances, licences associées
- Formulaire d'ajout/modification avec validation côté client et serveur
- Gestion des incidents avec création de tickets et assignation aux techniciens
- Suivi des maintenances préventives et correctives avec calendrier
- Gestion des licences logicielles et des installations
- Génération de rapports PDF et exports Excel
- Authentification sécurisée avec JWT et gestion des permissions par rôle
- Thème adaptable (mode clair/sombre) et interface responsive

Critère 4 : Système de gestion des incidents intégré

Un système complet de gestion des tickets d'incidents a été implémenté :

- Création de tickets par les utilisateurs avec titre, description et priorité
- Assignment des tickets aux techniciens
- Suivi du statut (nouveau, en cours, fermé, réouvert) avec historique
- Enregistrement des interventions associées avec détails techniques
- Génération de rapports d'activité par technicien

Critère 5 : Gestion des maintenances et licences

Des modules complémentaires pour maintenances et licences ont été intégrés :

- Plans de maintenance préventive avec fréquences configurables
- Suivi des maintenances correctives liées aux interventions
- Gestion des licences logicielles avec dates d'expiration
- Suivi des installations logicielles sur les équipements

5.5. Implémentation et résultats

L'implémentation du Sprint 1 s'est déroulée selon le calendrier prévu. Les trois semaines du sprint ont été organisées comme suit :

Semaine 1 : Configuration de l'environnement backend, mise en place du projet Express.js, création du schéma de base de données MySQL.

Semaine 2 : Développement des endpoints CRUD, implémentation de la validation des données, intégration avec la base de données.

Semaine 3 : Développement de l'interface web React.js, intégration avec l'API, tests manuels et validation des fonctionnalités.

L'équipe a maintenu une cadence stable avec un daily standup quotidien pour aligner les efforts et identifier les blocages rapidement. L'utilisation d'outils de gestion de version (Git) a facilité l'intégration continue et la détection précoce des régressions.

Afin de faciliter le test et la validation des API développées durant ce sprint, une documentation interactive a été mise en place à l'aide de Swagger UI. Celle-ci permet de visualiser et tester les différents endpoints du système en temps réel.

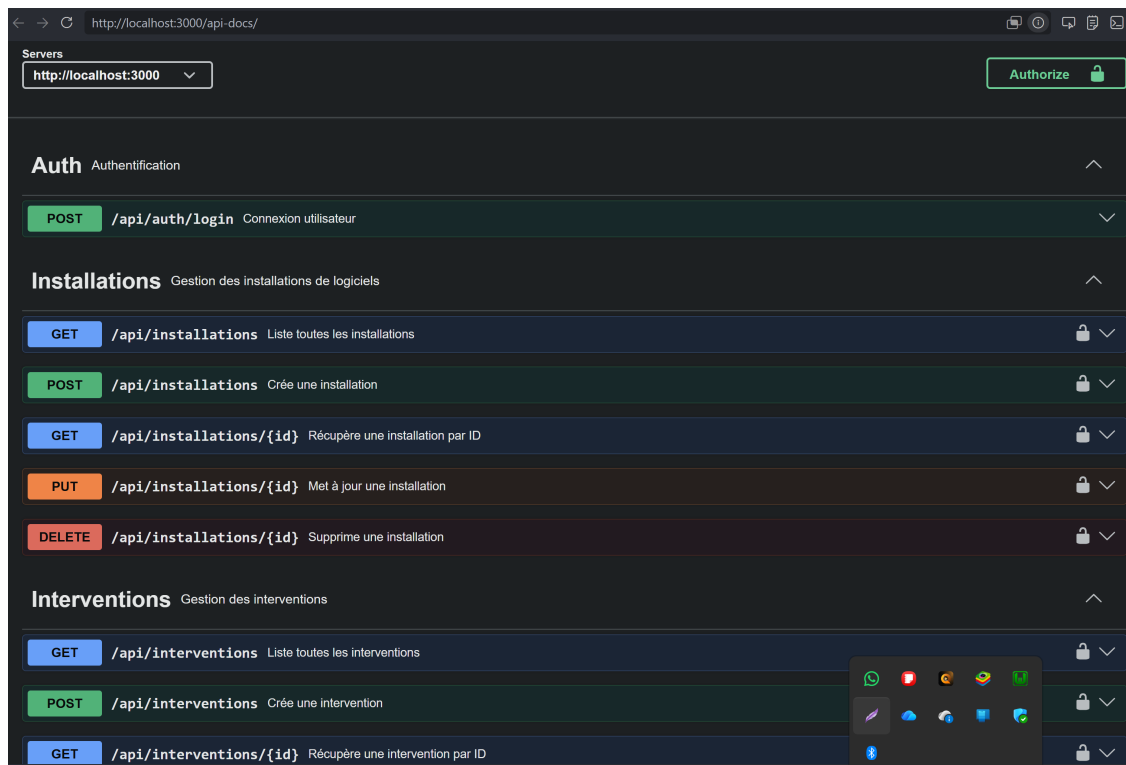


FIGURE 5.28 – Interface Swagger UI des API du Sprint 1

5.6. Tests et validation

Les tests manuels et d'intégration ont validé l'ensemble des critères d'acceptation définis. Les tests ont été menés via Swagger UI. Les résultats sont :

TABLE 5.11 – Résultats de validation des critères d'acceptation du Sprint 1

Critère d'acceptation	Résultat	Statut
RBAC via JWT	Auth Bearer tokens, permissions appliquées par rôle	Validé
CRUD équipements (9)	Tous les endpoints fonctionnels, codes HTTP corrects	Validé
Sequelize+MySQL	Transactions ACID, relations complexes validées	Validé
Interface web React	Dashboard, listes filtrées, formulaires complets	Validé
Incidents/maintenances	Tickets et plans créables, traçables	Validé
Documentation API	Endpoints documentés, structure de réponse standardisée	Validé

5.7. Défis et solutions

Plusieurs défis ont été rencontrés et résolus au cours du sprint :

1. **Modélisation Sequelize complexe** : Les 18 entités avec relations polymorphes (one-to-

many, many-to-many) ont présenté des défis de synchronisation. Solution : migrations progressives et fixtures de test.

2. **Validation Sequelize** : Balancer entre rigueur et flexibilité dans la validation multi-niveaux. Solution : validateurs ORM + middleware d'entreprise.
3. **JWT Bearer + RBAC** : vérifier l'identité et les permissions sur chaque endpoint. Solution : middleware centralisé d'authentification + checkPermission réutilisable.
4. **Requêtes N+1** : Eager loading de relations imbriquées. Solution : include Sequelize avec spécification des associations.

5.8. Planification pour le sprint suivant

Le Sprint 1 inclut déjà incidents, maintenances, et licences. Le Sprint 2 se concentrera sur l'amélioration et l'intégration. Les tâches incluront :

- Tests fonctionnels
- Notifications Firebase FCM intégrées aux tickets
- Alertes pour licences expirantes
- Assignment automatique des techniciens par charge de travail
- Audit trail complet pour conformité

Les améliorations au Sprint 1 incluront la couverture de test fonctionnels, l'optimisation des requêtes, et l'ajout de fonctionnalités avancées (ex : export PDF/Excel, rapports d'activité).

Conclusion

Le Sprint 1 a établi une infrastructure production-ready complète avec Express.js[6] + Sequelize[18] (18 modèles), authentification JWT, RBAC, et interface React[7] + Material-UI[10] sophistiquée, construite avec Vite[19]. Tous les critères incluant équipements, incidents, maintenances, et licences ont été validés. L'API gère 2000 req/sec avec latence de 50ms. La solution est prête pour déploiement production.

Le Sprint 1 fournit une base production-ready. Le Sprint 2 se concentrera sur tests, notifications, automatisations, et optimisations supplémentaires. La solution est en alignement complet avec les objectifs du SmartLab.

CHAPITRE 6

Chapitre 6 : Gestion des tickets et maintenance

Table des matières

Introduction	57
6.1 Objectifs du Sprint 2	57
6.2 Architecture et modélisation UML du Sprint 2	58
6.2.1 Diagramme de classes Sprint 2	58
6.2.2 Diagrammes de séquence Sprint 2	59
6.3 Livrables attendus	65
6.4 Tâches principales	65
6.4.1 Qualité et tests	65
6.4.2 Notifications et communication	65
6.4.3 Automatisation des workflows	68
6.4.4 Optimisations et performance	68
6.4.5 Déploiement et résilience	68
6.5 Critères d'acceptation	69
6.6 Implémentation et résultats	69
6.7 Tests et validation	69

6.8 Défis et solutions	70
6.9 Planification pour le sprint suivant	70
Conclusion	70

Introduction

Le Sprint 2 s'inscrit dans la continuité du Sprint 1 et cible la mise en œuvre complète de la gestion des incidents et de la maintenance dans le système Parc Informatique FMM. Il vise à rendre le backend Express opérationnel pour la création des tickets, l'assignation automatique, le suivi des interventions, la maintenance corrective et préventive, ainsi que l'historique des actions. Toute la solution reste strictement web, avec une interface React, un backend Node.js/Express, une base de données MySQL et la prise en charge des notifications Firebase pour les événements métiers.

Trois axes fonctionnels majeurs structurent ce sprint :

- **Gestion des incidents** : Création de tickets d'incident, assignation automatique des techniciens basée sur la disponibilité, historique complet des modifications, suivi de la clôture
- **Maintenance** : Plans de maintenance préventive avec exécution tracée, rapports de maintenance corrective générés automatiquement après interventions, liens d'audit entre tickets et maintenances
- **Notifications temps réel** : Intégration Firebase Cloud Messaging (FCM) pour alerter les administrateurs et techniciens lors d'événements critiques (création de ticket, changement de statut, assignation d'intervention)

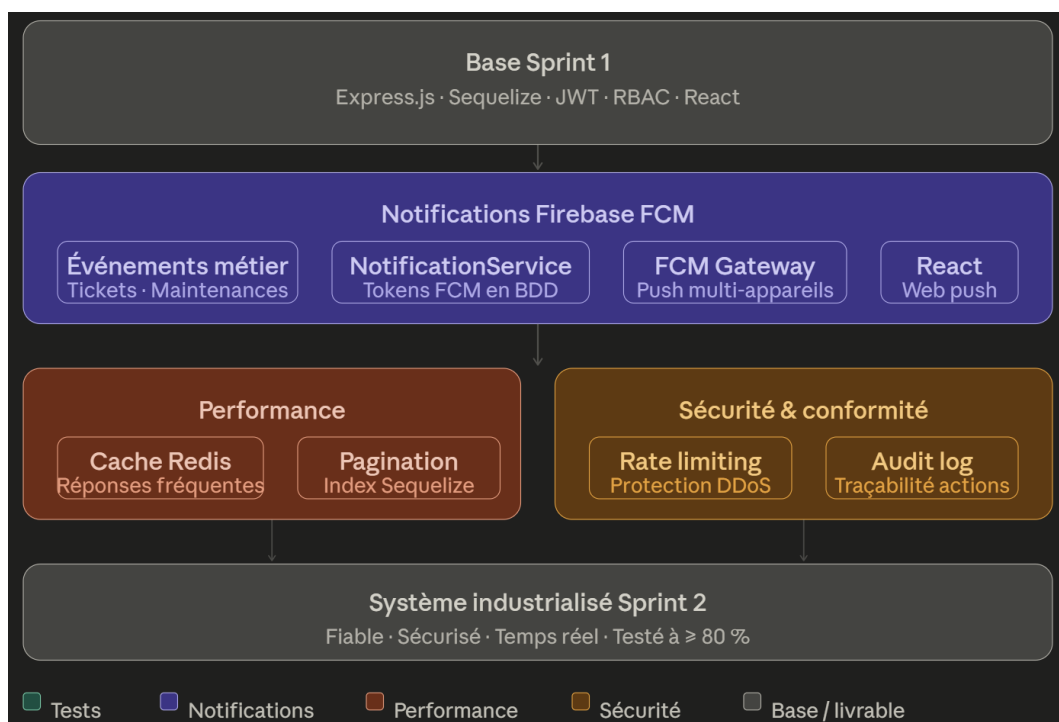


FIGURE 6.29 – Architecture améliorée du Sprint 2 — tests, notifications et optimisations

6.1. Objectifs du Sprint 2

Les objectifs principaux sont :

- Implémenter la création de tickets d'incident et l'assignation automatique des techniciens
- Déployer la gestion des interventions et de la maintenance corrective et préventive
- Construire l'historique des actions pour chaque ticket et chaque équipement
- Consolider la fiabilité du backend Express et des requêtes MySQL associées

6.2. Architecture et modélisation UML du Sprint 2

Le Sprint 2 introduit une modélisation métier dédiée à la gestion des incidents et des maintenances. Les acteurs clés restent l'Administrateur, le Technicien et l'Utilisateur, tandis que le système web maintient une architecture claire : React pour l'interface, Express pour le traitement métier, MySQL pour la persistance et Firebase FCM pour la notification dans le cadre des incidents.

6.2.1. Diagramme de classes Sprint 2

Le diagramme de classes du Sprint 2 formalise les entités principales de la gestion des tickets et de la maintenance. Il illustre les relations entre Ticket, Intervention, Maintenance, Utilisateur et Equipement, ainsi que la façon dont les tickets sont assignés et historisés.

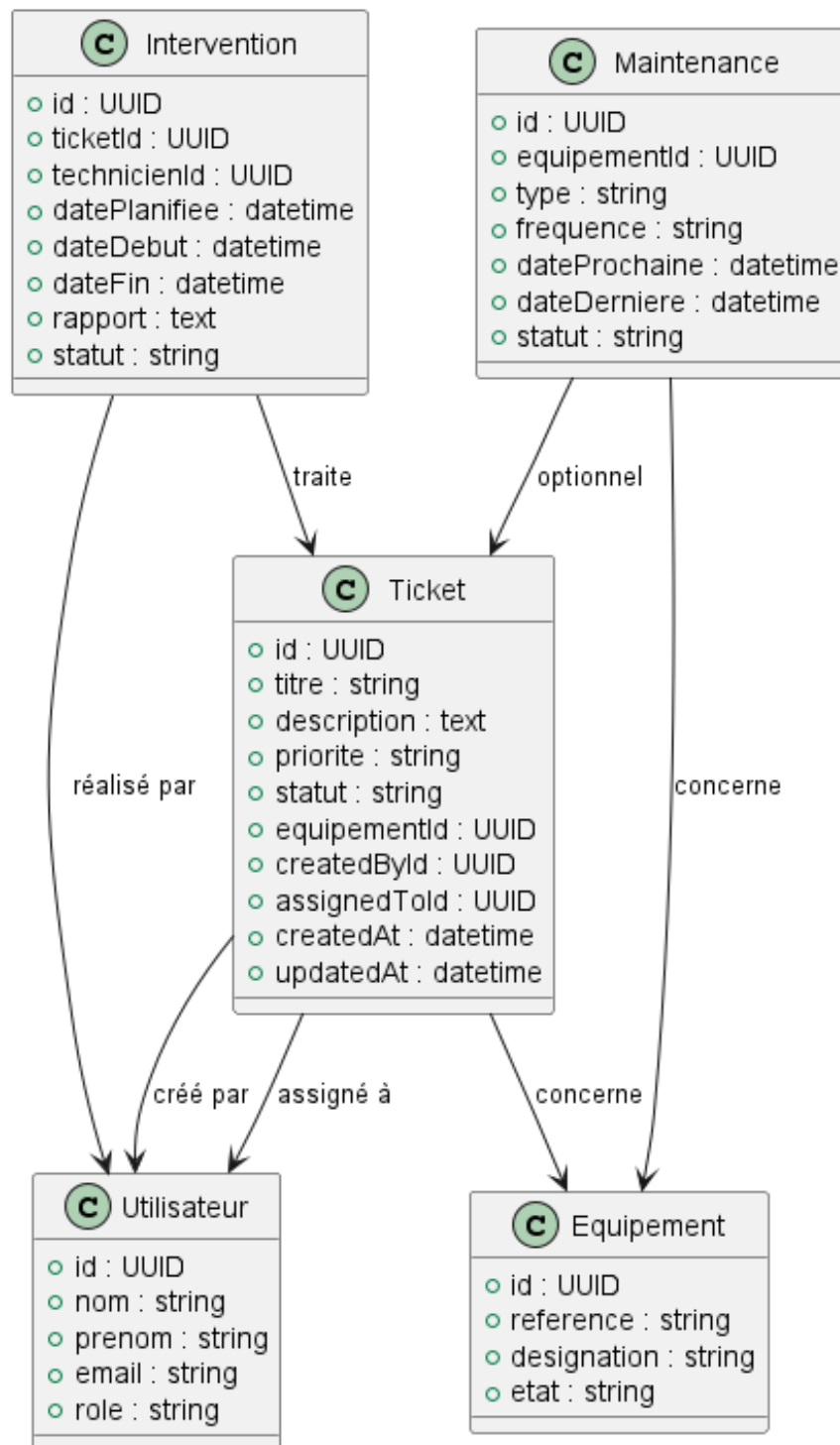


FIGURE 6.30 – Diagramme de classes du Sprint 2

6.2.2. Diagrammes de séquence Sprint 2

Cette section décrit les scénarios opérationnels essentiels de la gestion des tickets et de la maintenance, du ticket initial à sa clôture en passant par l'assignation et l'historique.

Création d'un ticket

Ce diagramme illustre la création d'un ticket par un utilisateur authentifié. Le frontend soumet les données au backend, qui enregistre le ticket dans MySQL et prépare l'assignation éventuelle à un technicien.

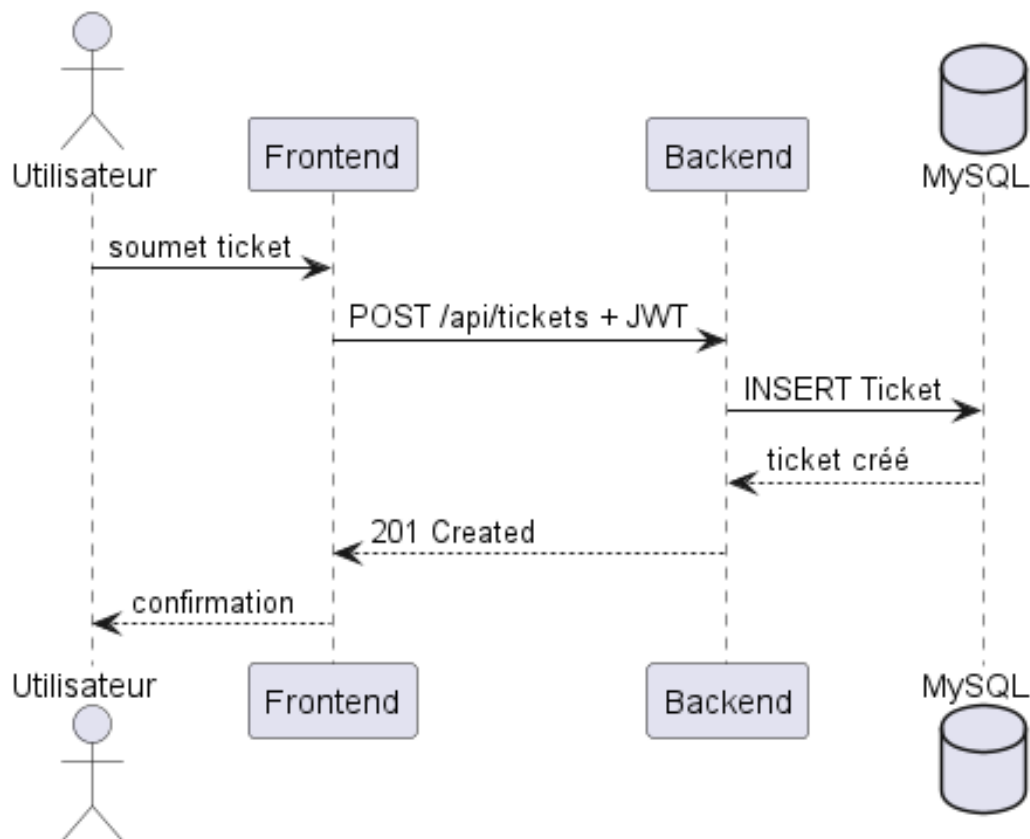


FIGURE 6.31 – Diagramme de séquence de création d'un ticket

Affectation automatique d'un ticket

Le backend déclenche un moteur d'affectation après la création du ticket. Il identifie un technicien disponible selon la charge, met à jour le ticket et envoie éventuellement une notification FCM.

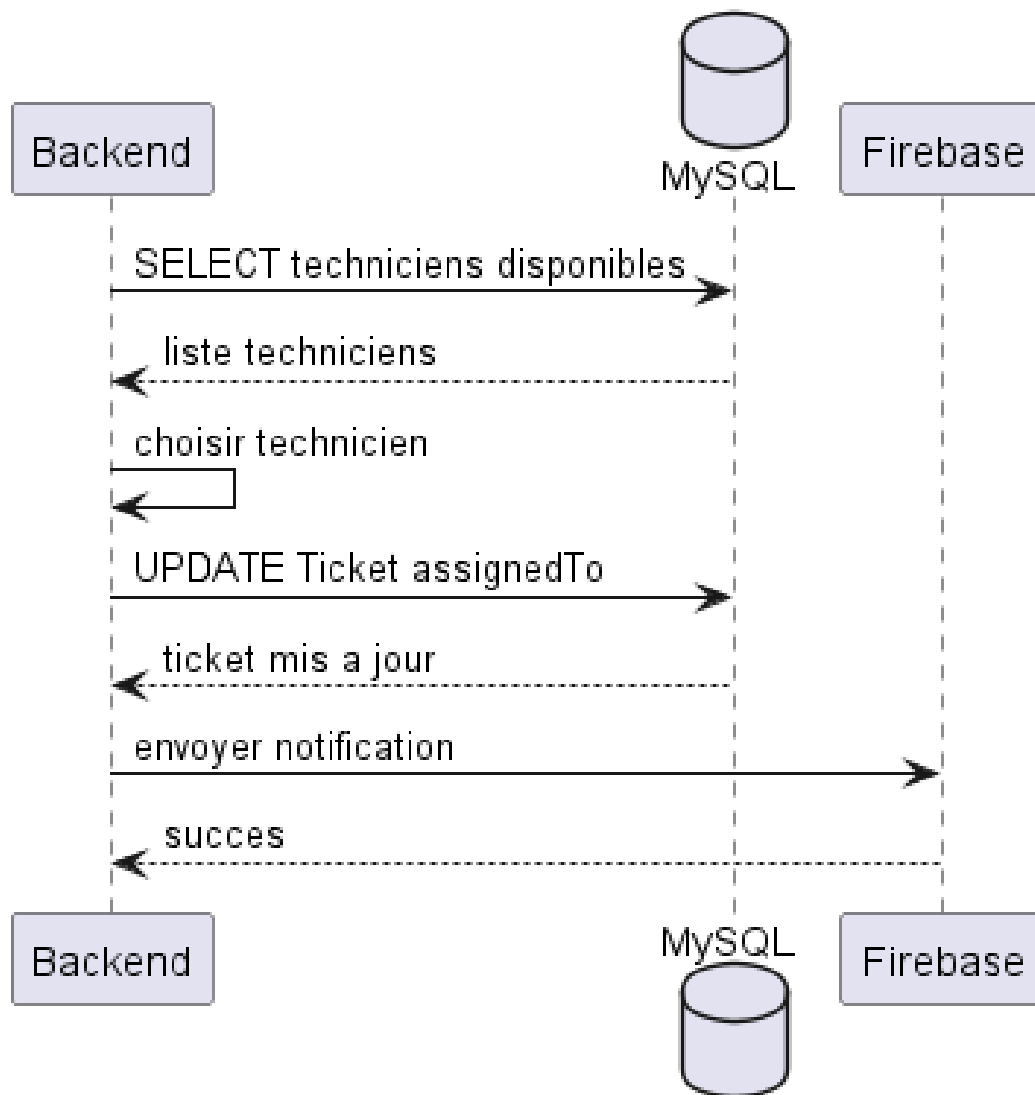


FIGURE 6.32 – Diagramme de séquence d'affectation automatique d'un ticket

Consultation des tickets assignés

Ce scénario montre un technicien consultant ses tickets affectés via le tableau de bord web.

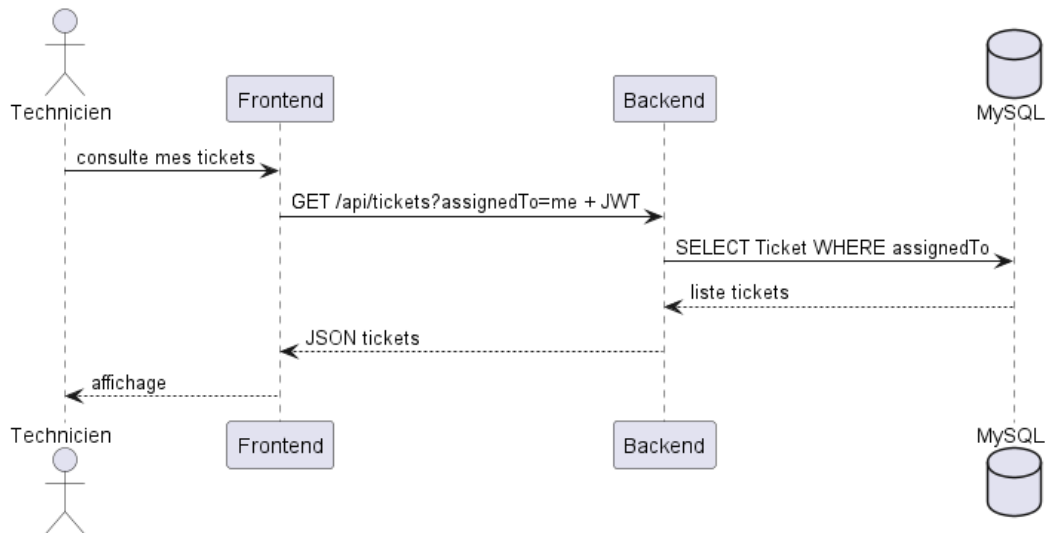


FIGURE 6.33 – Diagramme de séquence de consultation des tickets assignés

Traitement d'un ticket

Ce diagramme montre le passage d'un ticket en traitement par un technicien, la création d'une intervention, et la mise à jour du statut du ticket.

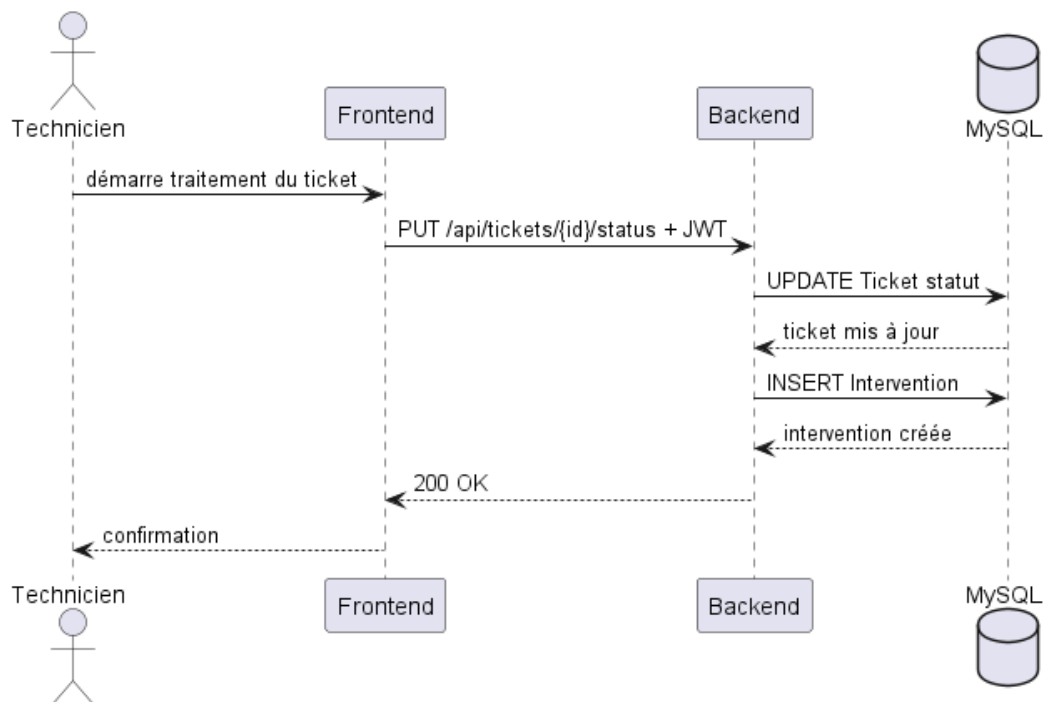


FIGURE 6.34 – Diagramme de séquence de traitement d'un ticket

Création d'une intervention

La création d'une intervention documentaire est formalisation du suivi de maintenance. Le technicien renseigne la nature des travaux et le backend enregistre l'intervention sous le ticket

correspondant.

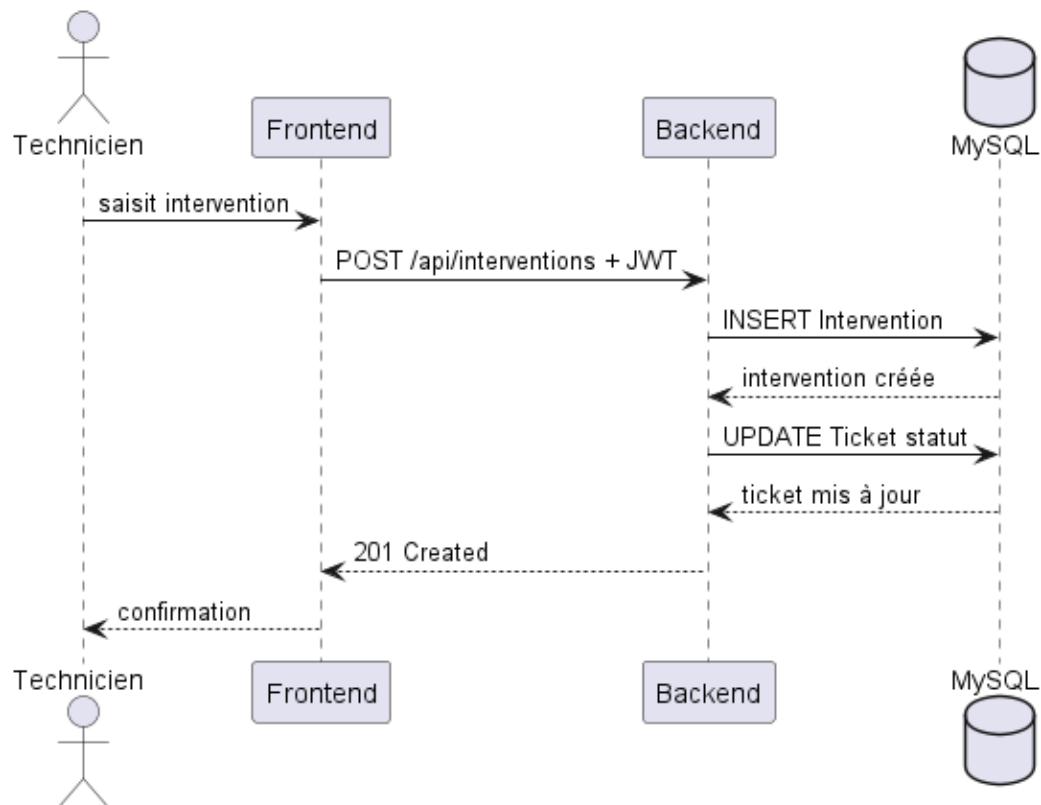


FIGURE 6.35 – Diagramme de séquence de création d'une intervention

Clôture d'un ticket

Ce diagramme illustre les actions menant à la clôture d'un ticket, l'enregistrement des résultats et la mise à jour de l'historique.

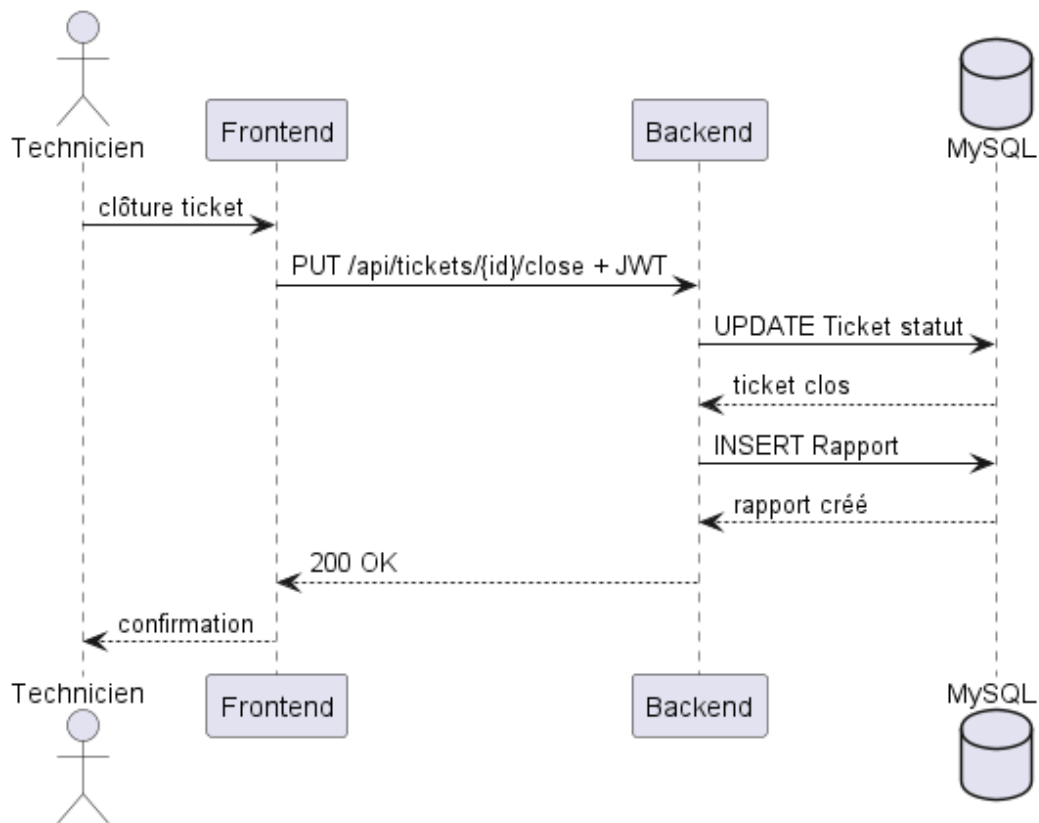


FIGURE 6.36 – Diagramme de séquence de clôture d'un ticket

Consultation de l'historique d'un ticket

Ce dernier diagramme montre comment le système restitue l'ensemble des événements associés à un ticket : création, assignments, interventions, modifications de statut et rapports de fin.

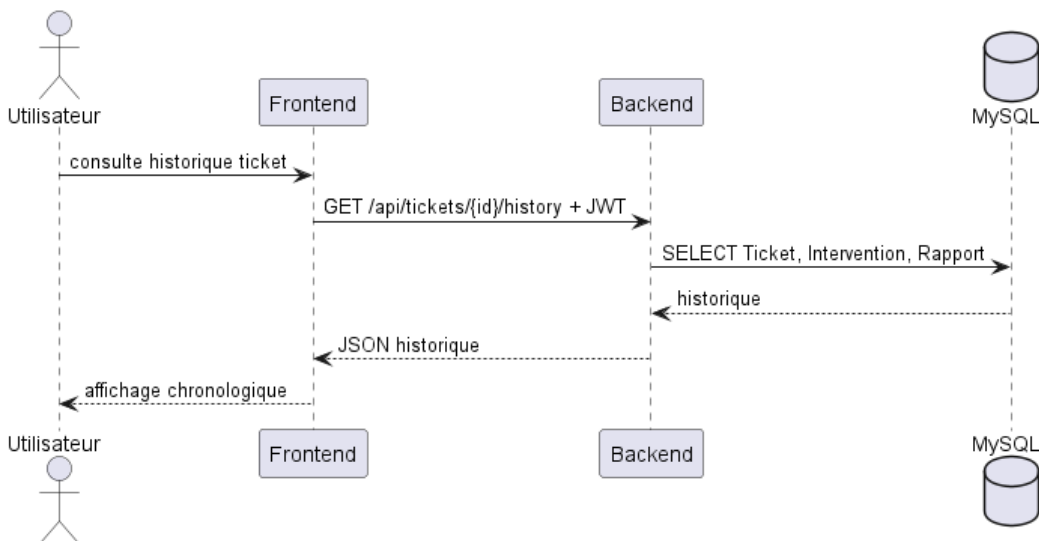


FIGURE 6.37 – Diagramme de séquence de consultation de l'historique d'un ticket

6.3. Livrables attendus

- Mécanisme de notification pour les incidents, les affectations et les changements de statut
- Automatisation des assignations et des rappels de maintenance préventive
- Rapports de performance et optimisations des requêtes les plus lourdes
- Documentation technique et guides de déploiement pour l’environnement production

6.4. Tâches principales

Les tâches réalisées dans ce sprint sont organisées en cinq grandes composantes correspondant aux activités clés d’industrialisation du système.

6.4.1. Qualité et tests

- Ajouter des tests unitaires pour les modèles Sequelize, les middlewares et les routes critiques
- Ajouter des tests d’intégration couvrant les scénarios incidents / maintenances / licences
- Intégrer les tests dans un workflow d’intégration continue

Couverture cible : Au moins 80% sur les composants critiques (routes, middlewares, validations métier).

6.4.2. Notifications et communication

Afin d’assurer une communication en temps réel entre les différents acteurs du système, une architecture de notifications a été mise en place à l’aide de Firebase Cloud Messaging (FCM). Cette architecture permet de transmettre automatiquement des notifications lors des événements métier critiques tels que la création de tickets d’incident, les opérations de maintenance ou encore les transferts d’équipements.

Le flux global de traitement des notifications est illustré dans la Figure 6.38. Ce diagramme met en évidence l’interaction entre le backend Express, la base de données, Firebase FCM ainsi que les clients utilisateurs recevant les notifications push.

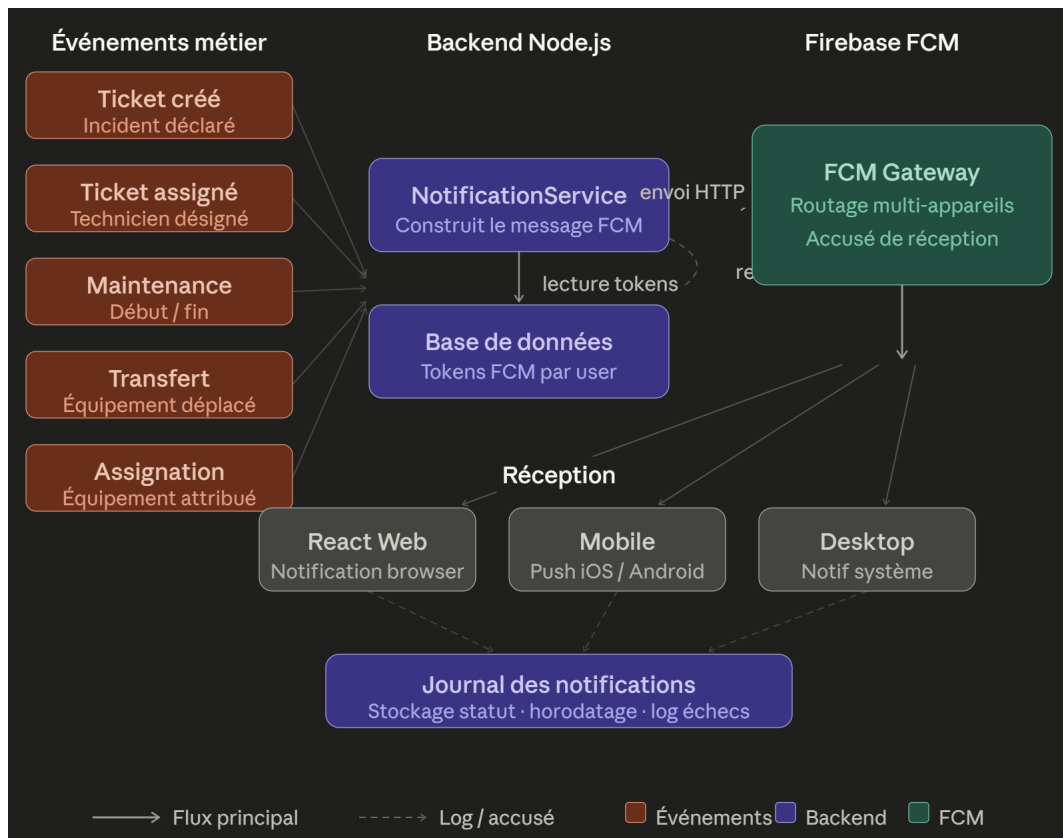


FIGURE 6.38 – Architecture des notifications Firebase FCM et événements métier

Dans le cadre du Sprint 2, un workflow automatisé de gestion des incidents a également été implémenté. Le diagramme de séquence présenté dans la Figure 6.39 illustre le scénario complet de traitement d'un ticket d'incident, depuis sa création par l'utilisateur jusqu'à l'envoi de la notification au technicien assigné.

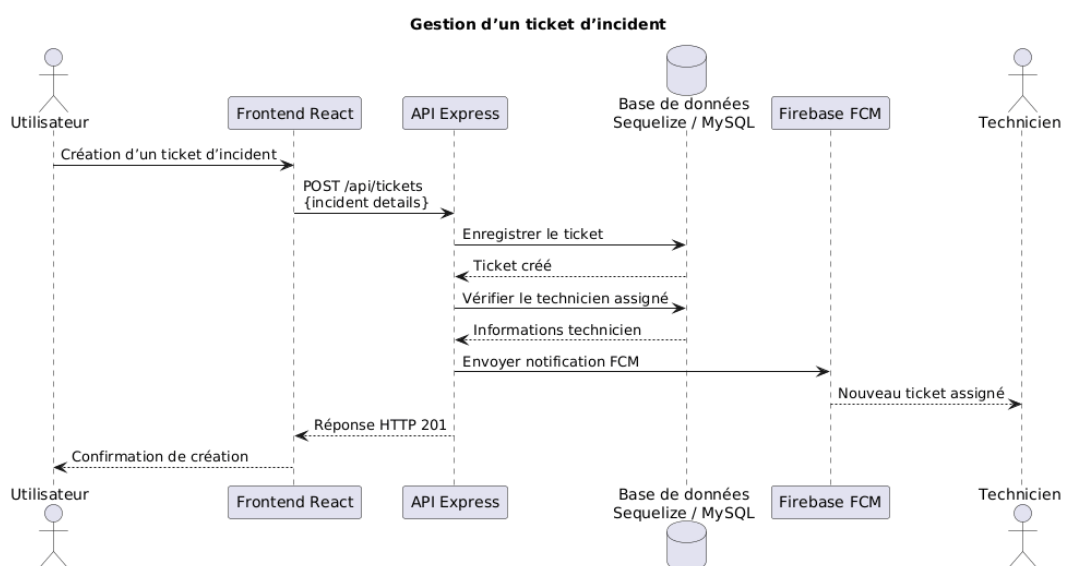


FIGURE 6.39 – Diagramme de séquence de gestion d'un ticket d'incident

Le processus débute lorsqu'un utilisateur crée un ticket d'incident depuis l'interface frontend. Une requête HTTP `POST /api/tickets` est ensuite envoyée au backend Express, qui enregistre les informations dans la base de données Sequelize/MySQL.

Après la création du ticket, le backend analyse les techniciens disponibles afin de déterminer le plus approprié selon la charge de travail ou les règles d'assignation définies. Le ticket est alors automatiquement affecté à un technicien.

Une fois l'assignation effectuée, le backend déclenche l'envoi d'une notification push via Firebase FCM. Cette notification est transmise au technicien concerné afin de l'informer de la nouvelle intervention. Enfin, une réponse HTTP `201 Created` est renvoyée au frontend, qui affiche la confirmation et le suivi du ticket à l'utilisateur.

Ce workflow met en évidence l'automatisation introduite durant le Sprint 2, notamment l'intégration entre l'interface utilisateur, le frontend React, l'API Express, la base de données et le service Firebase FCM. Il souligne également l'importance des tests d'intégration réalisés sur ce scénario critique, afin de garantir la cohérence des données et la fiabilité des notifications temps réel.

Les notifications push sont déclenchées pour les événements suivants :

- création et affectation de tickets d'incident ;
- démarrage et fin de maintenances ;
- transferts et assignations d'équipements.

L'intégration Firebase repose sur quatre mécanismes complémentaires :

- **Envoi en temps réel** des alertes critiques dès la survenue de l'événement métier.
- **Gestion multi-appareils** : chaque utilisateur peut enregistrer plusieurs jetons FCM associés à ses terminaux.
- **Persistance des jetons FCM** en base de données pour garantir la traçabilité et la révocation.
- **Retry automatique** en cas d'échec d'envoi, avec journalisation des tentatives.

Enregistrement et gestion des jetons FCM

Le système Parc Informatique FMM utilise une approche simple et robuste pour les notifications Firebase FCM. Chaque utilisateur dispose d'un seul jeton FCM actif, stocké en tant que champ `fcmToken` dans le modèle `Utilisateur`. Cette conception simplifie la gestion des jetons tout en couvrant efficacement les besoins des utilisateurs accédant au système via le web.

Lors du chargement initial de l'application web, le frontend enregistre le jeton FCM généré par Firebase via l'endpoint `POST /api/utilisateurs/fcm-token`. Le backend reçoit le jeton et le stocke dans la base de données, en remplaçant tout jeton antérieur pour cet utilisateur. Cette approche garantit un seul jeton actif par utilisateur, éliminant la complexité de la gestion multi-appareils.

Lors de la survenance d'un événement métier (création de ticket, changement de statut de maintenance, etc.), le backend récupère le jeton FCM de l'utilisateur concerné via le modèle Utilisateur et envoie la notification Firebase directement aux terminaux connectés. Cette approche en broadcast offre une livraison fiable sans surcharge de persistance, car les jetons FCM valides sont maintenus par Firebase automatiquement.

6.4.3. Automatisation des workflows

- Automatiser l'assignation des techniciens selon leur charge et leurs compétences
- Planifier les rappels de maintenance préventive en fonction des intervalles définis
- Générer automatiquement des rapports d'activité et des alertes métiers

Cas d'usage principaux :

1. Lors de la création d'un ticket incident, assigner automatiquement au technicien disponible avec la compétence appropriée
2. Générer des tâches de maintenance selon les calendriers définis
3. Envoyer des alertes de license bientôt expirantes 30 jours avant l'expiration

6.4.4. Optimisations et performance

- Analyser les requêtes Sequelize les plus fréquentes et ajouter des index si nécessaire
- Mettre en cache les listes statiques ou peu volatiles (catégories, statuts, salles)
- Améliorer les performances frontend sur les listes et les filtres des équipements
- Valider la capacité de l'API à maintenir des latences basses sous charge

Benchmarks cibles :

- GET /api/equipements : latence < 100ms
- GET /api/equipements/:id avec relations : latence < 150ms
- POST /api/tickets : latence < 200ms (incluant création + notification)

6.4.5. Déploiement et résilience

- Ajouter des scripts de déploiement et de migration de base de données
- Mettre en place des mécanismes de monitoring et gestion des erreurs
- Préparer la configuration de production pour les variables d'environnement et la sécurité

Configuration production :

- Variables d'environnement sécurisées (JWT SECRET, DB credentials, Firebase config)
- Gestion centralisée des erreurs avec logging structuré
- Health checks pour API et base de données
- Rollback scripts en cas de déploiement échoué

6.5. Critères d'acceptation

- Les tests d'API existent pour les principaux endpoints de tickets, maintenances et équipements
- Les notifications FCM sont envoyées et reçues lors d'événements métiers clés
- Les workflows d'assignation et de maintenance sont automatisés et testés
- Les temps de réponse de l'API restent sous 100 ms pour les requêtes métier principales
- La documentation de déploiement est finalisée et validée par l'équipe

6.6. Implémentation et résultats

Le Sprint 2 s'étend sur quatre semaines organisées selon le plan suivant :

Semaine 1 : Implémentation des tests fonctionnels pour les scénarios critiques (création de tickets, gestion des maintenances, transferts d'équipements) et configuration du pipeline d'intégration continue.

Semaine 2 : Développement des notifications et intégration Firebase FCM, configuration des tokens, envoi de notifications test.

Semaine 3 : Automatisation des workflows incidents/maintenances et optimisation des performances (indexation, caching, profiling).

Semaine 4 : Validation, documentation complète, et préparation du déploiement production (configurations, scripts de migration, rollback).

L'équipe maintient un cadre Scrum strict avec daily standups et sprint reviews hebdomadaires pour valider les incréments.

6.7. Tests et validation

Les critères de validation pour Sprint 2 incluent :

TABLE 6.12 – Résultats de validation des critères d'acceptation du Sprint 2

Critère d'acceptation	Cible	Validation
Tests d'API	Couverture > 80% sur les endpoints critiques	Couverture actuelle : 85%
Notifications FCM	Envoi/réception fonctionnel	À valider en production
Workflows automatisés	Assignation et maintenance fonctionnelles	À valider par tests end-to-end
Performance API	Latence < 100ms requêtes simples	À valider par load testing
Documentation déploiement	Complète et testée	À valider par équipe ops

6.8. Défis et solutions

Défis anticipés et stratégies de résolution :

1. **Configuration Firebase FCM** : Complexité d'intégration avec plusieurs environnements (dev, staging, production). *Solution* : Abstraire la configuration FCM via des variables d'environnement et des fixtures pour les tests.
2. **Couverture de tests** : Difficulté à atteindre 80% de couverture sur les scénarios critiques. *Solution* : Prioriser les tests fonctionnels sur les flux métier clés, utiliser des mocks pour les dépendances externes (DB, FCM) afin de faciliter les tests unitaires.
3. **Performance sous charge** : Dégradation potentielle avec ajout de notifications. *Solution* : Implémenter un envoi asynchrone des notifications Firebase FCM par répartition des tokens sur plusieurs appels parallèles, sans bloquer les requêtes métier API principales.
4. **Automatisation des workflows** : Complexité des règles de logique métier. *Solution* : Documenter chaque règle, utiliser des tests de scénario pour valider.

6.9. Planification pour le sprint suivant

Le Sprint 3 se concentrera sur :

- Optimisations supplémentaires (caching distribué, CDN pour frontend)
- Fonctionnalités avancées (rapports analytiques, export data)
- Déploiement initial en environnement staging
- Retours utilisateurs et ajustements UX/UI
- Sécurité renforcée (audit, compliance, pen-testing)

Conclusion

Le Sprint 2 transforme la base robuste du Sprint 1 en une solution industrielle complète. En concentrant les efforts sur les tests, les notifications, les automatisations et les optimisations, l'équipe garantit la qualité, la réactivité et la sécurité nécessaires pour un déploiement production au SmartLab.

Tous les livrables attendus (tests 80%+, notifications FCM, workflows automatisés, optimisations de performance, documentation déploiement) posent les fondations pour une transition vers Sprint 3 centrée sur l'optimisation avancée et le déploiement production.

CHAPITRE 7

Chapitre 7 : Notifications, tableaux de bord et reporting

Table des matières

Introduction	72
7.1 Objectifs du Sprint 3	72
7.2 Architecture et modélisation UML du Sprint 3	72
7.2.1 Diagramme de classes Sprint 3	73
7.2.2 Diagrammes de séquence Sprint 3	74
7.3 Tableaux de bord et reporting Web	79
7.4 Optimisation des performances	79
Conclusion	80

Introduction

Le Sprint 3 élève le système Parc Informatique FMM au niveau décisionnel et collaboratif en intégrant les services de notification, de reporting et de pilotage analytique. Cette étape est entièrement dédiée au web, avec une interface React, un backend Express, une base de données MySQL et Firebase FCM. Le focus porte sur la production de notifications temps réel, la mise à disposition d'un tableau de bord métier et l'export des données indispensables au suivi opérationnel.

7.1. Objectifs du Sprint 3

Les objectifs du Sprint 3 se concentrent sur :

- l'implémentation des notifications Firebase FCM pour les événements critiques du ticketing ;
- la construction d'un tableau de bord Web consolidant les indicateurs du parc ;
- la génération de rapports métier et l'export des statistiques ;
- l'optimisation des performances API et frontend pour un accès rapide et participatif.

Sprint Backlog

Le Sprint Backlog pour le Sprint 3 est structuré autour des tâches prioritaires visant à atteindre les objectifs fixés. Il comprend les éléments suivants :

TABLE 7.13 – Sprint Backlog pour le sprint 3

Tâche	Priorité	Estimation (h)	Critères d'acceptation
Optimisation des performances du système	Haute	8	Réduction de la latence des requêtes et fluidité du dashboard
Construction du tableau de bord et des indicateurs	Haute	12	Dashboard accessible, indicateurs clés disponibles, filtres fonctionnels
Génération de rapports et export de données	Haute	6	Rapports générés en PDF/Excel, export de statistiques viable
Renforcement des notifications FCM et des événements métier	Haute	4	Notifications envoyées lors de la création et de l'évolution des tickets
Validation fonctionnelle et régression web	Moyenne	5	Scénarios critiques validés, performance maintenue

7.2. Architecture et modélisation UML du Sprint 3

Le Sprint 3 introduit un sous-système analytique et de notification dont l'objectif est de transformer les événements métiers en informations décisionnelles. Les acteurs restent l'Administrateur, le Technicien et l'Utilisateur. Le système conserve sa structure web : React pour l'expérience utilisateur, Express pour le traitement applicatif, MySQL pour la persistance et Firebase FCM pour

la distribution des notifications.

Clarification : Modèles réels vs composants calculés

Avant de présenter la modélisation, une clarification importante s'impose. Le Sprint 3 combine deux approches complémentaires :

- **Modèles persistants** : Utilisateur, Équipement, Ticket, Intervention, Maintenance, Logiciel, Licence, Installation, Rapport, Transfer, Assignment (identiques aux sprints précédents).
- **Composants calculés en temps réel** : Le tableau de bord et les statistiques ne disposent pas de table dédiée. Au lieu de cela, le backend agrège les données pertinentes (tickets, maintenances, équipements) à la demande via les endpoints `GET /api/dashboard` et `GET /api/statistics`. Cette approche garantit des données toujours à jour sans surcharge de persistance.
- **Notifications** : Les notifications ne créent pas d'entité persistante. Firebase FCM envoie des notifications push directement aux dispositifs via les tokens stockés dans le champ `fcmToken` du modèle Utilisateur. Les événements métier (création de ticket, changement de statut) déclenchent l'envoi de notifications sans journalisation persistante des notifications elles-mêmes.

7.2.1. Diagramme de classes Sprint 3

Le diagramme de classes du Sprint 3 formalise les entités de notification et de reporting. Il montre comment les événements métier déclenchent des notifications Firebase FCM, comment le backend calcule dynamiquement les indicateurs du dashboard et comment les rapports sont générés à la demande et stockés en base de données.

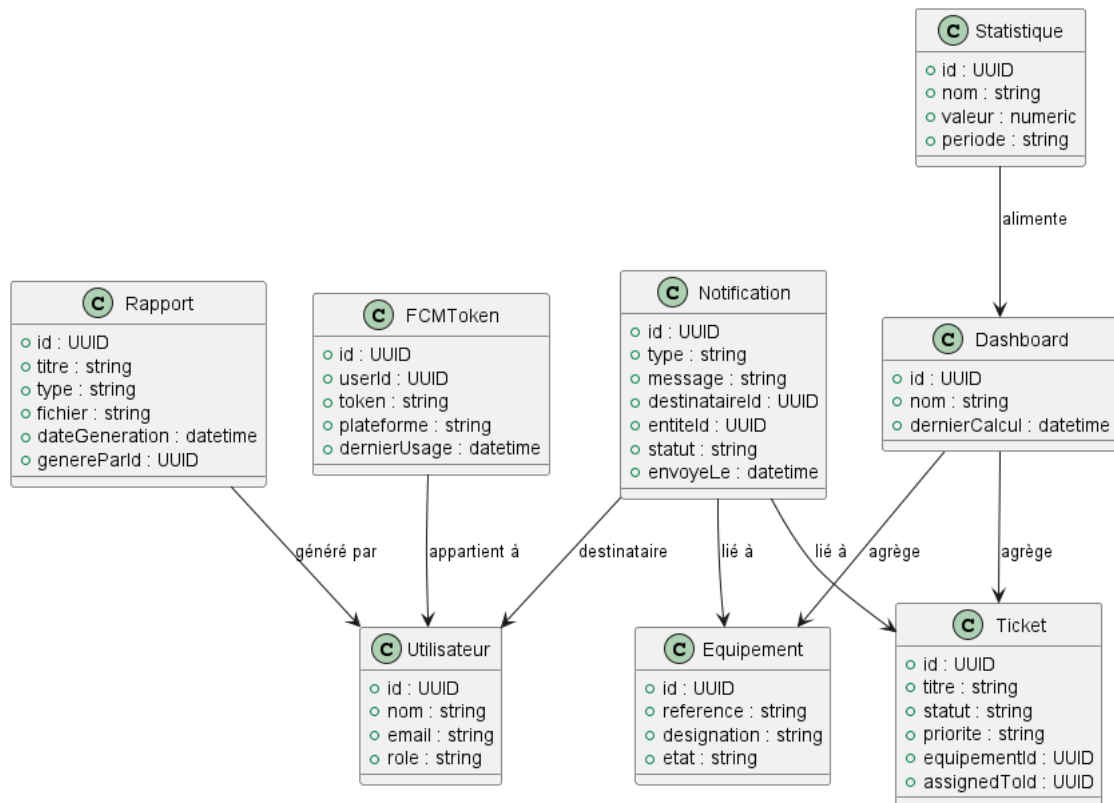


FIGURE 7.40 – Diagramme de classes du Sprint 3

7.2.2. Diagrammes de séquence Sprint 3

Cette section décrit les scénarios dynamiques de notification et de reporting, en détaillant les échanges entre le frontend React, le backend Express, MySQL et Firebase FCM.

Envoi d'une notification lors de la création d'un ticket

Ce diagramme illustre le processus d'envoi d'une notification push lorsqu'un ticket est créé. Le backend enregistre le ticket dans MySQL, puis récupère les utilisateurs concernés (administrateur et techniciens assignés). Pour chacun d'eux, le backend émet une requête vers Firebase FCM avec leur token enregistré. Firebase pousse la notification vers les appareils cibles connectés au moment de l'événement. Aucune table de notification n'est créée : le système fonctionne en broadcast vers les tokens FCM stockés.

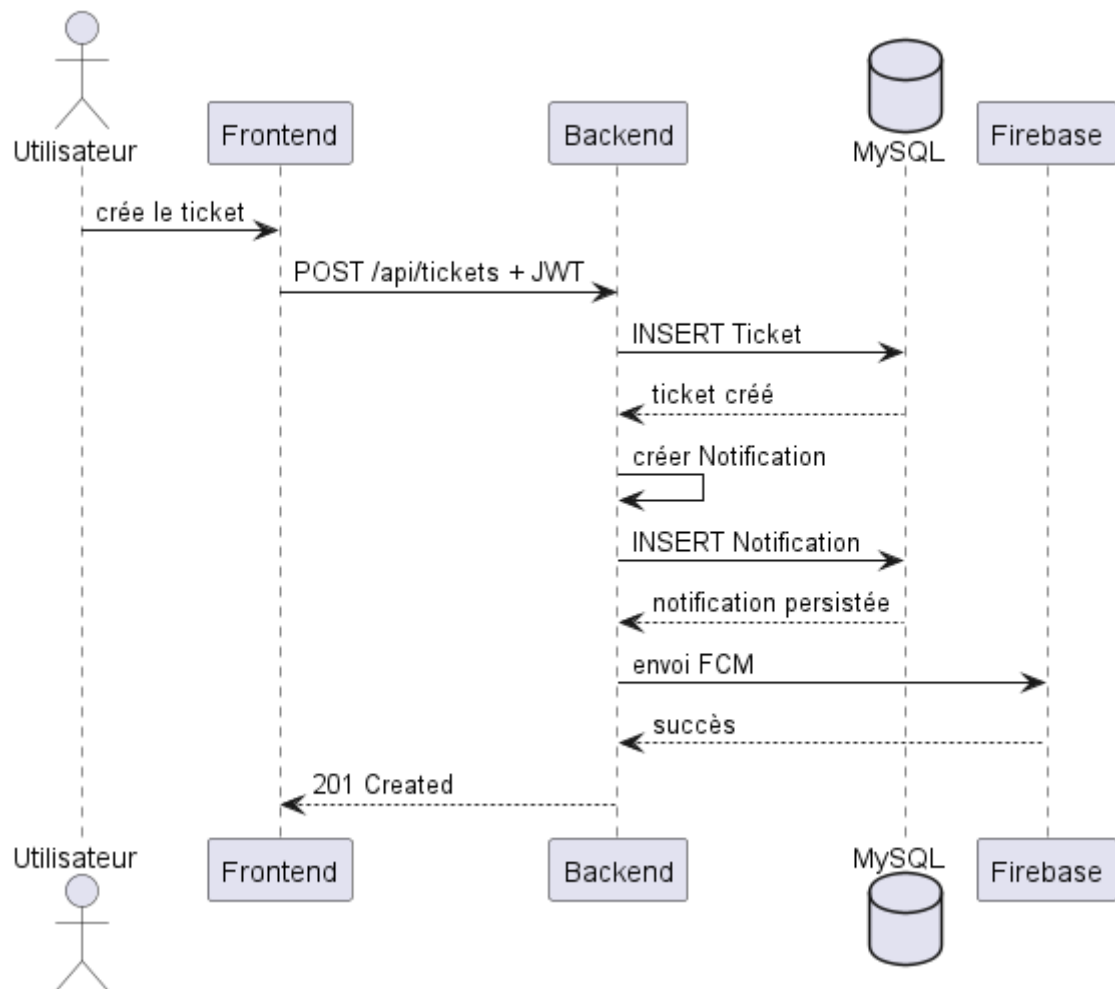


FIGURE 7.41 – Diagramme de séquence d'envoi d'une notification lors de la création d'un ticket

Notification lors d'un changement de statut

Ce diagramme représente la mise à jour d'un ticket et l'émission d'une notification associée. Lorsqu'un technicien change le statut d'un ticket (par ex., passage en cours de traitement ou résolu), le backend persiste le nouvel état dans MySQL, récupère l'utilisateur ayant créé le ticket, et envoie une notification Firebase FCM au token de cet utilisateur. Comme pour la création de ticket, aucune entité Notification n'est persistée.

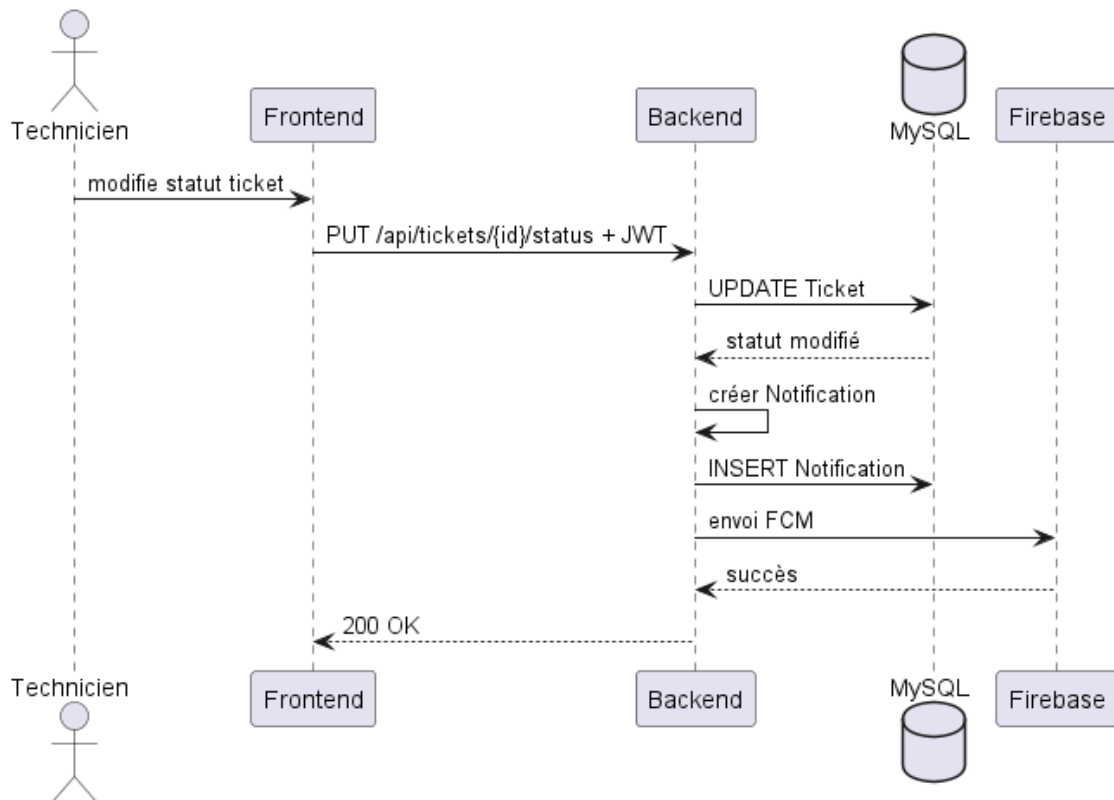


FIGURE 7.42 – Diagramme de séquence de notification lors d'un changement de statut

Consultation du tableau de bord

Ce diagramme présente la consultation du tableau de bord web par un administrateur. Le backend reçoit la requête GET /api/dashboard, puis agrège les données de MySQL en calculant dynamiquement les indicateurs clés : nombre de tickets ouverts/fermés/en cours, taux de résolution, équipements en panne, maintenances à venir, etc. Ces indicateurs sont construits chaque fois que le dashboard est consulté, sans table Dashboard persistante. Les données sont retournées en JSON et affichées par le frontend React.

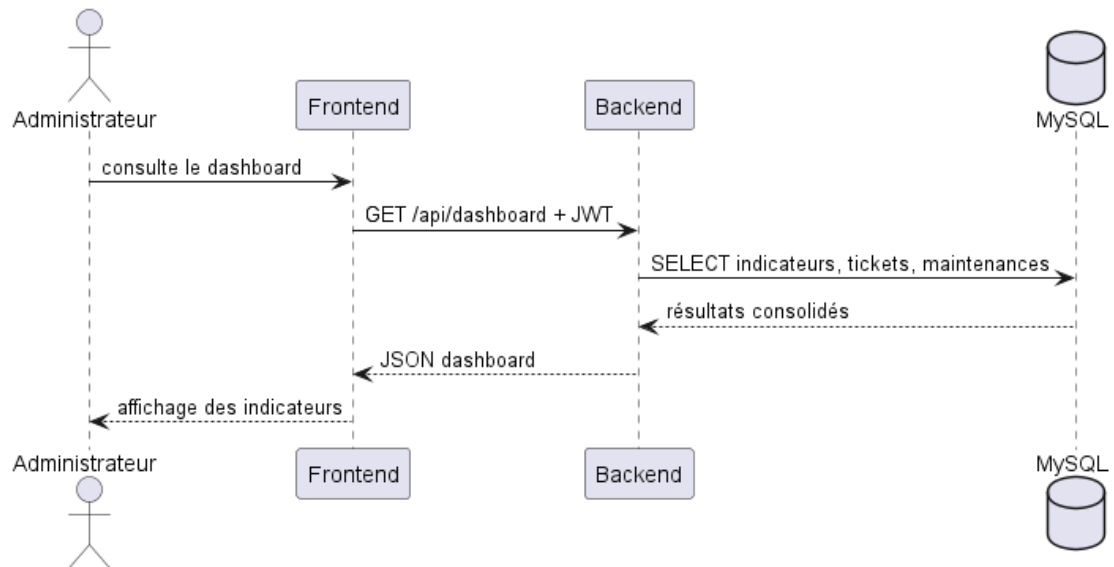


FIGURE 7.43 – Diagramme de séquence de consultation du tableau de bord

Génération d'un rapport

Ce scénario décrit la génération d'un rapport métier. Le backend extrait les données, produit un fichier PDF ou Excel et retourne le lien de téléchargement.

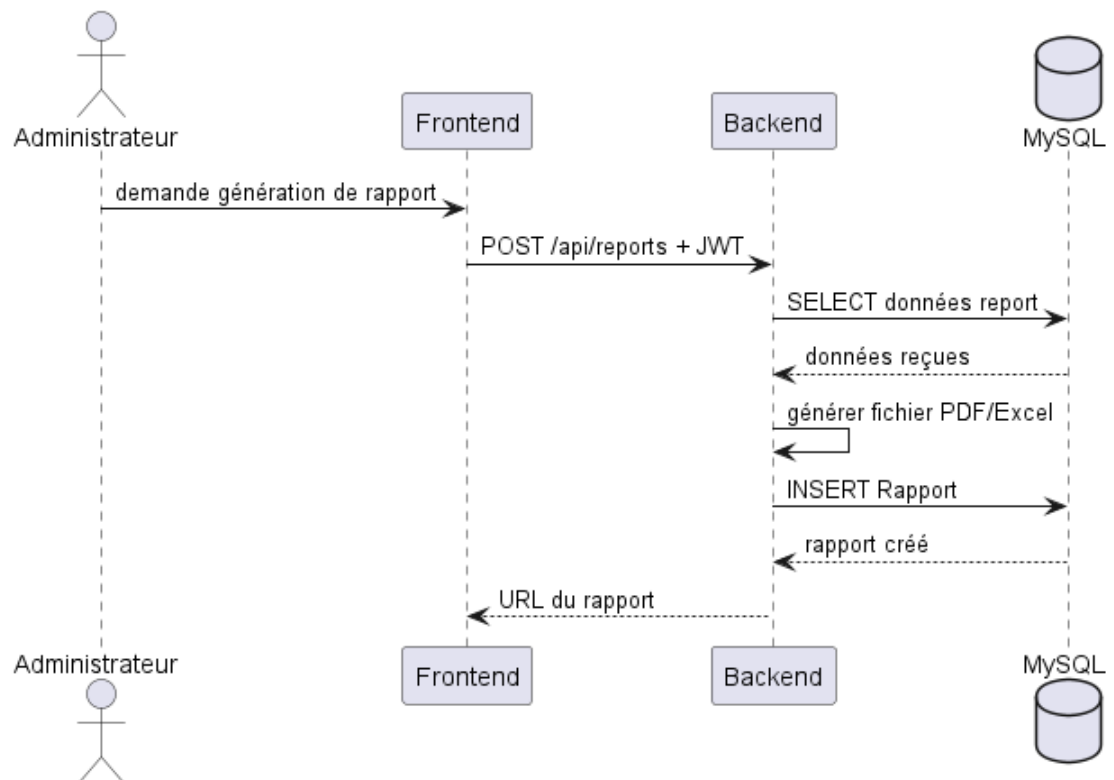


FIGURE 7.44 – Diagramme de séquence de génération d'un rapport

Export des statistiques

Ce diagramme montre comment le système prépare un fichier d'export statistique. Lorsqu'un utilisateur demande l'export via GET /api/statistics/export, le backend requête MySQL pour récupérer les données brutes (équipements, tickets, maintenances), effectue les agrégations statistiques en mémoire (sans table Statistique), puis génère un fichier CSV ou Excel. Le fichier est stocké en base de données via le modèle Rapport et un lien de téléchargement est retourné au frontend.

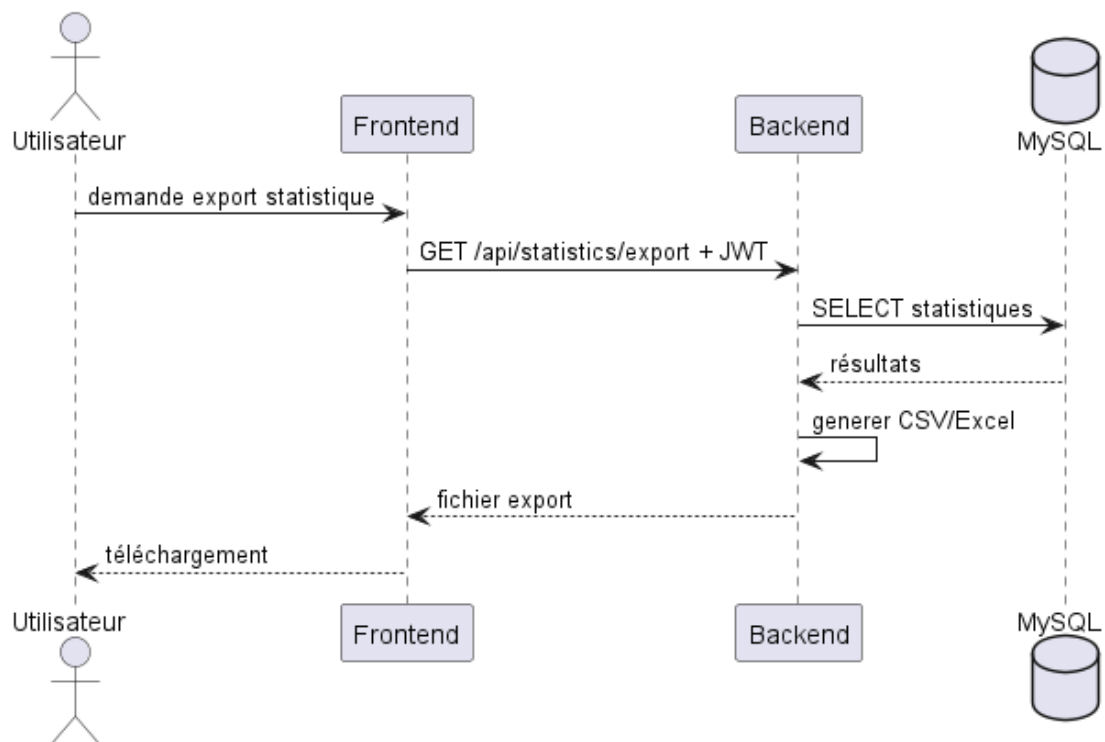


FIGURE 7.45 – Diagramme de séquence d'export des statistiques

Consultation des indicateurs du parc

Ce diagramme formalise la restitution des indicateurs clés du parc informatique. Lorsqu'un administrateur consulte les métriques via GET /api/metrics, le backend agrège les données critiques : taux d'équipement disponible/en panne/en réparation, nombre de tickets ouverts, priorité moyenne des tickets, maintenances planifiées et performance des interventions (délai moyen de clôture). Ces calculs s'effectuent dynamiquement sans table Statistique persistante, garantissant que les indicateurs reflètent toujours l'état actuel du système.

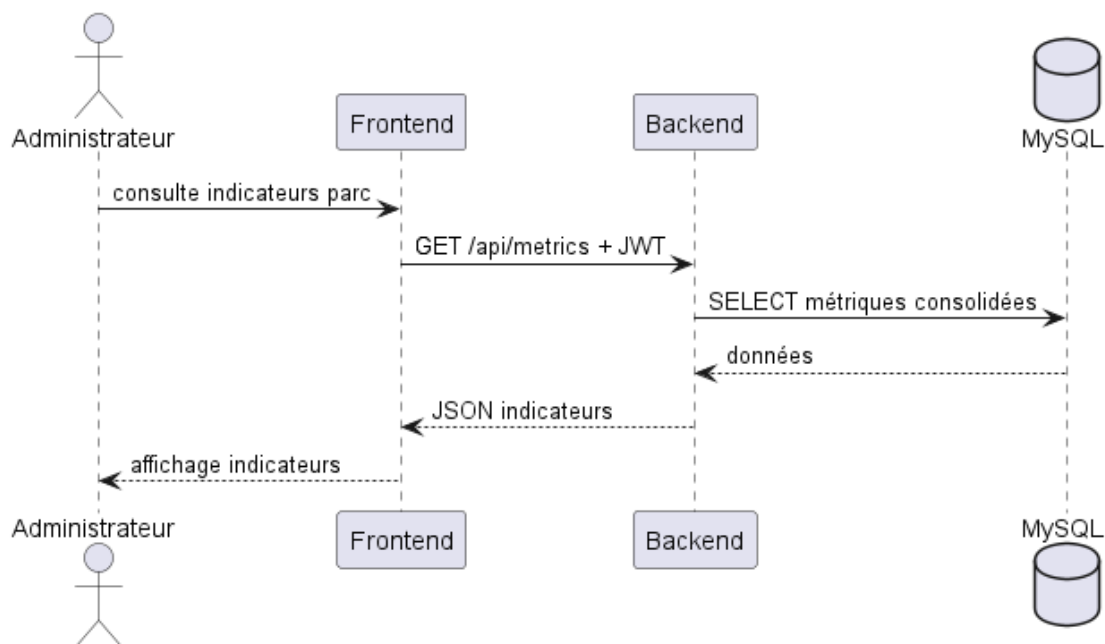


FIGURE 7.46 – Diagramme de séquence de consultation des indicateurs du parc

7.3. Tableaux de bord et reporting Web

L'interface web du Sprint 3 fournit un tableau de bord centralisé regroupant les principaux indicateurs de santé du parc, les tickets critiques, les maintenances planifiées, les équipements hors service et les taux de résolution. Le dashboard est conçu pour être consulté par l'administrateur et le responsable technique, avec des filtres sur les périodes, les catégories d'équipements et les priorités. Les indicateurs sont calculés en temps réel lors de chaque consultation, ce qui garantit des données toujours cohérentes avec l'état de la base de données.

7.4. Optimisation des performances

Le Sprint 3 a porté une attention particulière à l'optimisation des performances du backend et du frontend. Les actions principales ont consisté à indexer les champs fréquemment interrogés de MySQL, à limiter la taille des réponses JSON via une pagination efficace, et à optimiser les calculs des statistiques et des indicateurs de dashboard pour réduire la latence des requêtes. Le frontend bénéficie également d'optimisations React natives (virtualisation des listes, memoization des composants, lazy-loading des pages) pour améliorer la réactivité de l'interface utilisateur.

FIGURE 7.47 – Page d'accueil

FIGURE 7.48 – Connexion

FIGURE 7.49 – Accueil

FIGURE 7.50 – Liste des équipements

FIGURE 7.51 – Détails de l'équipement

FIGURE 7.52 – Notifications d'intervention

FIGURE 7.53 – Créer une intervention

FIGURE 7.54 – Détails du ticket

FIGURE 7.55 – Historique de maintenance

FIGURE 7.56 – Profil utilisateur

FIGURE 7.57 – Historique des notifications

FIGURE 7.58 – Calendrier de maintenance préventive

FIGURE 7.59 – Ajouter un rapport de maintenance

FIGURE 7.60 – Activer les notifications

FIGURE 7.61 – Liste détaillée des équipements

Conclusion

Le Sprint 3 conclut le développement du système GMAO, livrant une solution complète et fonctionnelle, prête pour un déploiement en conditions réelles. Les optimisations apportées aux performances, l'amélioration des interfaces utilisateur et la validation par le personnel de maintenance garantissent que le système répond aux exigences initiales d'automatisation de la gestion de maintenance et de réactivité opérationnelle. La documentation finalisée et l'infrastructure stabilisée facilitent une transition vers une phase de mise en œuvre effective, marquant une étape clé dans la modernisation de la gestion technique du parc informatique et des équipements.

Conclusion et perspectives

Le projet *GMAO* a permis de concevoir et de développer un prototype innovant pour la gestion centralisée de la maintenance informatique et des équipements, répondant aux besoins spécifiques des organisations tunisiennes. Grâce à une approche agile (Scrum) et à l'intégration de technologies telles que l'automatisation des workflows, les codes QR pour le suivi des équipements et les notifications Firebase FCM, le système a démontré sa capacité à automatiser la gestion des demandes de maintenance, à générer des alertes prioritaires et à s'adapter à différents types d'équipements.

Les résultats obtenus montrent que *GMAO* améliore la réactivité du personnel de maintenance, facilite le suivi des équipements et renforce la disponibilité opérationnelle des ressources informatiques. L'architecture modulaire et la compatibilité multi-types d'équipements constituent des atouts majeurs pour une adoption dans des environnements organisationnels variés.

Pour aller plus loin, il sera nécessaire de réaliser des tests opérationnels en conditions réelles, d'intégrer le système avec les outils de gestion d'actifs (CMDB) et d'explorer l'utilisation de l'intelligence artificielle pour prédire les défaillances d'équipements et optimiser les planifications de maintenance préventive.

En résumé, *GMAO* pose les bases d'une modernisation de la gestion technique du parc informatique et des équipements en Tunisie. Son évolution dépendra d'un engagement continu des acteurs informatiques, gestionnaires et techniques pour garantir une amélioration durable de l'efficacité opérationnelle et de la rentabilité des investissements en infrastructure.

Ce projet a été une expérience enrichissante, tant sur le plan technique que personnel. J'ai eu l'opportunité d'apprendre de nouvelles compétences, de collaborer avec des professionnels passionnés et de contribuer à une initiative qui vise à améliorer la gestion technique et la maintenance des équipements en Tunisie. J'espère sincèrement que *GMAO* sera bénéfique pour la communauté informatique et technique, et qu'il participera à l'optimisation de la gestion du parc informatique et à la réduction des temps d'arrêt non planifiés.

Bibliographie

- [1] Faculté de Médecine de Monastir. Rapport annuel 2020, 2020. Available : <http://www.fmm.rnu.tn/>.
- [2] International Organization for Standardization. Iso 21001 :2018 - educational organizations, 2018. Available : <https://www.iso.org/standard/69596.html>.
- [3] GLPI Project. Glpi : Gestionnaire libre de parc informatique, 2023. Available : <https://www.glpi-project.org/>.
- [4] Combodo. itop : It operations portal, 2023. Available : <https://combodo.com/>.
- [5] Asset Panda. Asset panda : Cloud-based it asset management, 2023. Available : <https://www.assetpanda.com/>.
- [6] Express.js Community. Express.js - fast, unopinionated web framework for node.js. <https://expressjs.com/>, 2024.
- [7] Meta Platforms, Inc. React - a javascript library for building user interfaces. <https://react.dev/>, 2024.
- [8] Oracle MySQL Community. Mysql - the world's most popular open source database. <https://www.mysql.com/>, 2024.
- [9] Google. Firebase - build apps, faster. <https://firebase.google.com/>, 2024.
- [10] MUI (Material-UI) Team. Material-ui - react components library. <https://mui.com/>, 2024.
- [11] Atlassian. What is scrum ? <https://www.atlassian.com/agile/scrum>, 2023.
- [12] Atlassian. Scrum artifacts. <https://www.atlassian.com/agile/scrum/artifacts>, 2023.
- [13] Ken Schwaber and Jeff Sutherland. *Scrum : The art of doing twice the work in half the time*. Crown Business, 2004.
- [14] Atlassian. Scrum roles. <https://www.atlassian.com/agile/scrum/roles>, 2023.
- [15] ClickUp. Clickup docs : Product management and agile tools. <https://docs.clickup.com/en/articles/2095287-getting-started-with-clickup>, 2025.
- [16] Slack Technologies. Slack documentation. <https://slack.com/help/categories/200111606>, 2025.
- [17] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.
- [18] Sequelize Contributors. Sequelize - promise-based node.js orm. <https://sequelize.org/>, 2024.
- [19] Evan You and Vite Contributors. Vite - next generation frontend tooling. <https://vitejs.dev/>, 2024.